

MANUAL TESTING

CONTENTS

1. SDLC– Software Development Life Cycle

a. Waterfall Model

b. Spiral Model

c. V Model

d. Prototype Model

e. Agile Model

2. Software Testing

a. White Box Testing (WBT)

b. Black Box Testing (BBT)

3. Defect Life Cycle

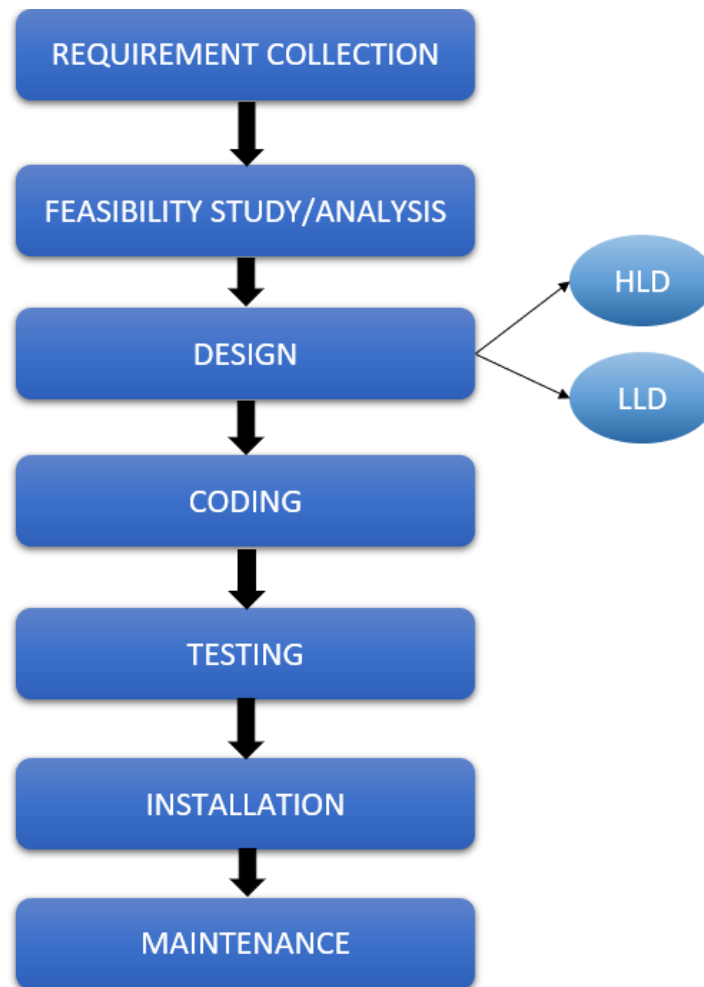
4. Test Plan

5. Test Case

6. STLC – Software Test Life Cycle

SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

- SDLC is a procedure to develop a software.
- There are 7 stages in SDLC:



When do we go for SDLC?

Whenever we want to develop a new software, we follow SDLC

What happens if you don't follow SDLC?

- i. We will not get to know how much money should be invested
- ii. We will not get to know how many resources are needed
- iii. We will not get detailed requirement document
- iv. There might be delay in releasing the software

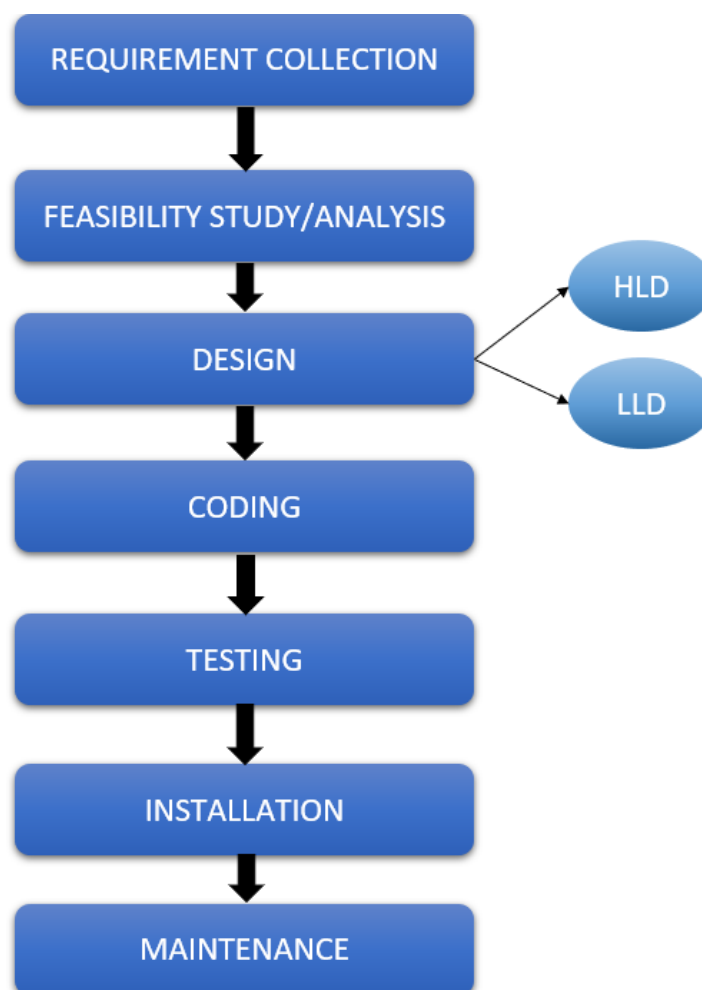
Models in SDLC

Some of the models in SDLC are listed below:

1. Waterfall Model
2. Spiral Model
3. V Model
4. Prototype Model
5. Agile Model

I. WATERFALL MODEL

- It is a traditional model
- There are 7 stages in Waterfall model:



1. Requirement Collection

- Here the Business Analyst/Product Analyst collects the requirement from the customer.
- Requirement is collected in the form of CRS. CRS is converted to SRS – meaning business language is converted to software language by BA/PA.

CRS — Customer Requirement Specification

It is the requirement document in the form of customer business language

SRS — Software Requirement Specification

It is the requirement document in the form of software language

BA — Business Analyst

BA converts the CRS into SRS in service-based company

PA — Product Analyst

PA converts the CRS into SRS in product-based company

Service based companies

They provide service and develop software for other companies according to their requirement.

Ex: Wipro, Infosys, TCS, Cognizant

Product based companies

They develop their own software and sell it to other companies which may need the software and earn profit.

Ex: Oracle, Microsoft, Google

2. Feasibility Study/ Analysis

* It is done by a team consisting of Project Manager, BA, Architect, Finance team and HR team.

* Here we check:

i. Technical Feasibility: We check technology is available or not to develop the software. It is done by Architects.

ii. Financial Feasibility: We check budget is available or not to develop the

software. It is done by Finance team.

iii. Resource Feasibility: We check resources are available or not to develop the software. It is done by HR team.

* Architect will do technical feasibility study and tell which technology to use in order to develop the software. Any Sr Developer can become an Architect.

3. Design

Here we do High Level Design (HLD) and Low-Level Design (LLD) of the software

HLD — High Level Design

- > It is the architecture of the software product to be developed
- > It is done by Architects and Sr Developers

LLD — Low Level Design

- > It describes how each and every feature in the product should work and how each and every component should work
- > It is done by Sr Developers

4. Coding

- * Here we start building the software/ writing the code for the product
- * It is done by Sr Developers, Jr Developers and Freshers

5. Testing

- * After coding we start testing wherein, we identify defects in the software
- * It is done by test engineers

Drawbacks of Developers involved in testing the software

1. They will always see the product from positive point of view, they will never see the product from negative point of view
2. They will always have over confidence that whatever they have built, works
3. They consume the time allocated for testing to build the product and they say time is not there to test
4. Even though they know that bugs are there, they might hide it and not fix it

6. Installation

- * After the software is developed and tested, it is installed in the customer's place for using
- * It is done by Installation Engineers

7. Maintenance

After the software is installed and used, if the customer finds any defect, the software company will fix it according to the agreement i.e., if the defect is found in the Maintenance period, then the software company will fix the defect free of cost.

Advantages

- i. Requirement/ Design doesn't change, so we get a stable product
- ii. Quality of software is good

Disadvantages

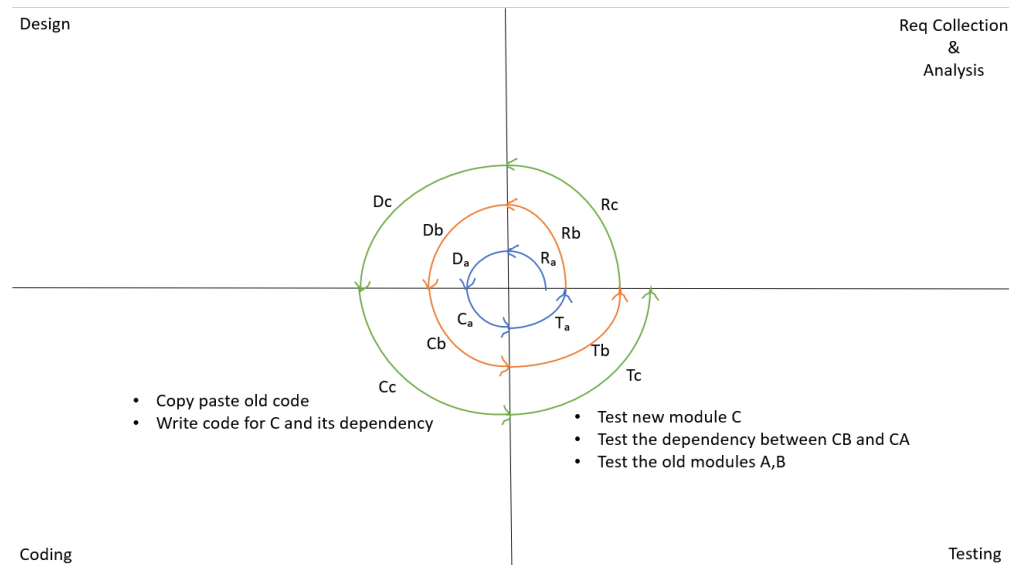
- i. Backtracking is not possible. For example, we can't change the requirement once the design stage is completed means the requirement is freezed and hence this model is not flexible.
- ii. Requirement is not tested, design is not tested, if there is bug in the requirement, it will flow till the end and leads to lot of rework.
- iii. It is traditional model where developers were involved in testing.

Applications/ When to go for Waterfall model?

1. Whenever we are building simple/ small applications.
 2. Whenever we go for short term projects
 3. Whenever we are sure that requirement is not going to change
- Ex: Calculator, Calendar

II. SPIRAL MODEL

- In order to overcome the drawbacks of Waterfall model, we go for spiral model.
- Whenever we have dependency, we go for spiral model



Advantages

1. Requirement changes can be done after every cycle
2. Customer will get the opportunity to see the s/w after every cycle
3. Testing is done in each and every cycle before going to the next cycle

Disadvantages

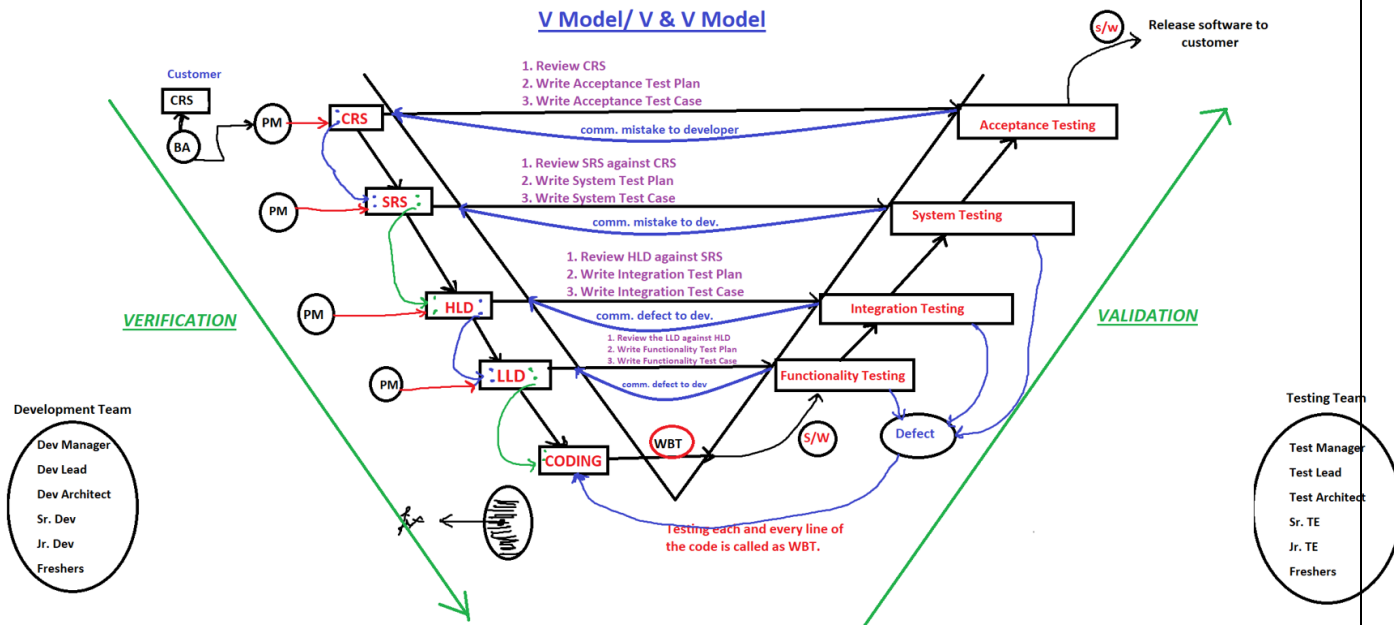
1. Requirement changes are not allowed in between the cycle
2. Req. collection and design are not tested
3. In this model also developers were involved in testing
4. Same process repeats for each module

Applications/ When to go for Spiral Model?

1. We go for Spiral model when there is dependency between the modules.
2. We go for Spiral model whenever customer gives the requirement in stages

III. V MODEL

Here both development team and testing team are present. Parallely both developers and test engineers are involved in verification process. Hence verification is done to prevent the defects. Once after s/w is ready, validation is done in order to identify and fix the defects.



Verification

The process of reviewing requirements (CRS & SRS), design (HLD & LLD), code, test cases and other related documents is called as Verification.

Validation

It is the actual testing done after the software is developed. Here we execute the test cases.

Advantages

1. Testing starts from early stage of product development (i.e., at requirement collection stage itself)
2. All the stages are tested (avoids downward flow of defects)
3. Work happens simultaneously because of that total time taken to complete then project is less
4. Total investment will be less

Disadvantages

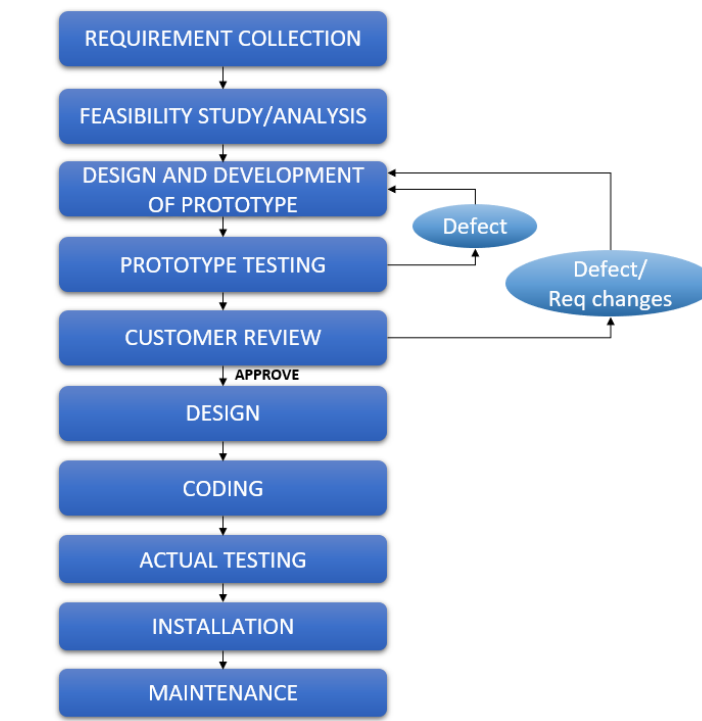
1. Documentation work is more.
2. Initial investment is more.

Applications/ When to go for V model?

1. Whenever we are building a very complex product
2. Whenever we go for long term projects
3. Whenever the customer is expecting very high- quality product with stipulated time

IV. PROTOTYPE MODEL

Prototype model is a dummy model prepared by the Web Designers/ Content Developers wherein they convert "text format" into "image format" by using tools like Adobe Photoshop, Paint, Picasa or Web html



What is the difference b/w Prototype Testing and Actual Testing?

Prototype Testing: Testing done to check whether customer given requirements is present in the prototype/ dummy page. (OR) Testing done to check whether all the text format is converted into image format.

Actual Testing: Testing the functionality of an application by entering all possible inputs into the software

Advantages

1. In the beginning only customer will be knowing what he going to get
2. If any changes are needed, customer gets an opportunity to ask for the changes in the beginning itself
3. Improved communication b/w customer and development team

Disadvantages

1. There is an investment needed just to improve the communication to build the prototypes
2. There will be lot of delay in starting the actual product development

Applications/When to go for Prototype model?

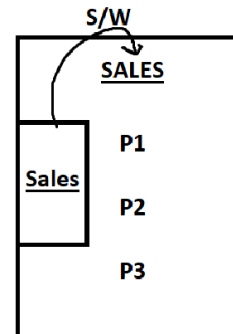
1. When customer is not clear about how to explain the requirements
2. Whenever customer is new to software/ when developers are new to domain

SOFTWARE TESTING

The process of finding defects in the software is known as Software Testing
(OR)

Verifying the functionality of an application against the requirement specification is called as Software Testing

Sales link: When clicked sales page should be displayed.
P1, P2, P3 should be displayed



Why do we do Software Testing?

1. Every software is developed to support business, if there is a defect in the software it affects the business, so, before we use the s/w for business it should be tested. All the problems must be recognized and solved
2. To check if the software is developed according to the customer's requirement
3. To improve the quality of the product

TYPES OF SOFTWARE TESTING

White Box Testing (Developers)

or

Unit Testing

or

Open Box Testing

or

Glass Box Testing

or

Transparent Testing

or

Structural Testing

or

Mutation Testing

GREY BOX TESTING

(WBT+BBT)

Black Box Testing (Test Engineers)

or

Functionality Testing

or

Behavioural Testing

or

Closed Box Testing

WHITE BOX TESTING (WBT)

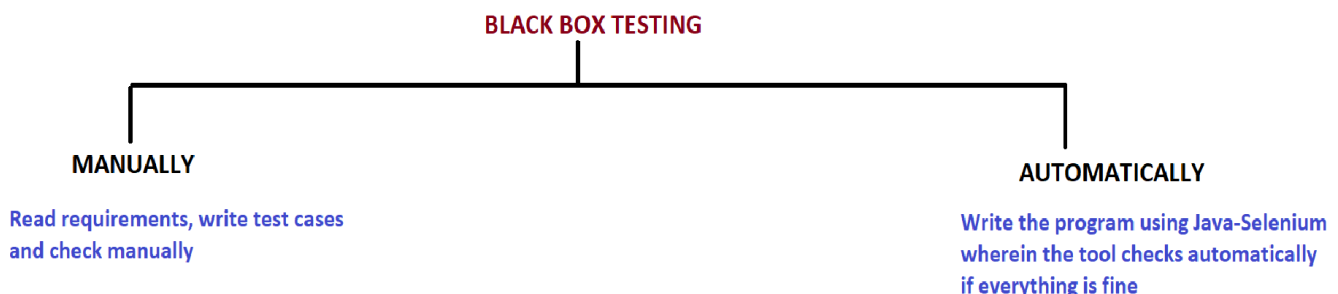
- Testing each and every line of code is called as White Box Testing
- WBT is done by Developers
- Open box testing: Because the developers are able to see the code it is called open box testing
- Unit Testing: Smallest unit of a s/w is one line of code, since you're testing every line it is called unit testing

BLACK BOX TESTING (BBT)

- Verifying the functionality of an application against the requirement specification is called as Black Box Testing
- BBT is done by Test Engineers
- Source code is not visible to Test Engineers

Why test engineers should not be involved in fixing the bug?

1. If we spend time on fixing one bug, we will not get time to catch some other bugs
2. Chances are there fixing 1 bug might introduce lot of bugs



MANUAL TESTING

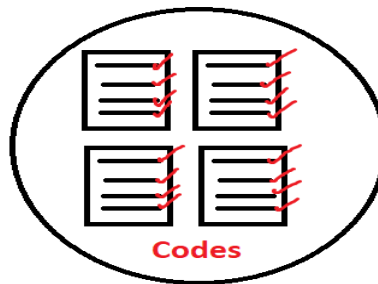
Testing the software/ application repeatedly or again and again manually in order to find the defects in the software according to the customer requirement is called as Manual Testing

AUTOMATION TESTING

Test Engineer write code/program/ script by using tools like Selenium/ QTP/ Appium and run the program against the application/ software wherein the tool will automatically test the application and give the result as Pass/Fail. This is called as Automation Testing.

WHITE BOX TESTING

Testing each and every line of code is called White Box Testing

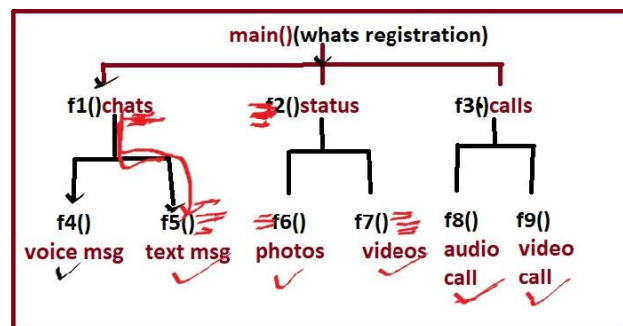
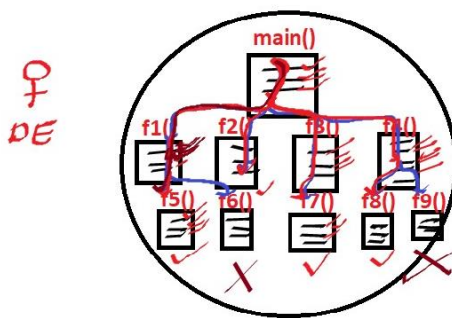


Types of White Box Testing:

- 1) Path testing
- 2) Condition testing
- 3) Loop testing
- 4) White box testing from memory point of view
- 5) White box testing from performance point of view

i) Path testing:

Here developer will write the flowchart/ flow graph & test all the individual path.

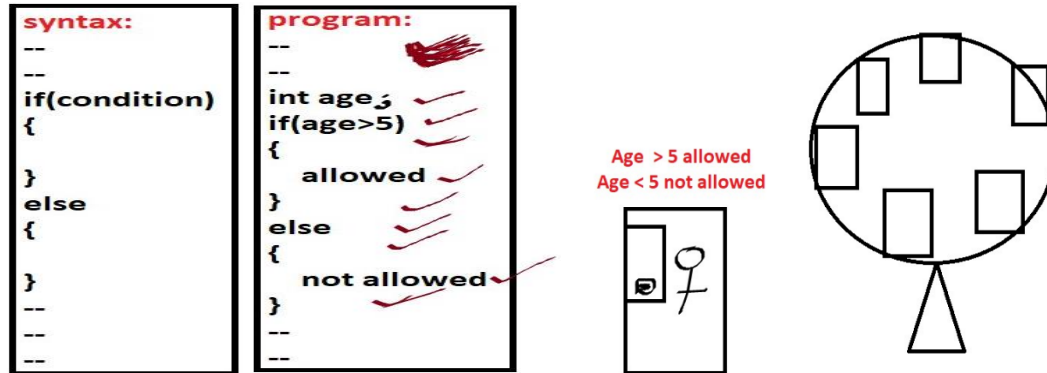


Advantages of flow graph:

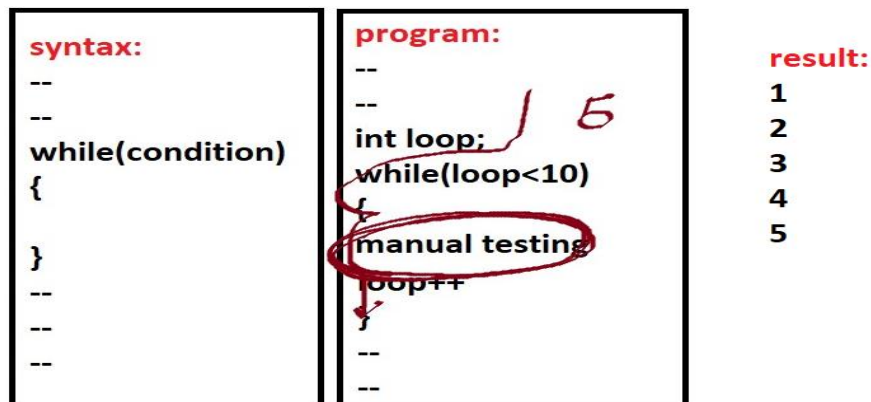
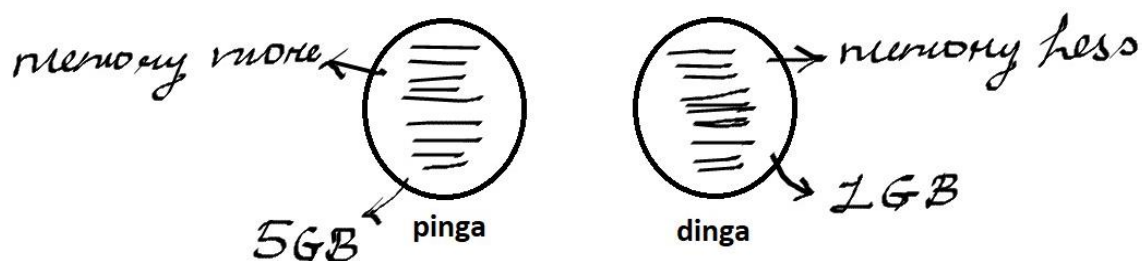
1. We will not miss any path
2. We will not repeat any path

ii) Condition testing:

Here the developers will test for all the logical condition i.e., for both true & false values.

**iii) Loop testing:**

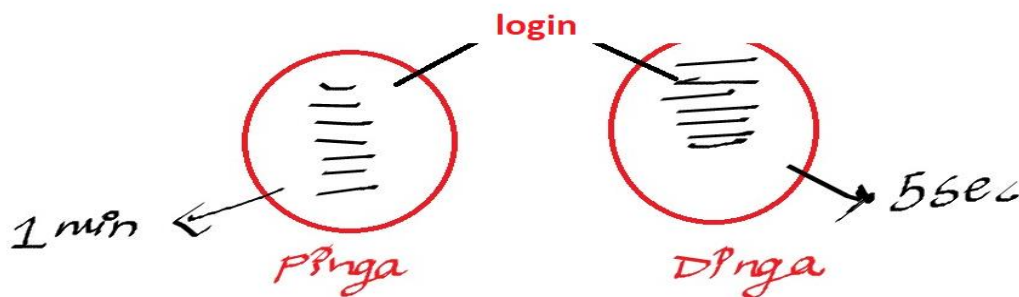
Here the developers will test the loop and ensure that the loop is repeating for all the defined number of cycles

**iv) White box testing from memory point of view:**

Typical mistakes done by developers because of which size of the code increases: -

- Because of repetition of same code instead of using functions
- Because of logical mistakes
- Because of not using inbuilt functions
- Because of unused variables & functions

v) White box testing from performance point of view:



Typical mistakes done by developers because of which program takes more time to run: -

- Because of not using better logic
- Using proper AND & OR operators

Difference between WBT and BBT

WBT

- Testing each and every line of code is called WBT
- It is done by developers
- To do this one should have very good knowledge of programming
- To do this one should have very good knowledge of programming
- Give the input and check and run every code of line and gives output

BBT

- Verifying the functionality of the application against the customer requirement specification
- It is done by test engineers
- No need to have knowledge of programming
- No need to have knowledge of internal design of code
- Give the input, don't check code and gives output according to requirement

BLACK BOX TESTING

- Verifying the functionality of an application according to requirement specification is called as Black Box Testing
- It is done by test engineers

Types of Black Box Testing:

1. Functionality Testing
2. Integration Testing
3. System Testing
4. Acceptance Testing
5. Smoke Testing
6. Adhoc Testing
7. Compatibility Testing
8. Exploratory Testing
9. Usability Testing
10. Globalization Testing
11. Performance Testing
12. Regression testing

FUNCTIONALITY TESTING

Testing each and every component thoroughly against requirement specification is called Functionality Testing.

Add user link:

1. when user clicked on add users link , add user page should be display.
2. user name text field: it should accept 5-32 char
3. pwd text field: it should accept min 6 char & include atleast 1 special char
4. designation dropdown: it is mandatory
5. emailid text field: it should accept valid mail id format
6. mobile no: it should accept 10 digit no
7. gender: it is mandatory
8. D.O.B text field: it should accept valid d.o.b
9. submit button: when clicked user should be created
10. cancel button: it should not create user

Software

Add user	Add user page
UN	
PWD	
Designation	Lead ▼ Manager TE
email id	
mobile no	
Gender	Male ☐ Female ☐
D.O.B	
Submit	Cancel

Components:

Link, text field, text area, drop down, button, widget, bento, hamburger, icon, radio button, kebab, ellipses, etc.

Thoroughly:

Testing the component by entering all the possible input (positive and negative) is called as thoroughly.

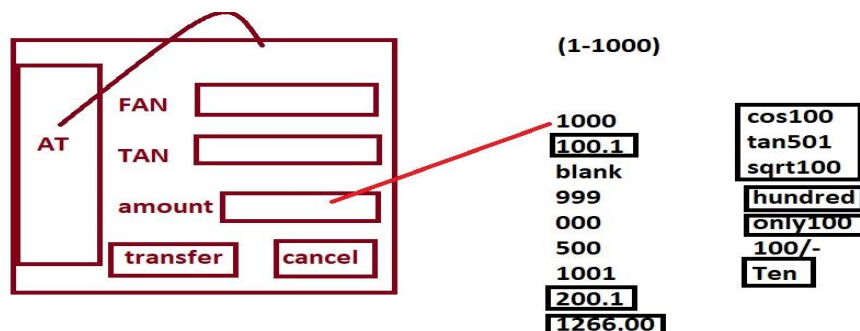
Why every requirement should be numbered?

- It is very easy to understand the requirement
- There will be clarity in the requirement
- Requirement becomes traceable
- Requirement becomes measurable
- It becomes very easy to communicate between developer, testing team & customer

We can do any type of testing in 3 different ways

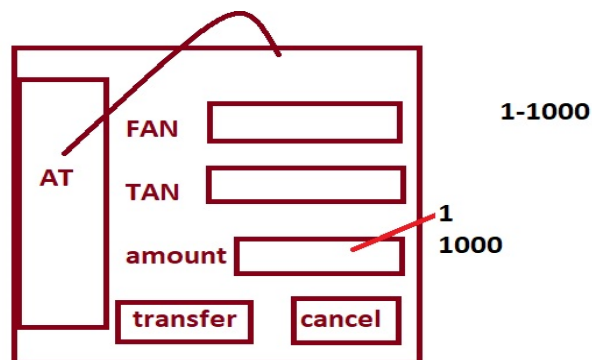
1. Over testing
2. Under testing
3. Optimized testing

- i. Over Testing: Testing the application with the same scenarios in different ways (OR) Testing the application with those scenarios which doesn't make sense is called as over testing



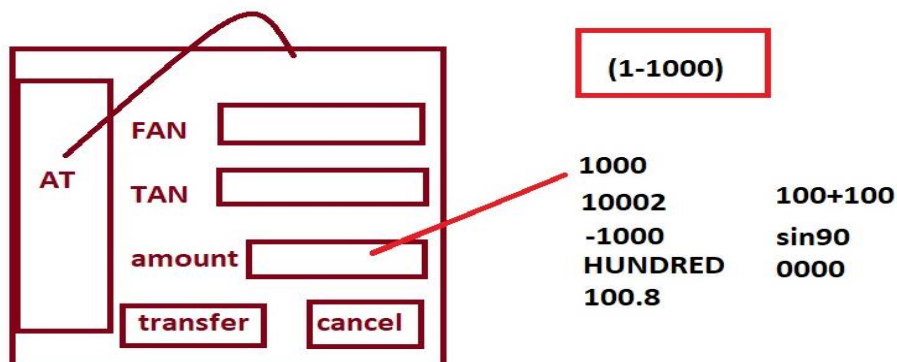
❖ By doing over testing we will waste lot of time

- ii. Under testing: Testing the application with insufficient set of scenarios is called as under testing



❖ By doing under testing we will miss lot of defect

- iii. Optimized testing: Testing the application only with those scenarios which make sense is called as optimized testing



❖ If we do this, we are going to find maximum bugs in optimum time.

Positive testing:

Testing each and every component of an application by entering valid or expected data which is according to the requirement specification is called as positive testing.

Negative testing:

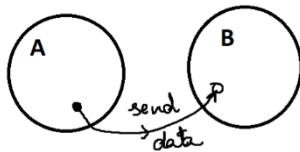
Testing each and every component of an application by entering invalid or unexpected data which is according to the requirement specification is called as negative testing

Rules/ Principles of Functionality Testing

- Always start testing the application with valid data. If the application works for valid data, then only test for invalid data
- If application is not working for the one of the invalid values you can continue for testing some more invalid value
- Test engineer should not assume (or) purpose the requirement. If you have any queries/ question you should interact with developer, customer(or) business analyst and get it clarified.

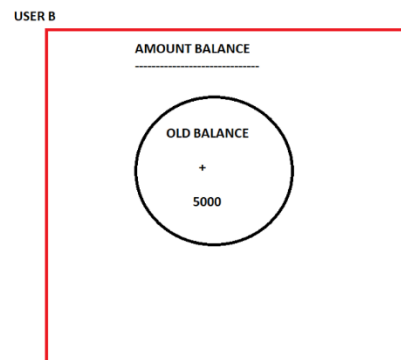
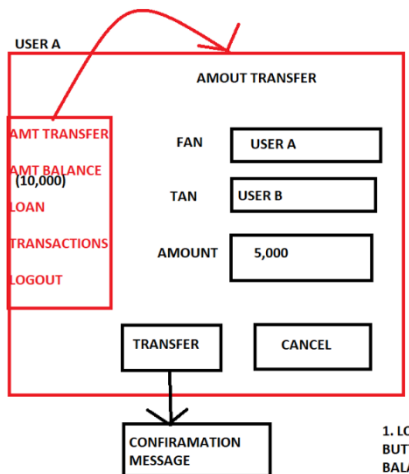
INTEGRATION TESTING

Testing the data flow (or) interface between two modules is called as integration testing.



IF A IS SENDING AND B IS RECEIVING, THEN INTEGRATION TESTING BETWEEN A AND B IS PASS

Ex:



1. LOGIN AS USER A, CLICK ON AMOUNT TRANSFER, ENTER ALL VALID DETAILS AND CLICK ON TRANSFER BUTTON, LOGOUT AS USER A. LOGIN AS USER B, CLICK ON AMOUNT BALANCE AND CHECK PROPER BALANCE AMOUNT IS DISPLAYED.

2. LOGIN AS USER A, CLICK ON AMOUNT TRANSFER, ENTER VALID DETAILS AND CLICK ON TRANSFER BUTTON. CLICK ON AMOUNT BALANCE AND CHECK PROPER BALANCE AMOUNT IS DISPLAYED.

3. LOGIN AS USER A, CLICK ON AMOUNT TRANSFER, ENTER VALID DETAILS AND CLICK ON TRANSFER BUTTON. CLICK ON TRANSACTIONAL AND CHECK PROPER TRANSACTION HISTORY IS DISPLAYED.

How to do Integration Testing?

1. Understanding the application is very important
 - a. You should understand each & every module in depth
 - b. You should also understand how all the modules are related or linked
2. Identify all possible scenarios
3. Prioritize the identified scenarios
4. Document the scenarios according to priority
5. Execute scenarios
6. If you find the defect send it to developers

Positive integration testing:

Amount transfer is less than or equal to balance, this testing is known as positive integration testing

Scenario:

Login as user 'A' check for amount balance, click on amount transfer, enter FAN as user 'A', TAN as user 'B' and enter amount as (Rs.1000). click on transfer confirmation message will be displayed, logout as user 'A'. login as user B, click on amount balance, there proper balance should be displayed (old balance+1000), logout as user 'B'

Negative integration testing:

Amount transferred is more than the balance

- If it transfers test is fail
- If it doesn't transfer negative integration is pass

Scenario:

Login as user 'A' check for amount balance, click on amount transfer, enter FAN as user 'A', TAN as user 'B' and enter amount as (Rs.15,000). click on transfer logout as user 'A'. login as user B, click on amount balance, there old balance should be displayed logout as user 'B'.

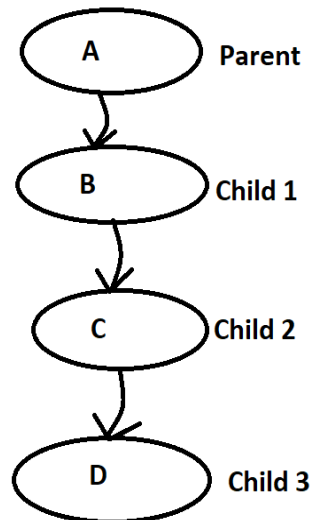
Type of Integration Testing:

1. Incremental integration testing:

Incrementally add the modules & test the dataflow between the modules.

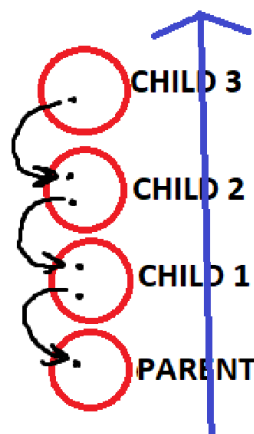
i. Top-down incremental integration testing:

Incrementally add the modules & test the dataflow between the module and make sure that the module we are adding is the child of the previous module.



ii. Bottom-up incremental integration testing:

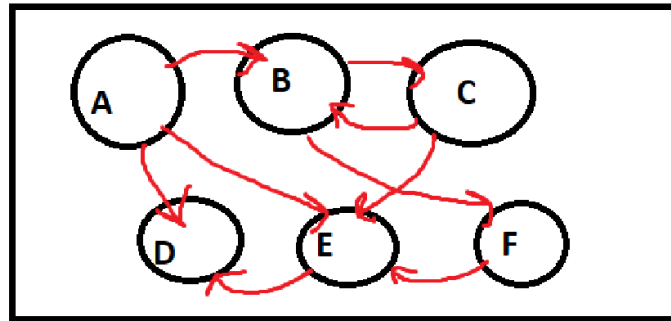
Incrementally add the modules & test the dataflow between the module and make sure that the module which is added should be the parent of the previous module.



Note: If there are n no. of modules, it is difficult to identify whether who is parent and child this is difficult in IIT, so we will go for NIIT.

2. Non-incremental integration testing:

Combining all the modules at once and testing the data flow between them is called as non-incremental integration testing.

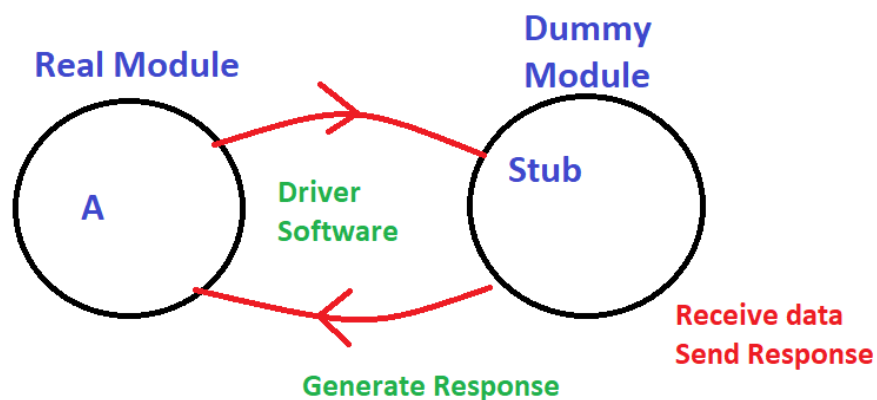


Drawback of non-incremental integration testing:

1. Chances is there we might miss some dataflow
2. Chances are there we might end up testing same thing again & again because of this time taken will be more
3. It is difficult to identify the root cause of the defect

Difference between stubs and driver?

If one module is ready & another module is not ready then how will you do integration testing?



Stubs: It is a dummy module, it acts like module which is not yet build, it generates the data & receives the data

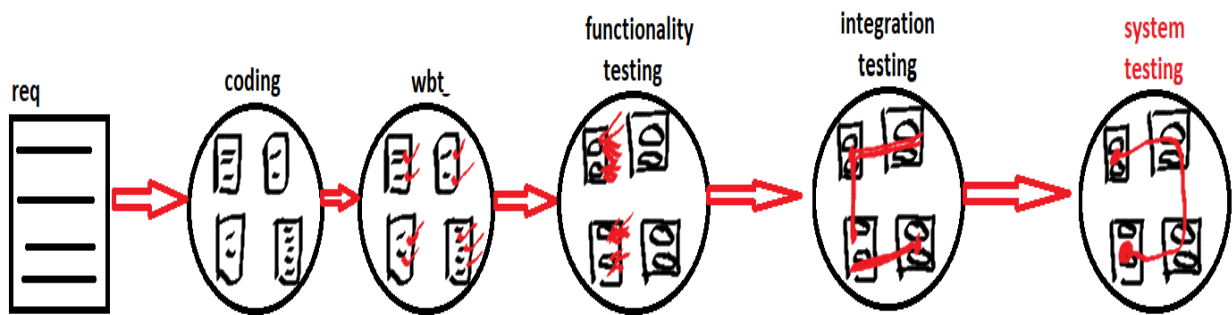
Driver: it is one which set up the testing environment and does lot of transaction, analyze the result & send the output (does the transaction between real module & stubs)

SYSTEM TESTING

It is an end to end testing where in the test environment is just similar to the production environment.

End-end testing:

Navigate through all the features and check whether the end feature is working as expected or not.



Ex: OD Flow

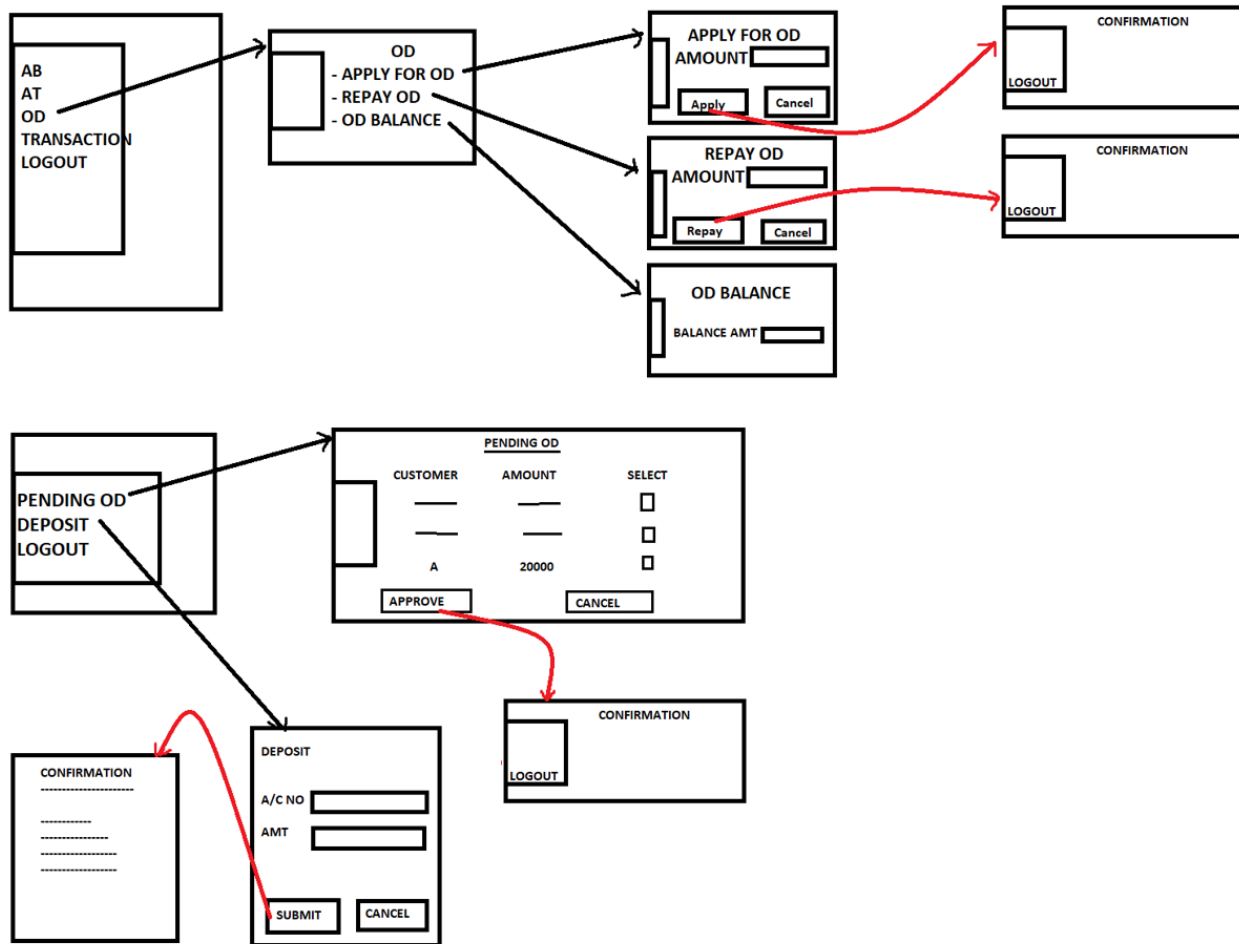
Scenario 1:

1. Login as user A, apply od for 20000, click on logout.
2. Login as manager, approve user A's od, click on logout.
3. Login as user A, check OD balance 20000, logout
4. Change the server date to next 30 days
5. Login as user A, check od balance need to be repaid 20650, logout
6. Login as manager, deposit Rs. 650, logout.
7. Login as user A, repay 20650
8. Check the od balance – 0000

Scenario 2:

1. Login as user A, apply od for 20000, click on logout
2. Login as manager, approve user A's od, click on logout

3. Login as user A, check od balance 20000, logout
4. Change the server date to next 30 days
5. Login as user a, check od balance need to be repaid 20400, logout
6. Login as manager, deposit Rs. 400, logout.
7. Login as user A, repay 20400
8. Check the od balance- 0000



Types of environment:

Development environment:

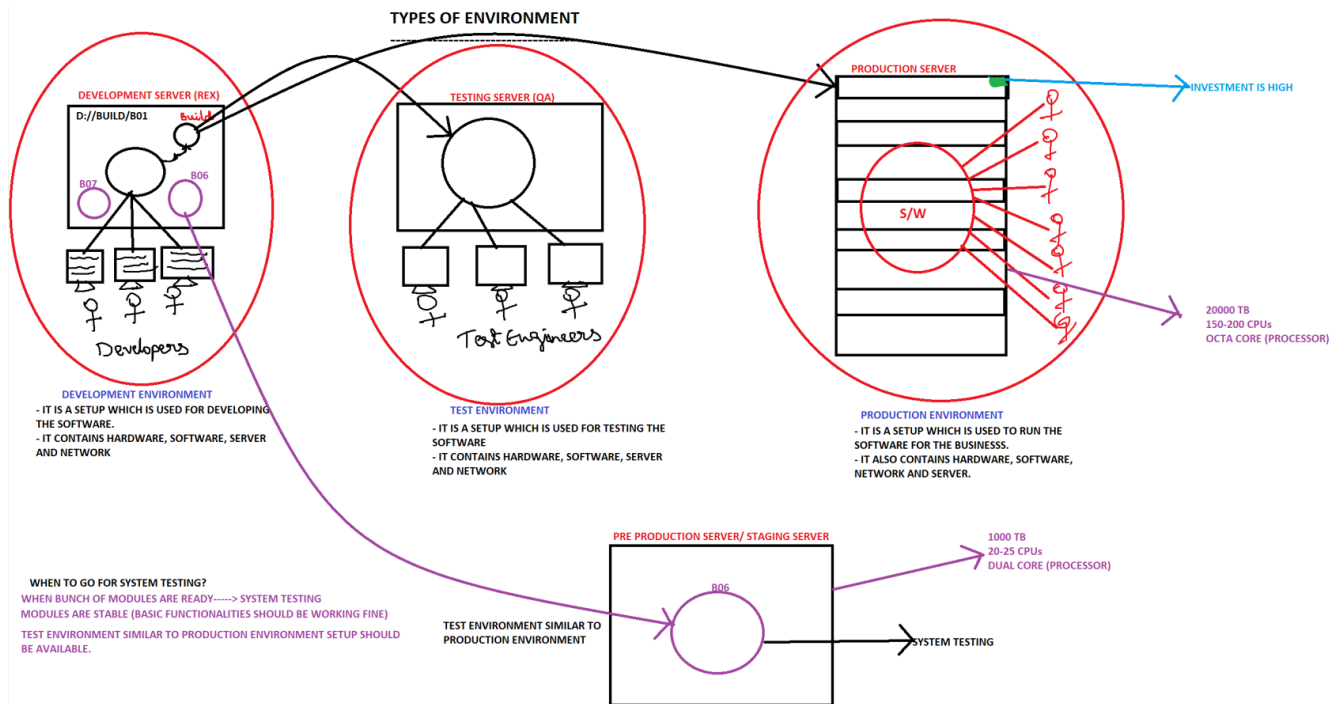
It is a setup which is used for developing the software. It consists of hardware, software, server, network.

Test environment:

It is a setup which is used for testing the software. It consists of hardware, software, server, network.

Production environment:

It is a setup which is used for run the software for business which contains hardware, software, server, network.

When new build comes:

- Test new module (probability of catching bugs is very high)
- Test integration between old and new module
- Retest fixed bugs
- Test the old modules

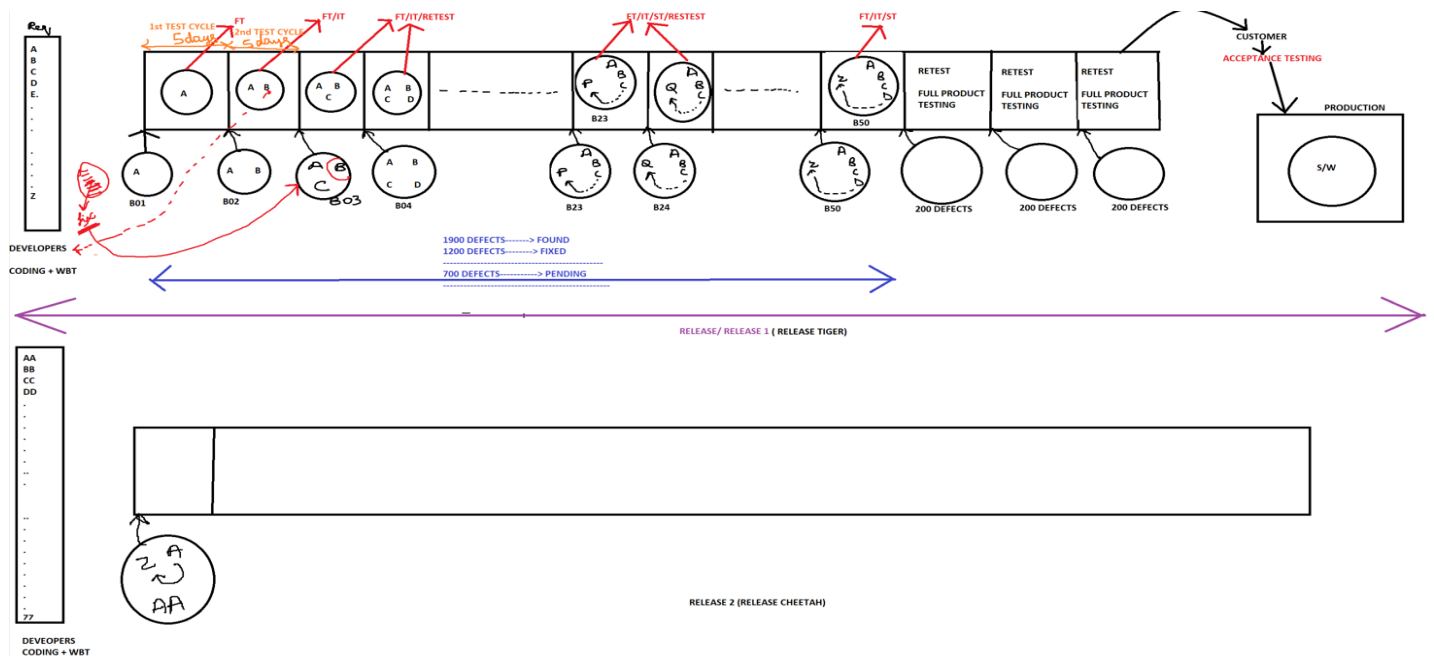
Why we find new bug in the old module?

- Chances are there adding new features would have introduced a bug in old module
- Fixing one bug would have introduced another bug
- Chances are there we would have missed it in previous cycle and found in current cycle

When we do System Testing?

- If the environment similar to the production environment is available
- When modules are stable (basic functionalities should be working fine)
- When minimum bunch of modules are ready

Release process:



What is Release?

Starting from collecting/gathering the requirement, developing the software and testing the software for many cycles and installing/ deploying the software to the customer is called as Release/ 1-Release.

Generally, 1- release might take 6 months – 1 ½ years. If it is Agile it might take 1 month.

When do you release the product to the production?

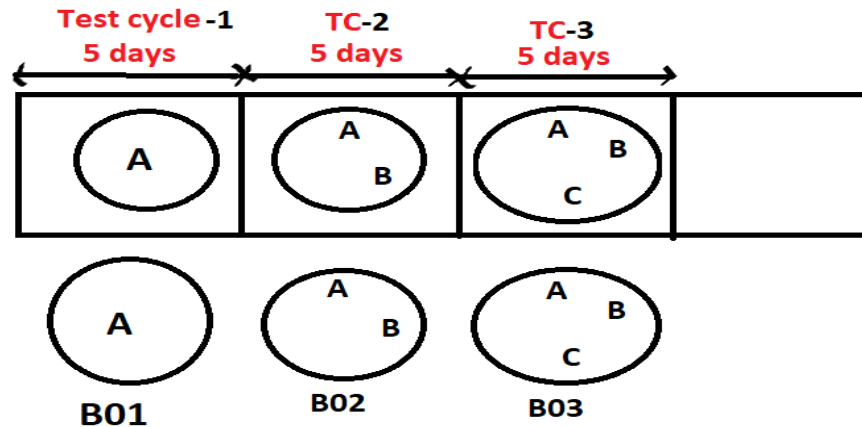
- When there is no blocker/ critical bug
- When all the end to end business scenarios are working fine
- When all the features requested by customer are ready
- If the product is tested in environment similar to production environment

What is Test Cycle?

It is the effort/ time/ duration spent by the test engineer to start and finish complete product testing is called as 1 test cycle **(or)** time taken to test one complete build.

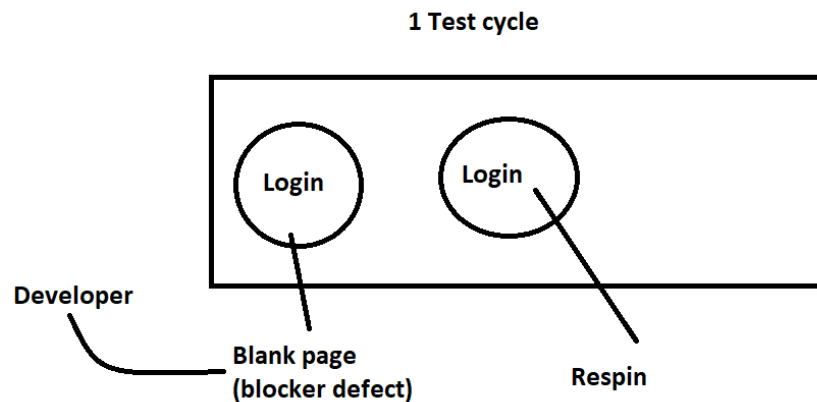
Each test cycle might take 3, 5 or 15 days. It just depends on:

1. No. of engineers
2. Complexity of the software
3. Size of the product



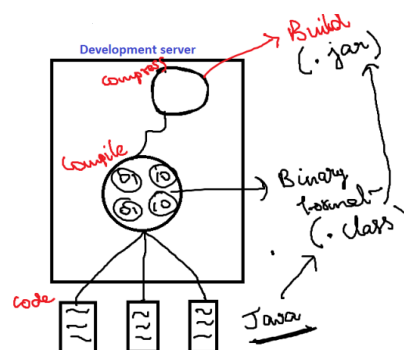
What is Respin?

- The process of getting a new build within one test cycle is called as Respin.
- If there are blocker defects, we will get respin.



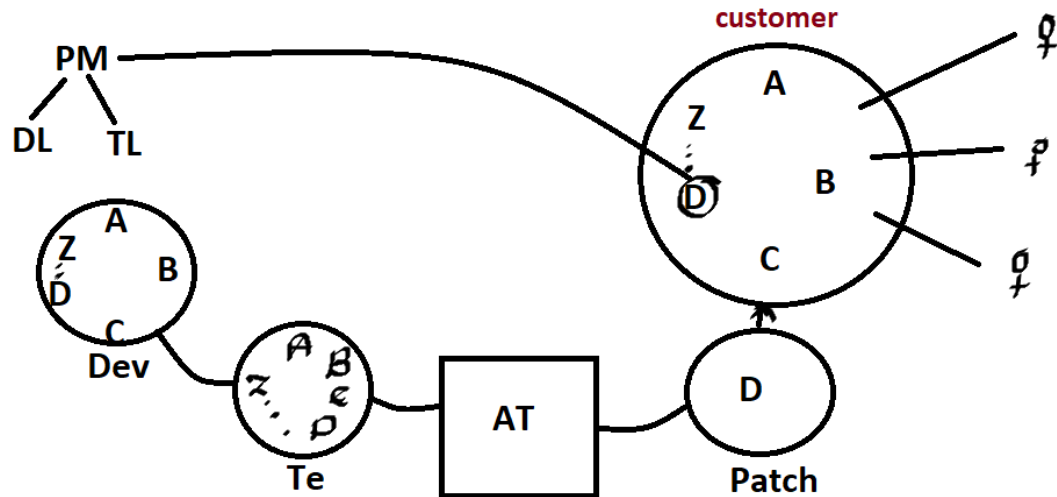
What is Build?

All the programs are compiled then we get binaries and all the binaries are archive/compressed and that is called as build.



What is Patch?

Patch is a small software which contains modified programs, new programs and record of deleted programs.



Who will be involved in the installation of software?

1. Anybody from the Testing team
2. Anybody from the Development team
3. Build engineer/ Release engineer

How the build mails come from development team: -

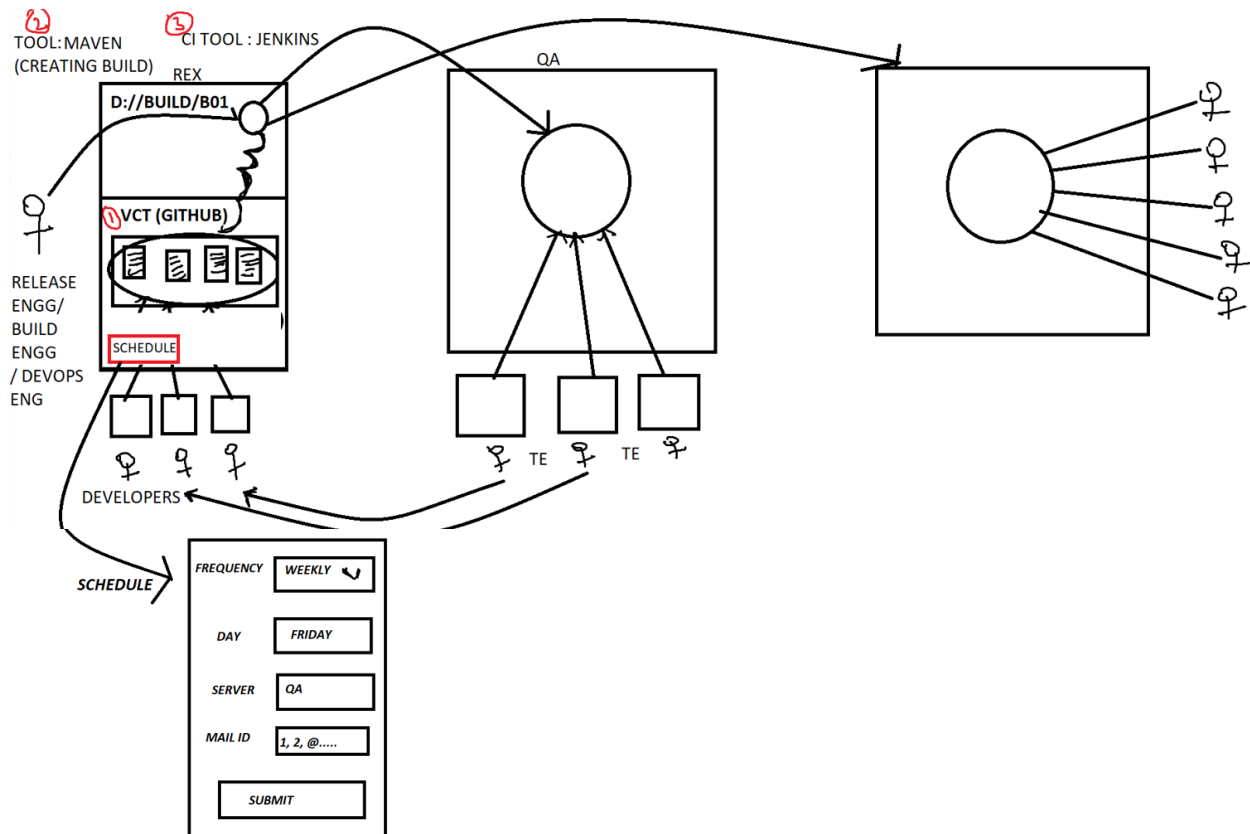
Hi Team,
 New build B01 is ready
 The test URL: <https://qa.citibank.com>
 UN: abcd
 PWD: xyz
 Happy testing!!!!

Role of Release Engineer

- Maintain the source code
- Create a build
- Install build in test environment
- Release the product to the production

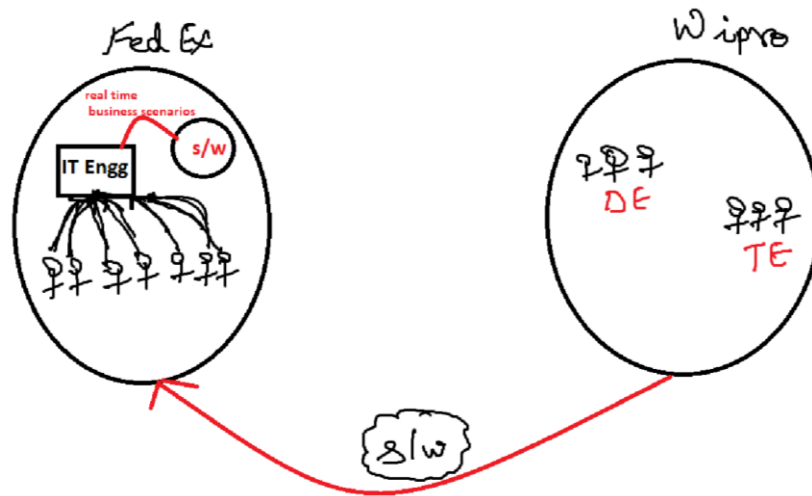
Continuous Integration (CI): Continuous synchronization between development server and testing server.

DevOps Engineer: Manages the code, compile, compress, install and release to the production. This activity is an operation. So, it is called development operations.



ACCEPTANCE TESTING

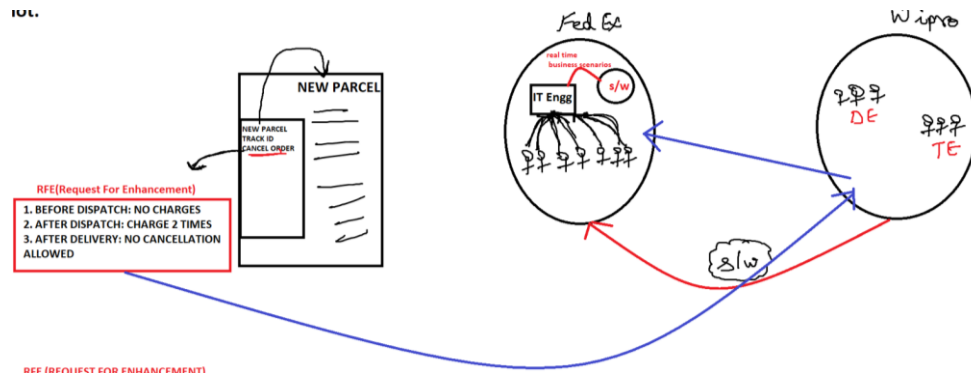
Approach 1: It is an end-to-end testing done by IT engineers, sitting at customer's place where in they take out the real time business scenarios & check whether software is capable of handling it or not.



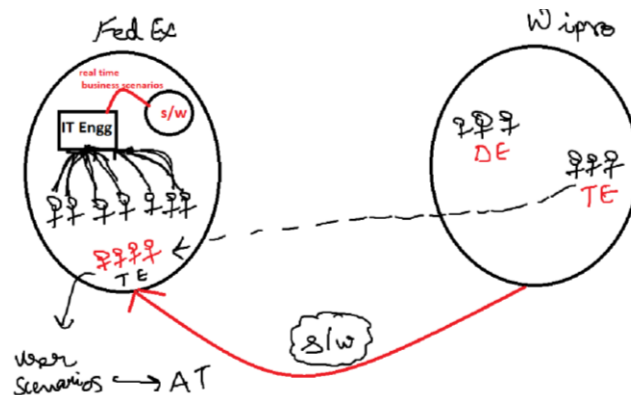
Why do we do Acceptance testing?

1. Chances are there under business pressure software company might push software to the customer with critical bugs, to prevent that they do Acceptance testing.
2. If they use software with the critical bugs for business they will undergo severe loss, to avoid that they do acceptance testing.
3. Chances are there development team would misunderstand the requirement & develop wrong features, to find such features customer will do Acceptance testing.

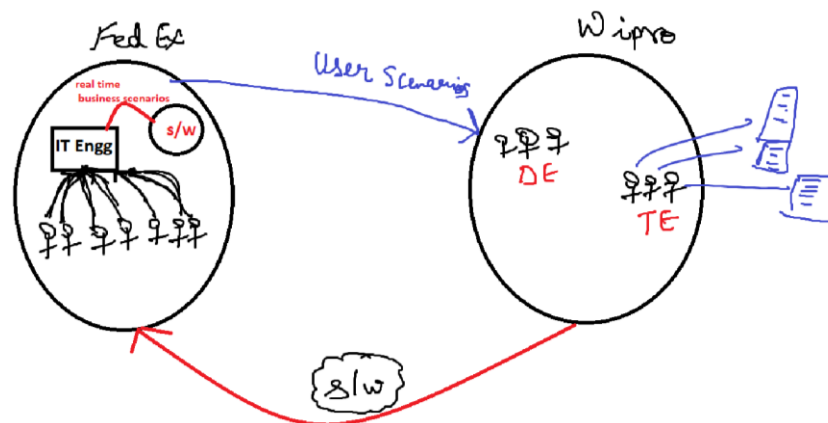
Approach 2: It is an end-to-end testing done by end-users wherein they use software for the business for a particular period of time and check whether software is capable of handling all real time business scenarios.



Approach 3: It is an end-to-end testing done by our own TE sitting at customer's place where in they refer user scenarios given by the customer and check whether software is capable of handling all real time business scenarios.

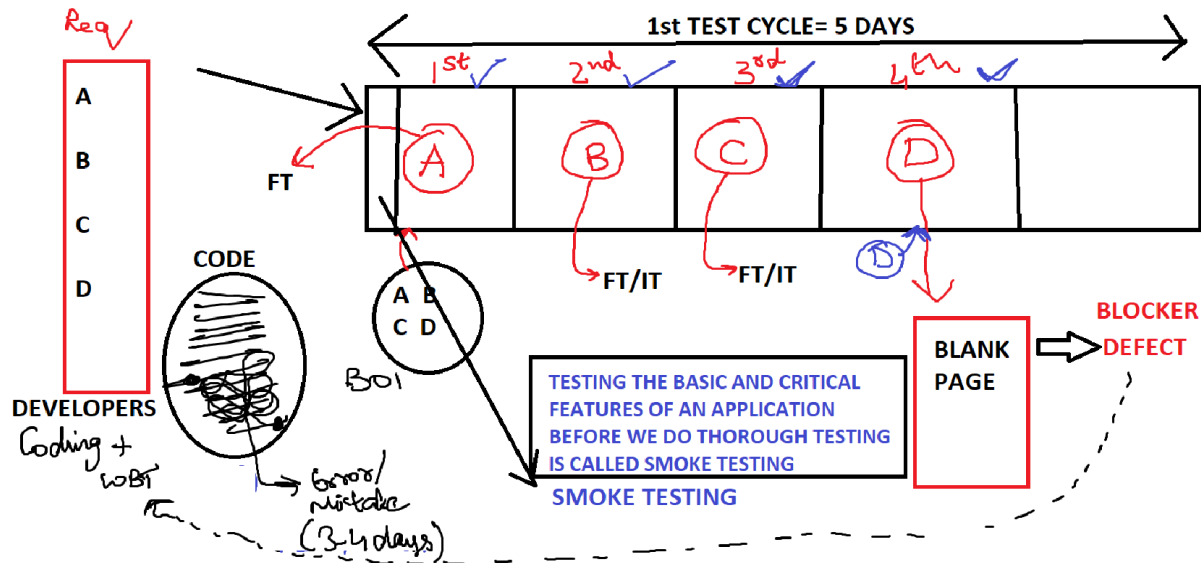


Approach 4: It is an end-to-end testing done by our own T.E sitting at our own place where in they refer user scenarios given by the customer and check whether software is capable of handling all real time business scenarios.



SMOKE TESTING

Testing the basic or critical features of an application before going for thorough testing is called as Smoke Testing.



Advantages of Smoke testing:

1. T.E can find all the blocker defects in the early stage itself.
2. Developers will get sufficient time to fix the defect.
3. Test cycle will not get postponed and the release will not be delayed.

How to do Smoke Testing?

LIST OF ALL THE FEATURES IN GMAIL APPLICATION

SIGNUP, LOGIN, COMPOSE, ALL MAILS, SENT ITEMS, STARRED, DRAFT, SPAM, SNOOZED, , TRASH, OUTBOX, SCHEDULED, CALENDER, IMPORTANT, FORWARD, REPLY ALL, SEARCH, SETTINGS, ARCHIVE, CONTACTS, PROMOTIONS, HELP, FEEDBACK, ATTACHMENT, LOGOUT

FEATURES TO BE TESTED AS A PART OF SMOKE TESTING	FEATURES THAT NOT BE TESTED AS A PART OF SMOKE TESTING
SIGNUP LOGIN COMPOSE ALL MAILS INBOX SENT ITEMS FORWARD REPLY ALL SEARCH LOGOUT SETTINGS	TRASH CHATS SPAM CALENDER DRAFTS HELP FEEDBACK OUTBOX ARCHIVE PROMOTIONS

1. LOGIN

- a) TO CHECK THAT WHEN USER CLICK ON LOGIN, LOGIN PAGE SHOULD BE DISPLAYED
- b) TO CHECK THAT WHEN USER ENTER VALID DATA INTO ALL THE FIELDS AND CLICK ON LOGIN BUTTON, HOME PAGE SHOULD BE DISPLAYED.

2. INBOX

- a) TO CHECK THAT WHEN USER CLICKS ON INBOX, INBOX SHOULD BE DISPLAYED.
- b) TO CHECK THAT WHEN USER CLICKS ON ANY ONE MAIL, MAIL SHOULD BE DISPLAYED AND CONTENT SHOULD BE DISPLAYED.

Note:

- Here we test only basic and critical features.
- We take every basic or critical feature and test for 1 or 2 important scenarios.
- Here we do only positive testing.
- In the beginning we will not be able to identify basic or critical features, we will learn it only after getting very good product knowledge.

When do we do smoke testing?

1. Whenever we get a new build from the development team we should always start with smoke testing because adding, modifying, removing the features or fixing the defects might affect basic or critical features, to find that in the beginning they do smoke testing.
2. Customer before he does acceptance testing he should also do smoke testing:
 - a) To check whether he has received the complete product or not.
 - b) To check whether the product is properly installed and configured.
3. One who installs the product in the server should do smoke testing to check whether the product is installed properly or not.
4. Before they give build to testing team, developers should do smoke testing so that too many defects are there means they need not give build to testing team.

Why do we do smoke testing?

1. To check whether the product is testable or not. (In the beginning if you find too many defects, it means product is not testable so better stop testing and spend all time in identifying some more scenarios)
2. Do smoke testing in the beginning itself if you find any blocker defect send it to the developer in the beginning itself so that developers will have sufficient time to fix the defect.

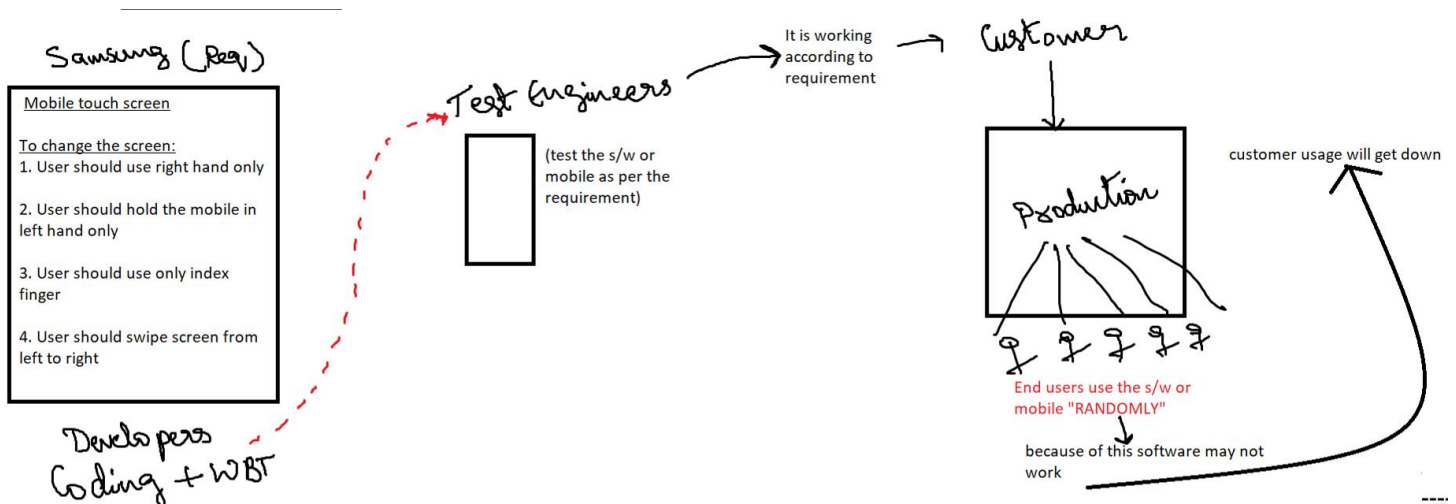
3. To check whether we have received broken build by the development team.
4. We do this to check that the product is installed properly or not.
5. It is like a health check of the product.

Difference between smoke testing & sanity testing

Smoke testing	Sanity testing
Smoke testing is a shallow and wide testing (shallow means high level testing, wide means cover all basic or critical features)	It is deep and narrow testing (here we take one feature and test it deeply)
Smoke testing is positive testing.	It is both positive & negative testing.
Here we document scenarios and test cases.	Here we don't document scenarios and test cases
Here we go for automation.	Here we don't go for automation.
Smoke testing is done by both developers and T. E	Sanity testing is done by only T. E

ADHOC TESTING

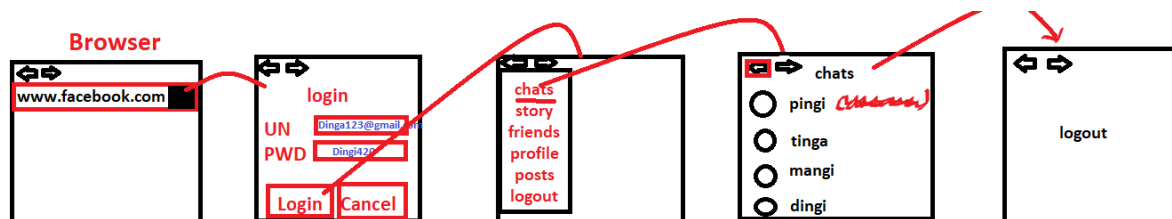
Testing the application randomly is called Adhoc Testing wherein we don't refer any kind of formal document like test cases or scenarios.



Why do we do Adhoc Testing?

1. Chances are there when the product is launched end users might use the application randomly and find defects, in order to avoid that test engineer should do Adhoc Testing.
2. If you see the requirement and test the application, no. of defects which you are going to catch will be less (so think out of box to catch more defects)
3. Since requirements are not followed by the end users, we should do Adhoc Testing. We do this to have better test coverage.
4. The intention of doing Adhoc testing is to somehow break the product.

How to do Adhoc testing?



Login as User, click on compose fill the details for all fields click on send, logout. Click on the browser back button and check whether the login page should be displayed or the application should ask to enter UN and PWD.

When do we do Adhoc testing?

1. When the product is functionally stable then we should think about Adhoc testing.
2. When we are doing smoke testing, we should not do negative testing/Adhoc testing. If you do this you will not be able to test basic or critical features.
3. Whenever the testing team is free they should spend time doing Adhoc testing.
4. Whenever we are doing FT/IT/ST, if we get some Adhoc scenarios we should pause our regular testing & do Adhoc testing, if we get too many scenarios we should document it & execute it when we get time (it is not a formal document like test case)

AGILE MODEL

- Agile is a model wherein we develop the software in an incremental and iterative process.
- They came up with this model in order to overcome the drawbacks that were there in the traditional model. Here we build large products in shorter cycles called Sprint.

Scrum:

It is the process used to build an application in agile model.

Scrum team:

It is the group of Engineers working towards completing committed features or stories.

- a. Generally, scrum team will have 7-12 members.
- b. It includes Shared team and Core team members
- c. Core team includes Scrum master, Development Engineer and Test Engineer.
- d. Shared team includes Architect, Product Owner, Database admin, network admin, UI and UX designers, BA.
- e. Scrum master leads this entire scrum team and he facilitates everyone to complete their task.

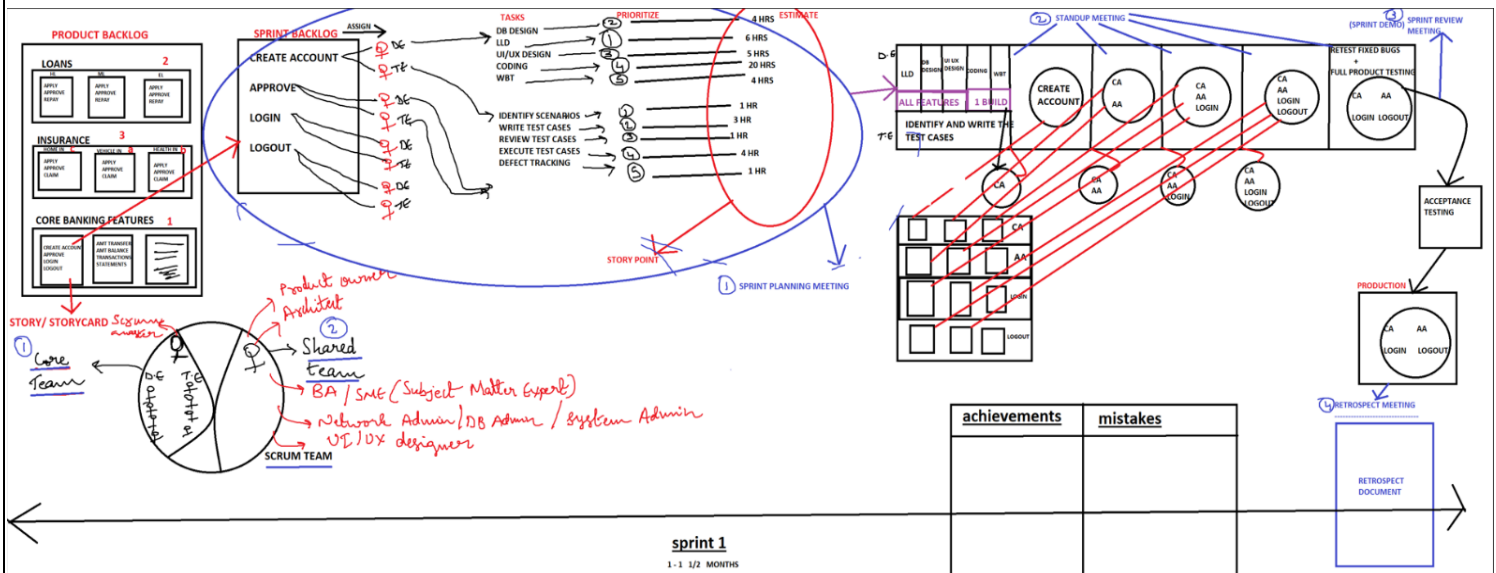
Product Backlog:

It is a prioritized list of stories or requirements that must be developed in the complete project.

- a. Generally, Product owner, Customer, Business analyst, architect, Scrum master will be involved in building it.
- b. Generally, stories in product backlog need not be in detail.

Sprint Backlog:

It is a list of stories and the associated task committed by the scrum team that must be delivered within one sprint.



I. Sprint Planning Meeting:

- Here the entire scrum team sits together and pulls the stories from the product backlog.
- Scrum master assigns each story to development engineer and test engineer.
- Now each engineer derives the tasks to be completed to build the stories.
- Each engineer will estimate the time taken to complete each task i.e. they derive the story point.

Following are the roles played by different people in Sprint planning meeting:

1. Scrum master:

- This complete meeting is driven by the scrum master.
- His prime role is to facilitate the complete meeting and co-ordinate between all stakeholders

2. Product owner:

- He clarifies if there are any questions related to stories or requirements.

3. Development Engineer:

- He should derive the task for building every story.
- He prioritizes which story to be built first and which story to build later in the sprint.

- c. He prioritizes the tasks.
- d. He derives the story point.

4. Test engineer:

- a. He derives the task to be completed to test each feature or story.

Ex: Create a/c : Identify scenarios

Write test cases

Review test cases

Execute test cases

Defect tracking

II. Daily Stand-Up Meeting/ Role Call Meeting/ Daily Scrum Meeting:

- a. Here the entire scrum team meets.
- b. This meeting is completely driven by the scrum master.
- c. Here every engineer should explain:
 - What they have done yesterday?
 - What were the impediments/hurdles they faced yesterday?
 - What are the Activities he is planning to do today?
 - What are the impediments he is expecting in order to complete today's task?
- d. The scrum master tries to solve certain impediments right there in the meeting. If it takes too much time then scrum master notes it down in 'Impediment backlog' and solves it later.
- e. Generally, this meeting should be completed within 10-15 mins.
- f. This meeting should be conducted at the beginning of the day.
- g. Here everybody should stand-up in the meeting so that people only talk to the point.

III. Sprint Review Meeting:

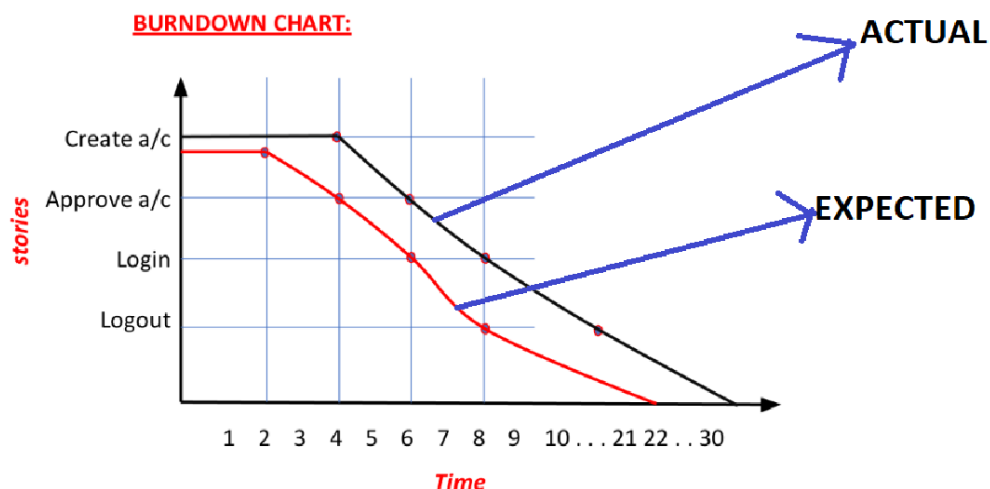
- Sprint review meeting should be done at the end of the sprint, where the engineers will give a demo to the product owner.
- They will also discuss how to plan for the next sprint.

IV. Retrospective Meeting

- Here the entire scrum team meets and discusses all achievements (good process followed) and mistakes (wrong activities performed) and it will be documented. This document is called as Retrospect document.
- When the next sprint starts while doing sprint planning meeting, we refer this document & we plan it in such a way that old mistakes should not be repeated and good activities are once again adopted.

Burndown Chart:

It is a graphical representation of work left vs time.



Storyboard/White Board:

It is a board which contains a list of pending tasks, tasks in progress and completed tasks.

	PENDING	TASK IN PROGRESS	COMPLETED
E1	☐ ☐	☐	☐ ☐
E2	☐ ☐	☐ ☐	☐
E3	☐ ☐	☐	☐

Chicken

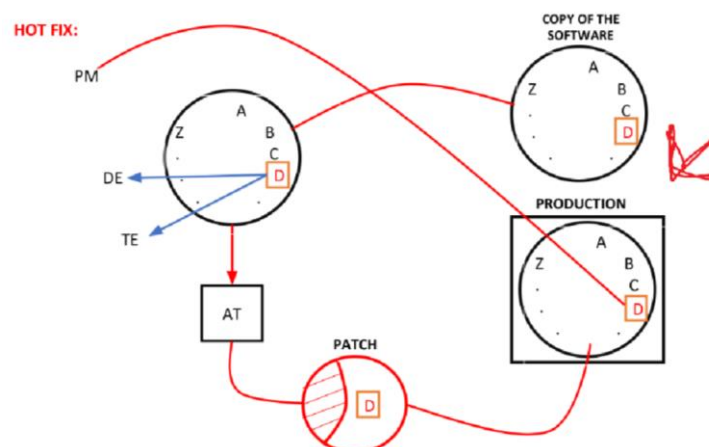
It is one who observes and tries to understand how sprint is going on and he will not be doing any task

Story point

It is estimation of how long task would take to complete or how long story is going to take to develop and test

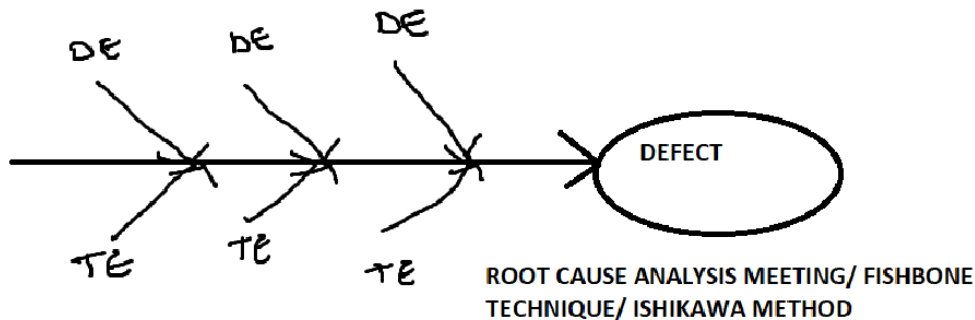
Hot Fix/ Incident Management:

- In the production if the customer faces any blocker or critical defects it will be communicated to the company, developers will immediately fix the defect and TE will retest and patch will be created & it will be installed in the production server.
- This is called as Hot Fix or Incident management.
- This may take up to 3 hours or 3 days.

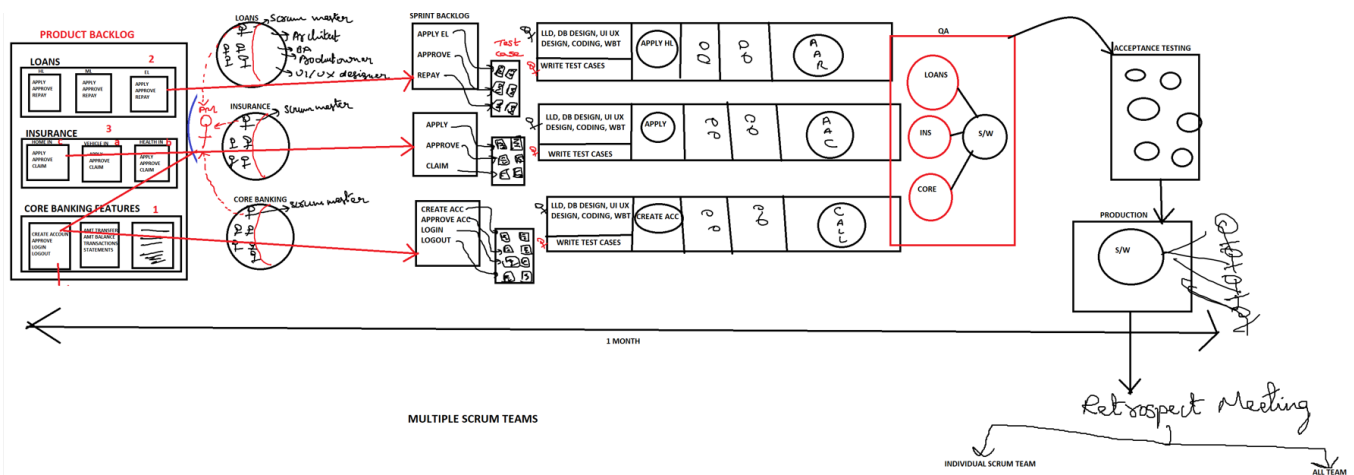


Root Cause Analysis:

- Here the entire team sits together and finds the root cause of the defect and shares it in a common folder where everyone can access it & present it to the entire team.
- This technique is called Fish bone technique or Ishikawa method or RCA meeting (Root Cause Analysis)

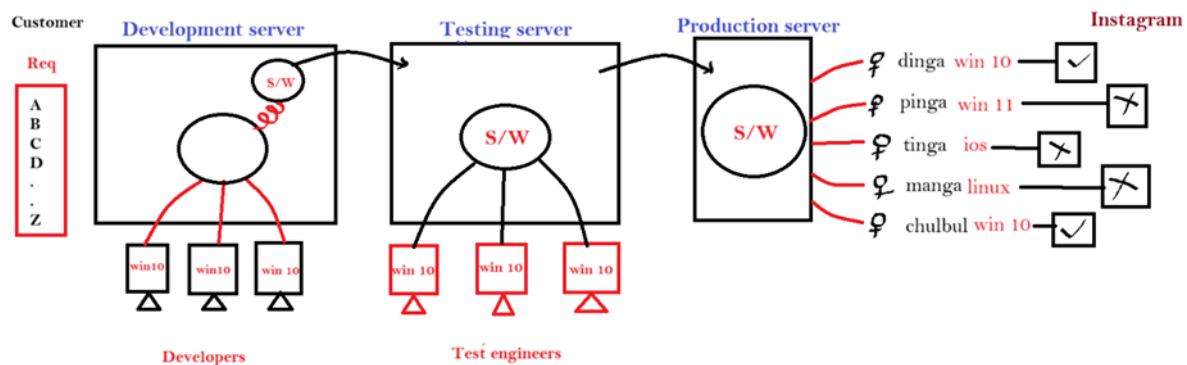


Multiple Scrum Teams:



COMPATIBILITY TESTING

Testing the functionality of the application in different hardware and software environments is called as Compatibility testing.



Why do we do Compatibility Testing?

1. Chances are their developer might develop the software in one platform and TE would test the software in same platform and when it is released to the production end-users might use the application in different platform, software which works in one platform, but might not work in another platform because of some defects, due to this end-user's usage will go down & customer will undergo a huge loss, to avoid that all this we do compatibility testing
2. To check whether the application is working consistently in all the platforms we do compatibility testing
3. DE might write common code & claim that application works in all the platform or else DE might write platform specific code & say that it is works in all respective platforms
4. We have to test it in every platform & confirm that it really works or not

Common code

```

If(win any)
{
  Display help
  -----
  Work help
  -----
}

```

Platform code

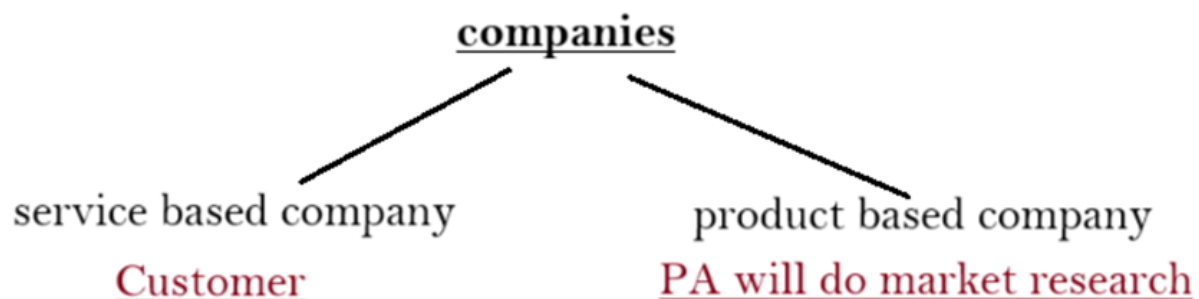
```

if(win 10)
{
  Display help
  work help
}
if(win 8)
{
  Display help
  Work help
}
if(win 11)
{
  Display help
  work help
}

```

When we do Compatibility Testing?

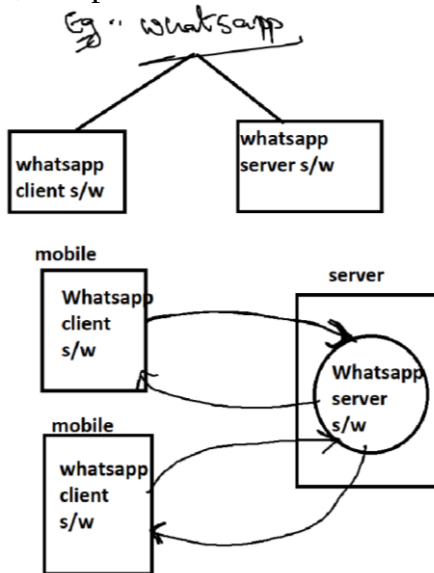
When the product is functionally stable in the base platform only, we think about testing the application in different platform

**How we do compatibility testing?**

- It depends on the type of application
- There are 3 types of application:
 1. **Web application:** Any application that can be accessed by opening the browser and entering the URL is called as Web Application
 Ex: www.gmail.com, www.amazon.com, www.facebook.com, etc.

2. **Client server application:** It is a kind of application where two types of software are there- client software & server software wherein we use client software to interact/ communicate with server software. This kind of application requires both internet as well as server.

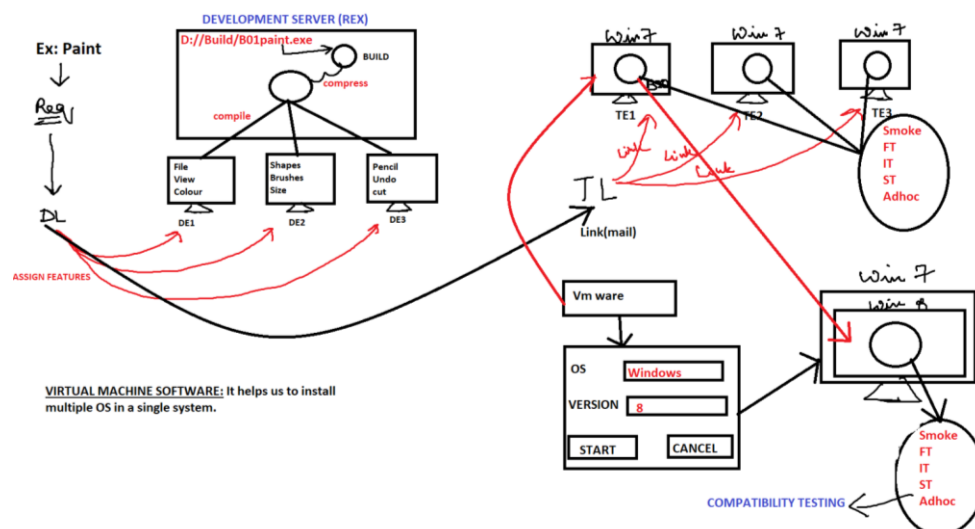
Ex: WhatsApp, Instagram, Snapchat



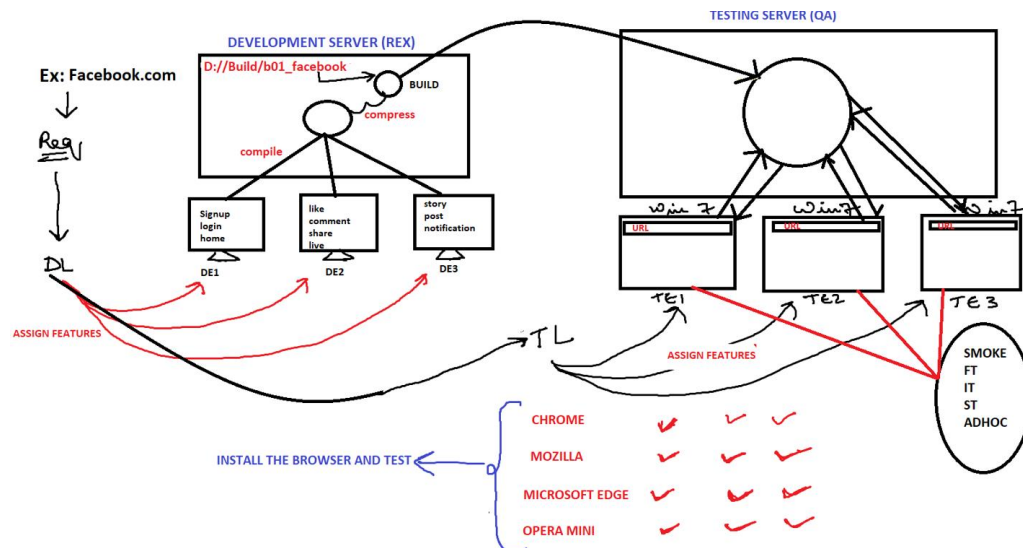
3. **Stand-alone application:** It is a kind of application where we take one setup file and install it in computer or a mobile, only one user can access the software at a time and no internet as well as server is required, this kind of application is called as stand-alone application.

Ex: Calculator, Alarm, MS Paint

How to do Compatibility Testing for Standalone Application?

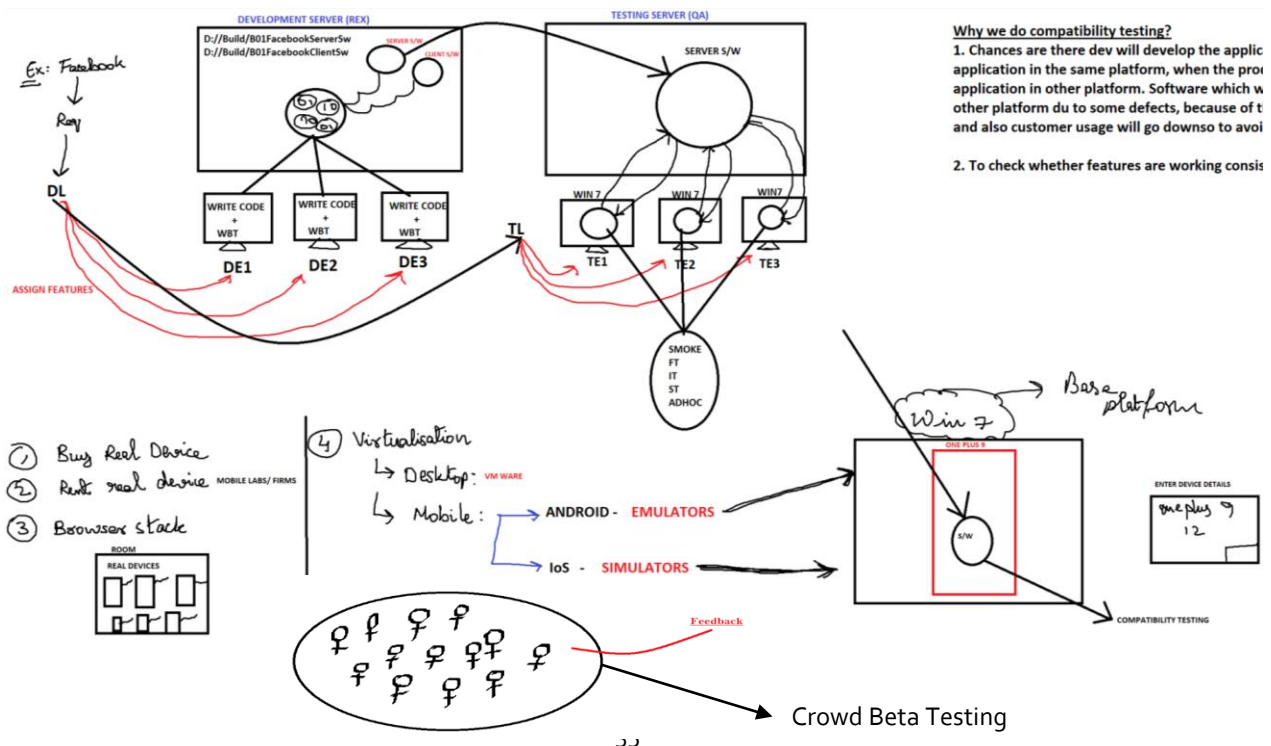


How to do Compatibility Testing for Web Application?



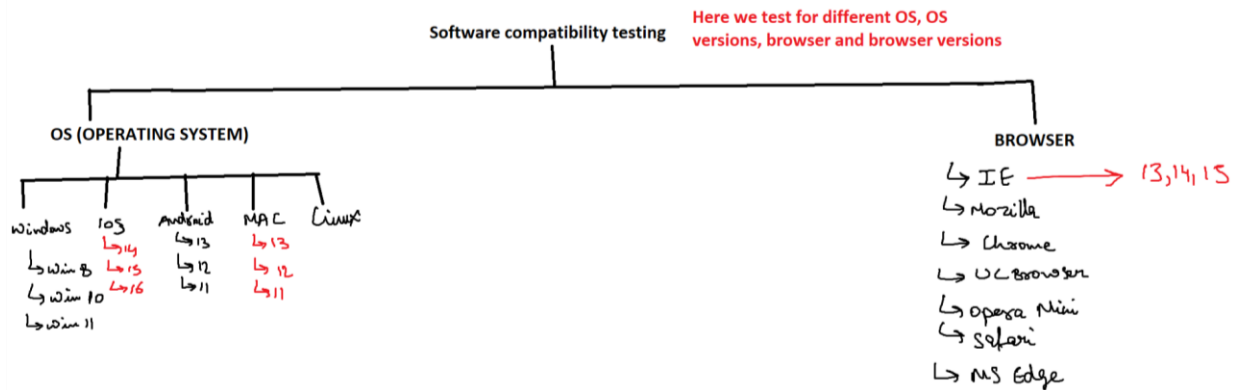
How to do Compatibility Testing for Client Server Application?

1. Buy the real device
2. Rent the real device
3. Browser stack
4. Virtualization
5. Crowd beta testing



Types of Compatibility Testing:

1. Software Compatibility Testing
2. Hardware Compatibility Testing

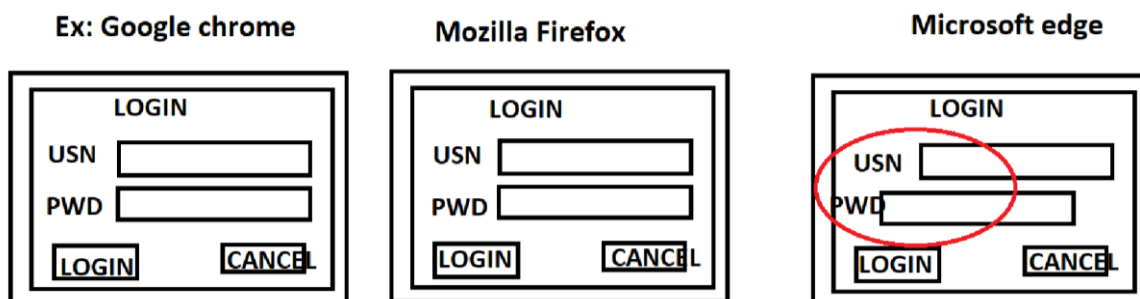


2. HARDWARE COMPATIBILITY TESTING

1. Test for different processors (We should test for different make and manufacturers) Ex: AMD, Intel, Snapdragon
2. Test for different RAM (We should test for different make and manufacturers) (For different size and speed) Ex: HP, Sandisk, Kingston
3. Test for different motherboard (We should test for different make and manufacturers) Ex: Intel, Acer, Gigabyte
4. Test for different VGA cards (We should test for different make and manufacturers) Ex: Faxon, Nvidia, ATT.

What are the bugs found while doing Compatibility Testing?

i. Alignment issue



ii. Object Overlapping

Ex: Google chrome Mozilla Firefox Microsoft edge

The image displays three login form mockups side-by-side, labeled 'Ex: Google chrome', 'Mozilla Firefox', and 'Microsoft edge'. Each form contains a title 'LOGIN', two input fields labeled 'USN' and 'PWD', and two buttons labeled 'LOGIN' and 'CANCEL'. In the 'Microsoft edge' version, the 'LOGIN' and 'CANCEL' buttons are circled in red, highlighting a specific design or compatibility issue related to object overlapping.

iii. Look and feel issues

Image with certain formats may not be displayed in certain browsers.

iv. Scroll bar issues

Sometimes scroll bar may not be displayed or it might be displayed but may not work because it acts like an image

PERFORMANCE TESTING

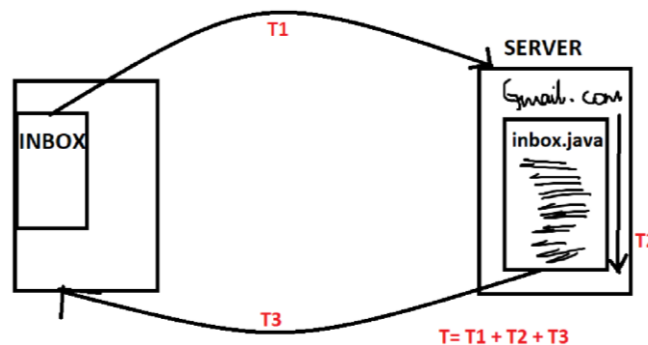
Testing the stability and response time of an application by applying load is called as Performance Testing.

Stability: Ability of an application to withstand load

Load: Designed number of users

Response time: It is the total time (t) taken to receive the request (t1), execute the program (t2) and send the response (t3).

$$T = t1 + t2 + t3$$



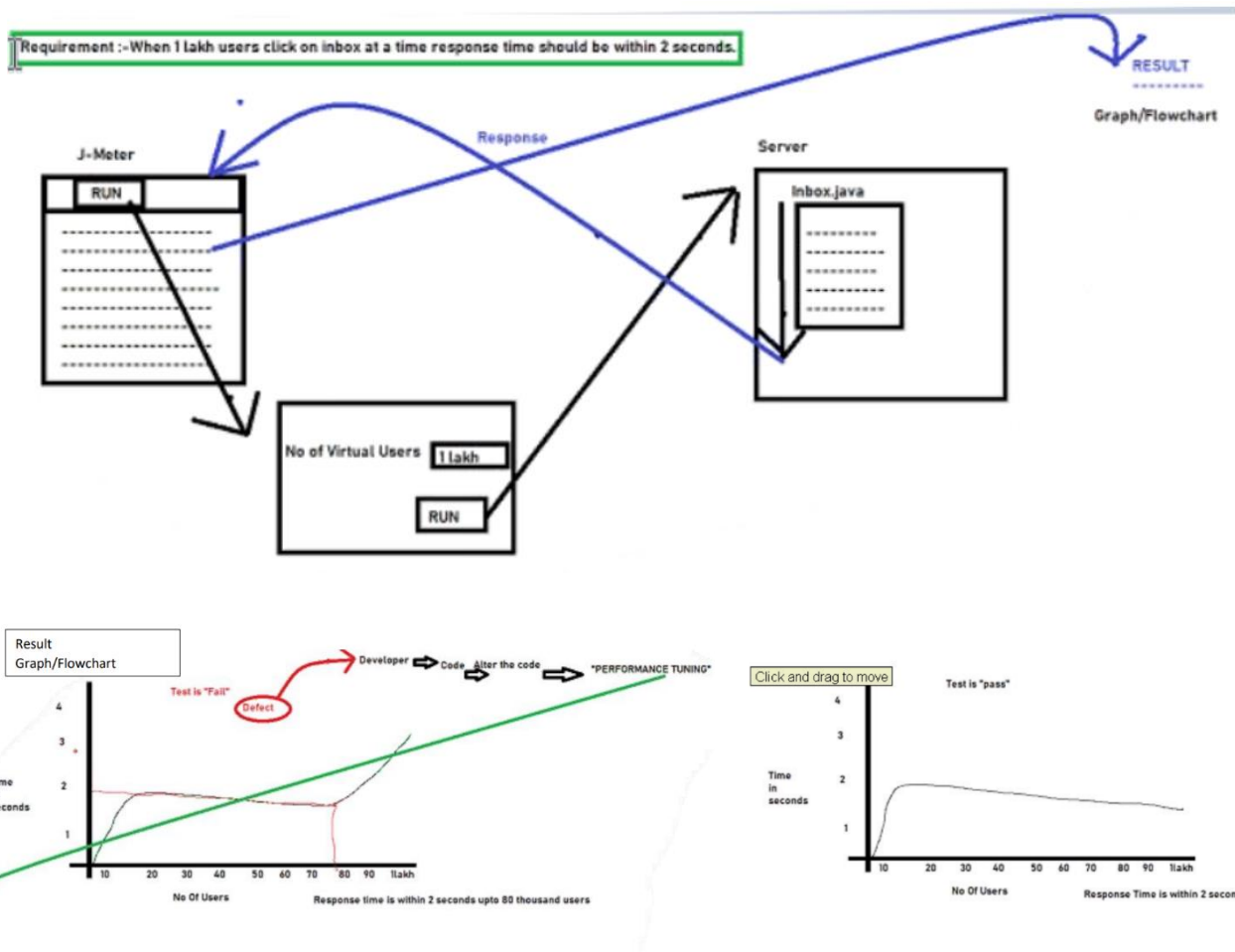
Performance tools available in market

1. J meter
2. Neo load
3. Load runner
4. Rational performance tool
5. Silk performance tool

How to do performance testing using J meter?

1. Take performance testing tool (J-meter), write the scripts
2. Click on run, it will ask you how many users (enter 1lakh and click on run)
3. 1 lakh requests are fired to the server now there is a heavy load on the server
4. Server sends the response to the performance testing tool
5. Tool analyze the response and gives the result in the form of graph
6. TE should analyze the result and decide whether test is pass or fail

7. If it is failed then send it to the developers they will alter/change the code to improve the Performance i.e. they do "performance tuning"
8. Once again TE should run the test script
9. Repeat all the steps

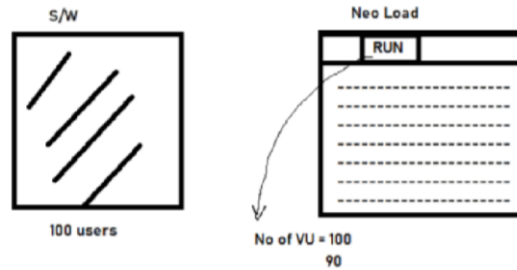


Types of Performance Testing

1. Load testing
2. Stress testing
3. Volume testing
4. Soak testing

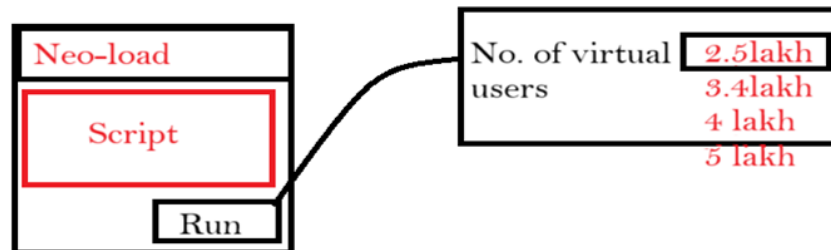
1. Load testing

Testing the stability and response time of an application by applying load less than or equal to designed no. of users is called Load testing.



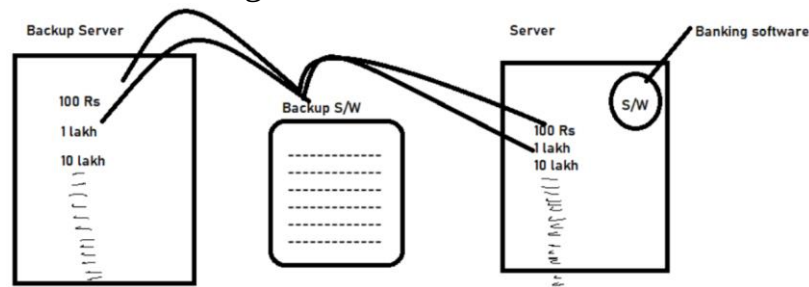
2. Stress testing

Testing the stability and response time of an application by applying load more than the designed no. of users is called Stress testing.



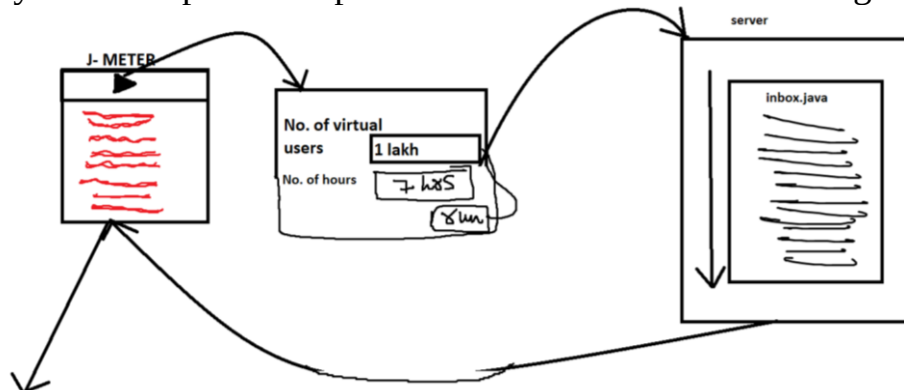
3. Volume testing

Testing the stability and response time of an application by transferring huge volume of data is called Volume testing.



4. Soak testing

Testing the stability and response time of an application by applying load continuously for some particular period of time is called Soak testing.

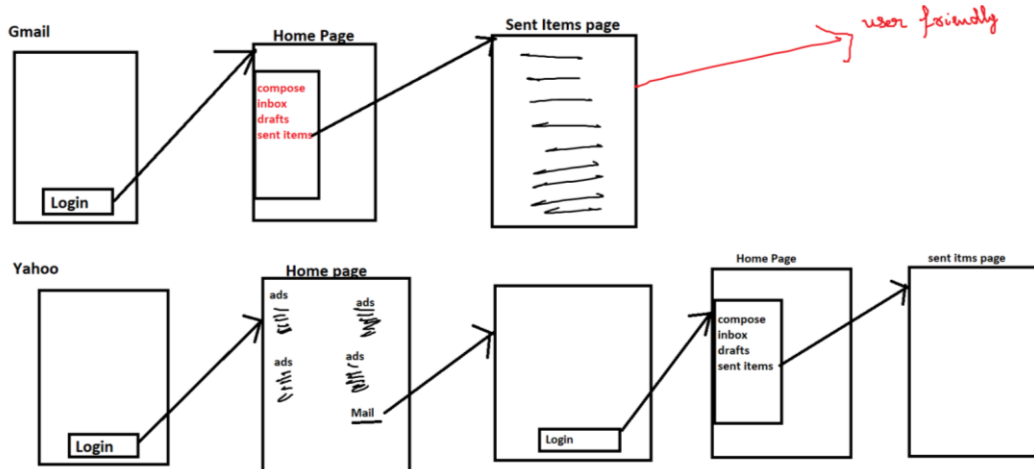


For what kind of applications, we should do Performance Testing?

1. Any application which generates lot of revenue
2. Any application which is used by multi-users

USABILITY TESTING

Testing the user friendliness of an application is called as Usability Testing



How to do Usability Testing?

1. Here we will check whether look and feel of the application is good or not.
2. Here we will check whether it is easy to understand or not (or) I will check whether I'm able to understand within less time.
3. Frequently used features/important features must be given to users within 3 clicks
4. Frequently used features/important features must be easily accessible.

Three examples of login forms are shown:

Ex 1: A simple form with 'USN' and 'PWD' labels, each followed by a text input field with a blue checkmark. At the bottom are 'L' and 'C' buttons, both with blue checkmarks.

Ex 2: A form with 'Username' and 'Password' labels, each followed by a text input field with a blue checkmark. Below these is a 'Rem Pass' checkbox. At the bottom are 'Login' and 'Cancel' buttons, both with blue checkmarks.

Ex 3: A more complex form with 'Username' and 'Password' labels, each followed by a text input field with a blue checkmark. Below these is a 'Rem Pass' checkbox. Further down is a 'Forgot password?' link. At the bottom are 'Login' and 'Cancel' buttons, and a 'Sign Up' button. A blue arrow points from the 'Forgot password?' link to the 'Sign Up' button.

For what kind of applications, we should do usability testing?

1. Any application which is used by variety of users
2. Any application which generates lot of revenue
3. Any application wherein we don't provide training to user about how to use the application, we are expecting the end users himself to understand the application and use.

When we should do usability testing?

In different projects they start usability testing in different stages:

- a. In certain projects they do UT when the product becomes functionally stable
- b. In certain projects they do UT in the beginning of SDLC itself

REGRESSION TESTING

Testing the unchanged features to make sure that is not affected or broken because of changes is called as Regression testing (here changes can be addition, modification, removal of features or bug fixes)

(OR)

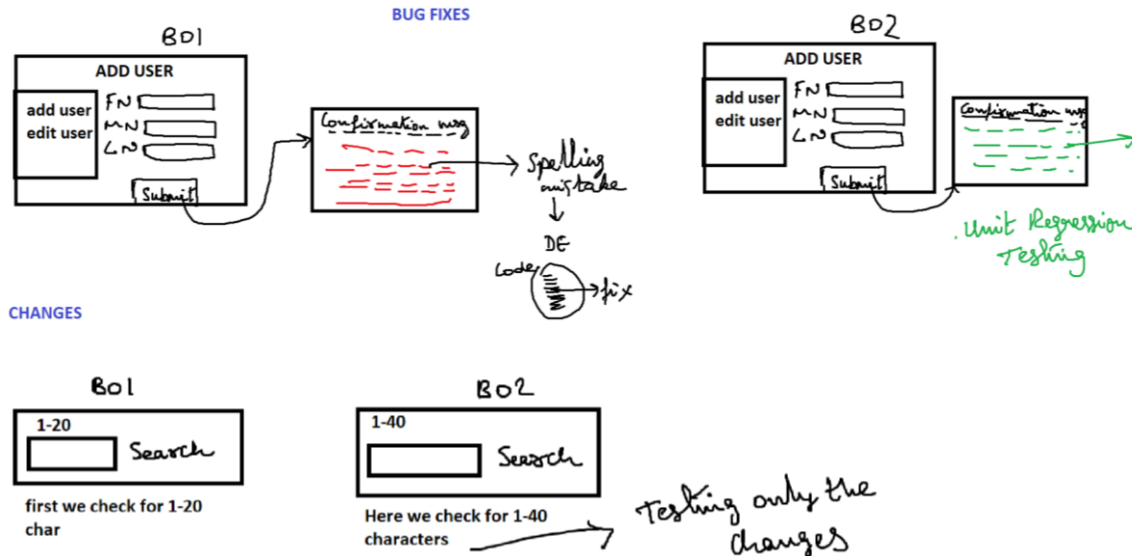
Re-execution of same test case is different test cycle/ sprint/ release to make sure that changes are not introducing any defects in unchanged features (Changes can be addition, modification or removal of features) is called Regression testing

Type of Regression testing:

- 1) Unit regression testing
- 2) Regional regression testing
- 3) Full regression testing

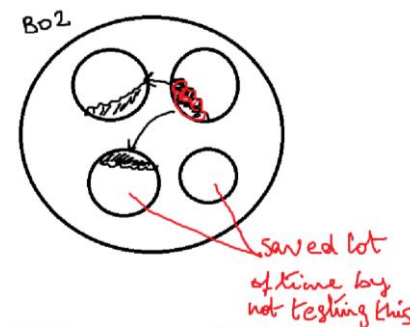
a) Unit regression testing:

Testing the changes (or) only the bugs which is fixed is called as unit regression testing.



b) Regional regression testing:

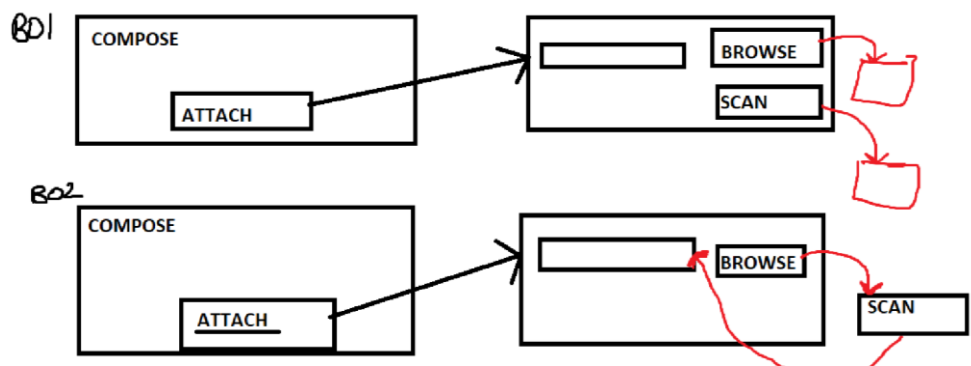
Testing the changes & only the impacted regions is called regional regression testing



How will you identify impacted Region?

1. Based on product knowledge

As a TE in-depth I will be knowing how each & every module works & also I will be knowing how all the modules are related based on that knowledge, I will be able to identify impacted areas



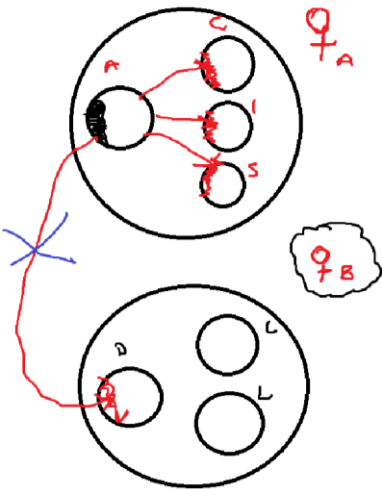
2. By preparing Impact matrix

Here we list the changes & also all the features in the application, then mark the impacted areas.

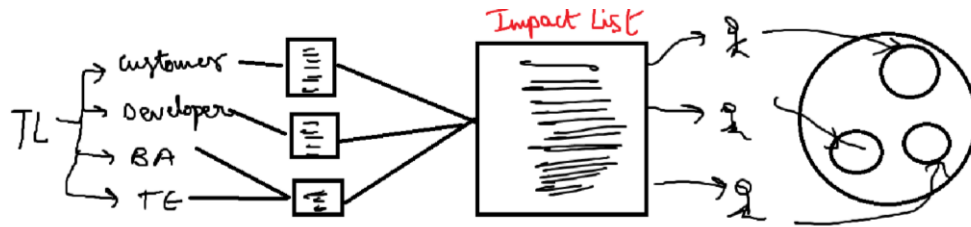
changed features	signup	login	logout	compose	inbox	SI	all mails	drafts	trash	search	spam	starred	scheduled	calender	contacts
ATTACH	X	X	X	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	X	X
FORGOT PWD	✓	✓	X	X	X	X	X	X	X	X	X	X	X	X	X

3. By conducting Impact analysis meeting

As soon as the new build comes entire testing team meets & discuss about list of bugs fixed & the impacted areas



- By interacting with customer, business analyst, development team, testing team we should gather the impacted areas & create an impact list, based on that we should do Regression testing



Advantages of Regional Regression Testing:

- By not testing certain features we are saving testing time which intern reduces the testing cost
- Test cycle duration Reduces because of that turnaround time taken to deliver the product to the customer reduces

Dis-advantages of Regional Regression Testing:

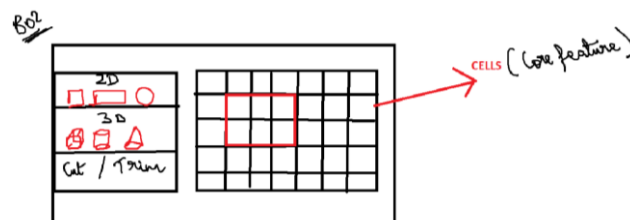
- Chances are there we might miss identifying the impacted area because of that we might miss the bugs.

c) Full Regression testing:

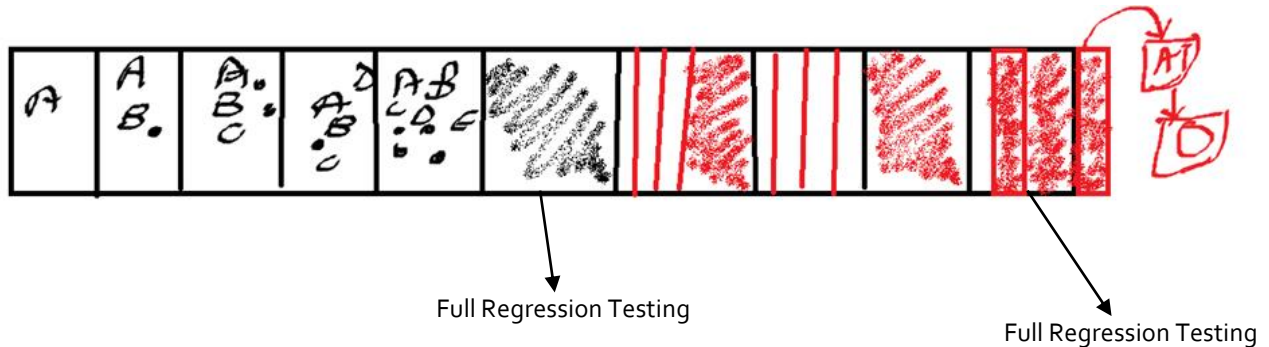
Testing the changes and all the remaining features is called as Full Regression testing

Why/when we do full regression testing?

- Whenever too many changes are done in the product better to do full regression testing
- If the changes are done in core features



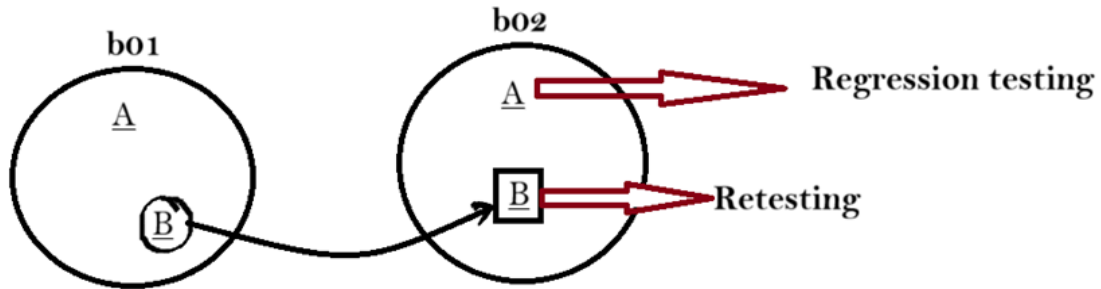
3. Every few (4-5) cycles once we should do full regression testing & last few cycles we should do full regression testing because we are about to launch the product to the production to not to take any risk.



What is the difference between Regression Testing and Re-testing?

<u><i>Regression Testing</i></u>	<u><i>Retesting</i></u>
Fixing the bugs or doing changes might have impact on other features so testing the unchanged feature to make sure that it is not broken because of changes is called as Regression Testing	Whenever developer gives build, checking or verifying whether defect is fixed or not is called Retesting.
Regression Testing is done for passed test cases	Retesting is done for failed test cases
We go for automation	We don't go for automation

Progression testing: Testing newly added features is called as progression testing.



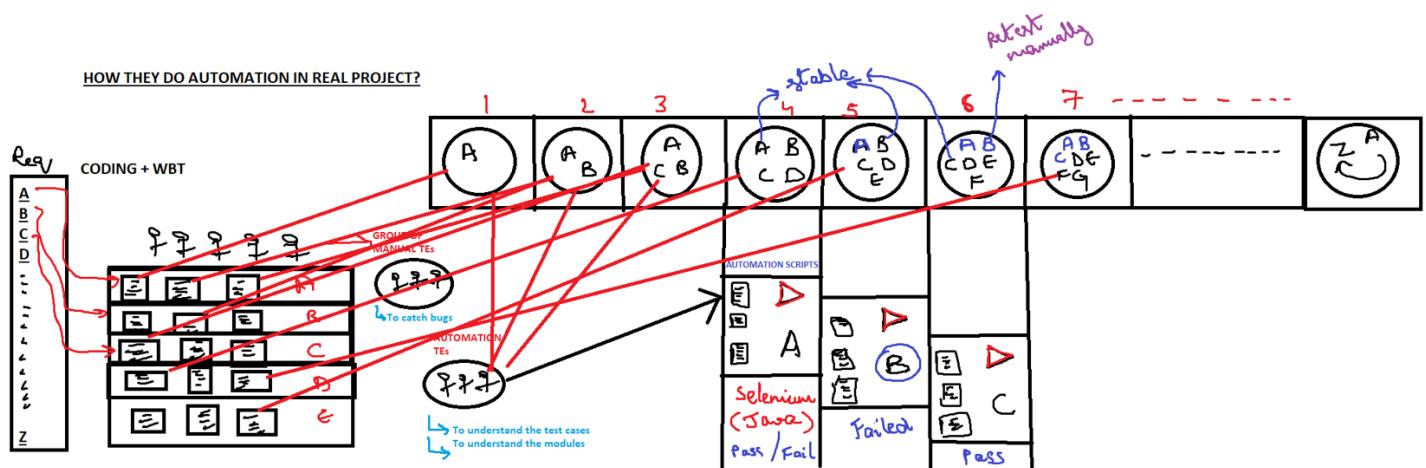
When we do regression testing?

- 1st release 2nd build only we do regression testing
- 2nd release 1st build only we do regression testing

What are the drawbacks of manual repeated regression testing (OR) What is the drawback of manual testing?

1. Manual testing is repetitive in nature over the period of time it becomes monotonous because of that testing engineer may not be effective in testing
2. As the size of the application increases test cycle duration also increases because of that turnaround time taken to deliver the product to the customer increases
3. Man power is expensive

How they do automation in real project?



What is the role of Manual test engineer?

1. Write manual Test cases
2. Test new feature manually.
3. Test modified feature, fixed bugs manually
4. Find the defects & communicate to the developers.

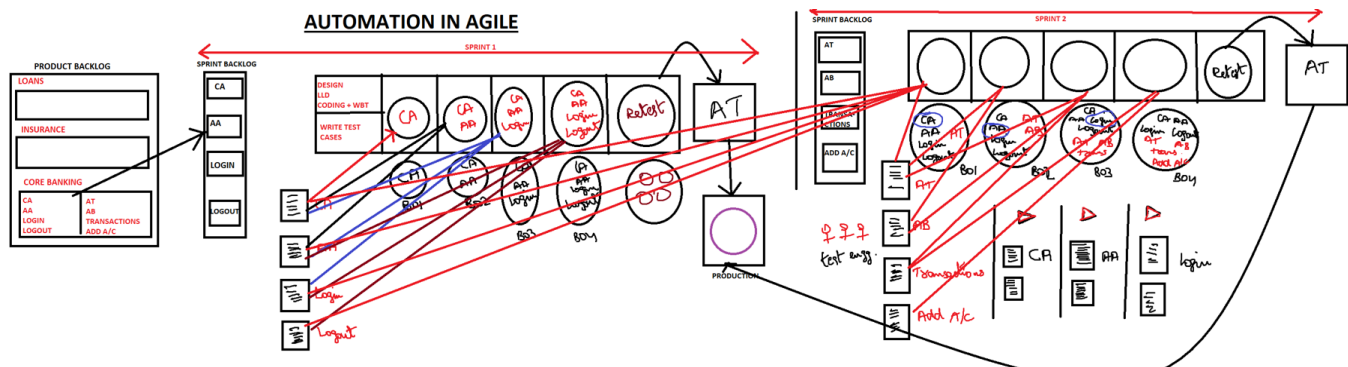
What is the role of Automation Test engineer?

1. Understand the application
2. Understand the test cases
3. Convert the manual test cases into automation test scripts (manual test cases of the stable features)
4. Execute the automation scripts when the new build comes.
5. Maintain the automation scripts (Whenever the requirement changes, test cases also should be changed, when the test cases changes, we should change the automation scripts and also if there are problems in old script, we should fix it)

Why we go for Test Automation?

1. To reduce manual repeated testing efforts.
2. To reduce test cycle duration.
3. To reduce the turnaround time taken to deliver the product to the customer.
4. To reduce the no. of engineers.
5. To reduce the cost of testing.
6. To improve the test efficiency.
7. To have the consistency in quality of test execution.

How we do automation in Agile?



EXPLORATORY TESTING

Understanding the application, identifying all possible scenarios, documenting the scenarios and testing the application by referring the document is called as Exploratory Testing

(OR)

Exploring the application, understanding how each and every feature works and testing the application as per your understanding is called as Exploratory Testing

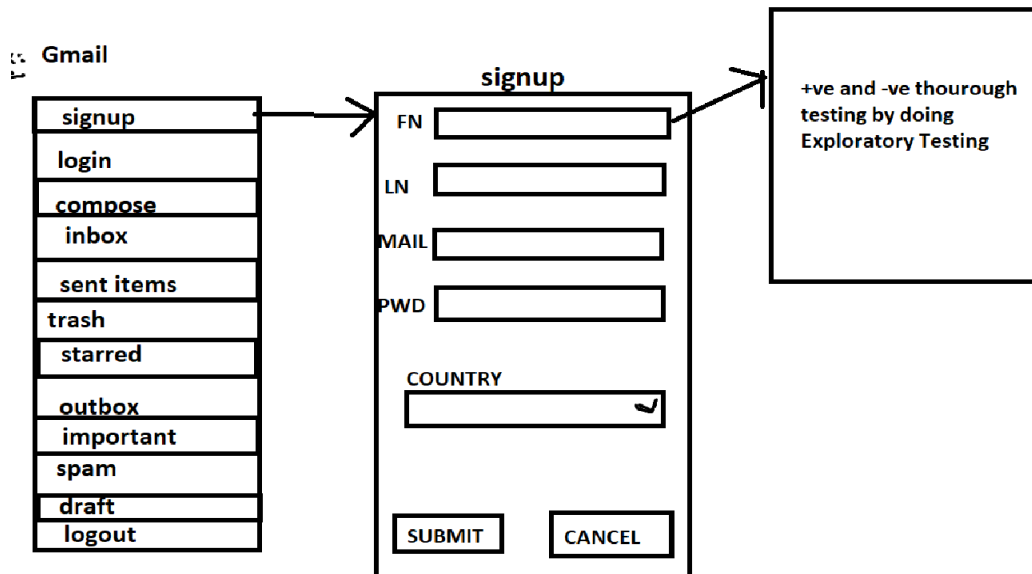
When to do Exploratory Testing?

- When there are no requirements
- Requirements are present but TE doesn't have time to understand the requirement (Some companies they hire TE in the mid of the project and they will not give sufficient time to understand the req.)
- There is requirement, but it is not understandable (it means it's very complex)

Why requirements will be missing?

- Chances are there if project is very old and big project, for the old features requirements might be missing.
- Chances are there in the product-based companies, for some products requirements might be missing. (some startup companies might not document the requirements)

How to do Exploratory Testing?



1. I will understand the application and I will try to enter all possible inputs to each and every component and I will do Exploratory Testing.
2. I will understand the application and check the dataflow b/w the modules by doing Exploratory Testing.
3. I will explore the application and I will understand how each and every feature works and I will try to test for all possible end to end scenarios by doing Exploratory Testing.

Drawbacks of Exploratory Testing

1. Chances are there we might misunderstand 'feature' as a defect.

Eg: Citibank

Ref

2. Chances are there we might misunderstand 'defect' as a feature

Eg: Citibank

Reg ~~X~~
User id
Ext field
Should
accept
2-8 digits

ABCD X
#@? X
12345 ✓
12345678901 ✓

3. If a feature is really missing, we will not get to know the feature is really missing.

Eg: Citibank

Forgot Password? → missing feature

4. It is more time consuming because we will spend more time in understanding the application.

How to overcome these drawbacks?

1. TE will interact with Sr. TE, Sr. Dev, BA, customer or talk to anybody who is having good knowledge on the application.
2. Based on product knowledge (If a person has worked on the same project for around 4-5 years and has good knowledge on that project then we can tell he is a product expert/product knowledge)
3. Based on domain knowledge (If a person has worked on multiple projects which belongs to same domain and he has got very good knowledge on the domain then we can tell he is domain expert/ domain knowledge)

4. By comparing with similar kind of application which is released in the market
5. By using some common sense, we can overcome the drawback.

Interview questions

- i. Do you write Exploratory scenarios and test cases?
Yes
- ii. How to test the project when there is no requirement?
I will understand the application and I will do exploratory testing. Chances are there I might misunderstand feature as a defect or defect as a feature. So, in order to overcome this drawback, I will talk to my Sr. TE, Dev, BA, customer and based on my product knowledge, I will test the application.

GLOBALIZATION TESTING

Developing the software for multiple languages is called as Globalization. Testing the software which is developed for multiple languages is called as Globalization Testing.

Types of Globalization Testing:

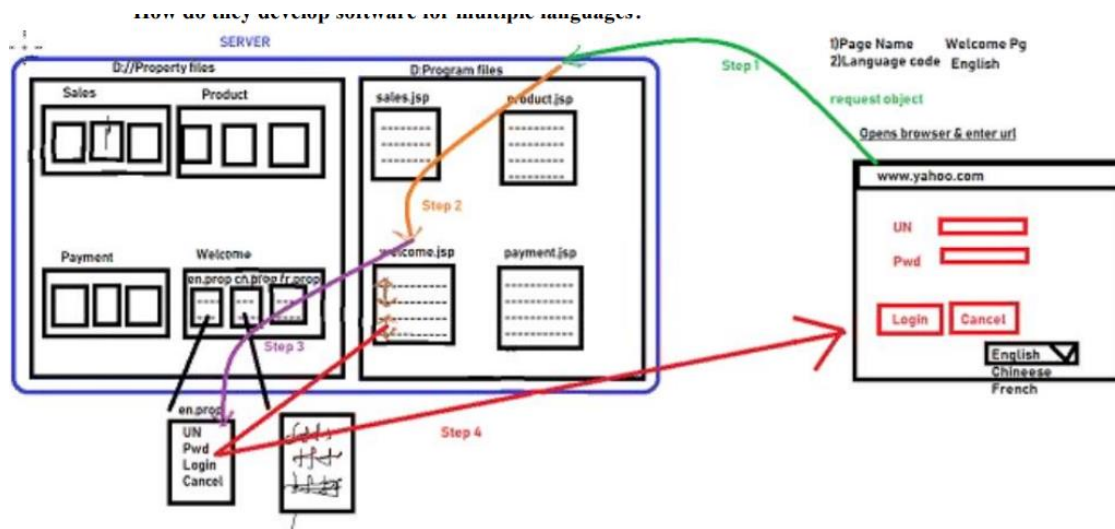
1. Internationalization Testing (I18N Testing)
2. Localization Testing (L10N Testing)

How application works for multiple languages?

1. First TE will open the browser and enter the URL. When he clicks go, request goes to server. The request object has 2 things: (1) Page name (2) Language code [In this example page name= Welcome, Language code= English]
2. Server is waiting for the request. As soon as the request object comes in, it goes and checks for page name. [In this example page name= Welcome. So, it runs welcome.jsp]
3. The program starts running, it has 2 sections. The 1st section runs and receives the request object and goes inside the request object and checks the language code. [In this example language code= English. So, it connects with English]

property files. So, the connection b/w Program files and Property Files are established]

4. The remaining program copies the content from the property files and displays it in the browser.



What is I18N Testing?

Testing the application which is developed for multiple languages is called I18N Testing.

Here we check whether:

Content is in right language or not

We check whether right content is displayed in right place or not

We check whether features are broken if the language is changed

How to do I18N testing for Chinese language?

- a) Go to Chinese property file
- b) Add prefix and suffix to the content
- c) Save the property file
- d) Open the application & select the language (Chinese) corresponding page comes
- e) I will check for prefix, if prefix is correct means content is in right language
- f) I will check for suffix, if suffix is correct means content is in right place

Localization Testing

Check whether the certain features are changing locally according to the country standards.

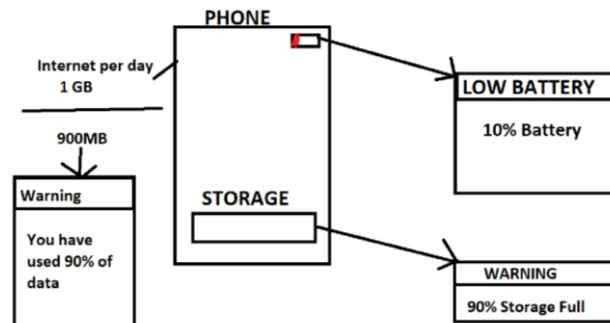
Example: Here we check currency format, date and time format, pin code format

How do we test images?

1. Image should not contain the text
2. If you want to have the text on the image then multiple images must be present in the server
3. Tool tip (or) alt tag must be displayed in the corresponding language

YELLOW BOX TESTING

Testing the warning message of an application is called as yellow box testing.



COMPARISON TESTING

Testing the newly build application with the similar kind of application which is released in the market, here we compare the application, check the advantages and dis-advantages and check whether all the features are present in our newly built application or not.

Ex: Zomato with Swiggy
Amazon with Flipkart
Ola with Uber

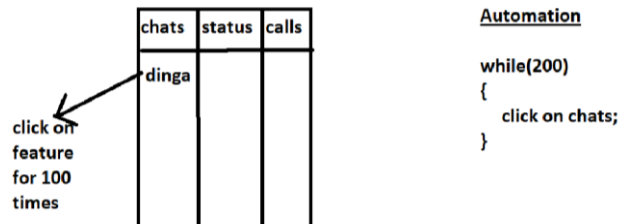
ACCESSIBILITY TESTING

Testing the user friendliness of an application from physically challenged people point of view.

**Ex: We check just by entering keyword whether we are able to access the objects
ATM machine for blind people**

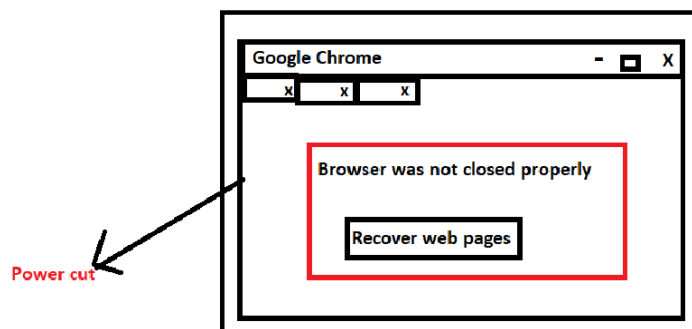
RELIABILITY TESTING

Testing the functionality of an application continuously for a particular period of time is called as reliability testing.



RECOVERY TESTING

Testing the functionality of an application to check well the application recovers the data from the crash or disasters.



DEFECT

What is defect?

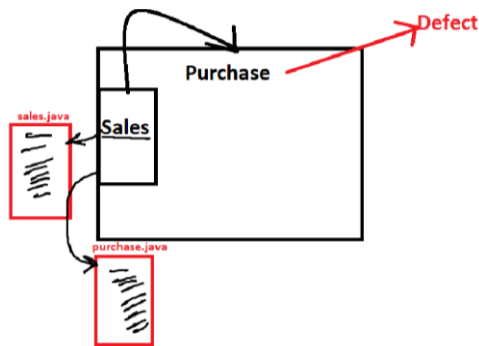
Any feature which is not working according to the requirement specification is called as Defect.

OR

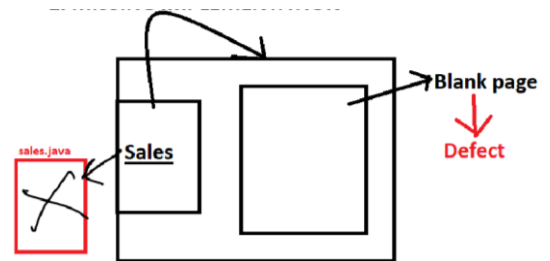
Deviation from requirement specification is called as Defect.

Why do we get Defects?

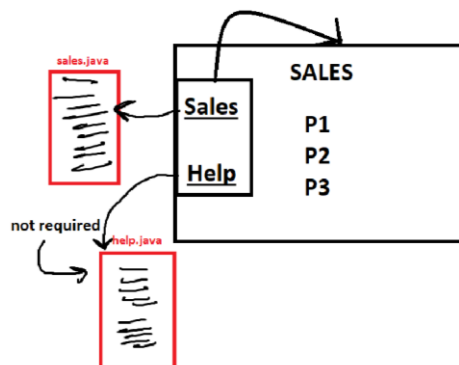
1. Wrong Implementation



2. Missing Implementation



3. Extra Implementation



What is the difference between Error, Defect, Bug and Failure?

Error: Error is a mistake done in the program by the programmer because of which we will not be able to compile the program and run the program.

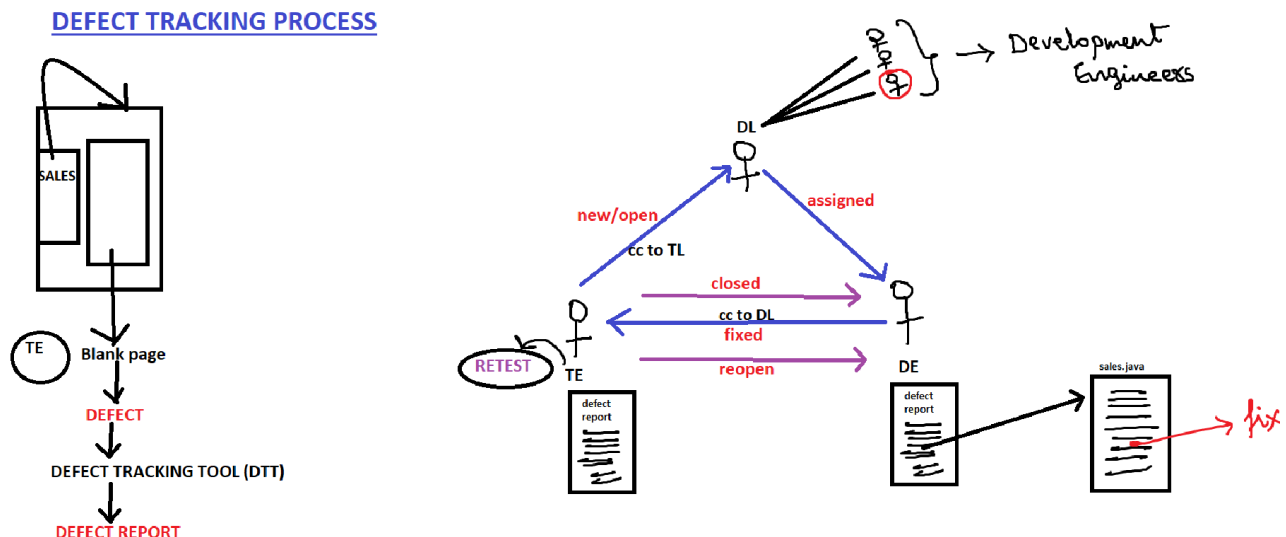
There are 2 types of error: i) Compile time error ii) Run time error

Defect: Deviation from customer requirement specification is called as defect

Bug: It is an informal name given to defect.

Failure: Defect leads to failure. One defect might lead to one failure/ multiple failures

DEFECT TRACKING PROCESS:



Why test engineer should not wait for the permission from test lead for sending report to development lead?

- There will be delay in communication report to developer
- As a test engineer will be having knowledge in depth about his feature, so better take a decision and send report directly to development lead without the permission of test lead

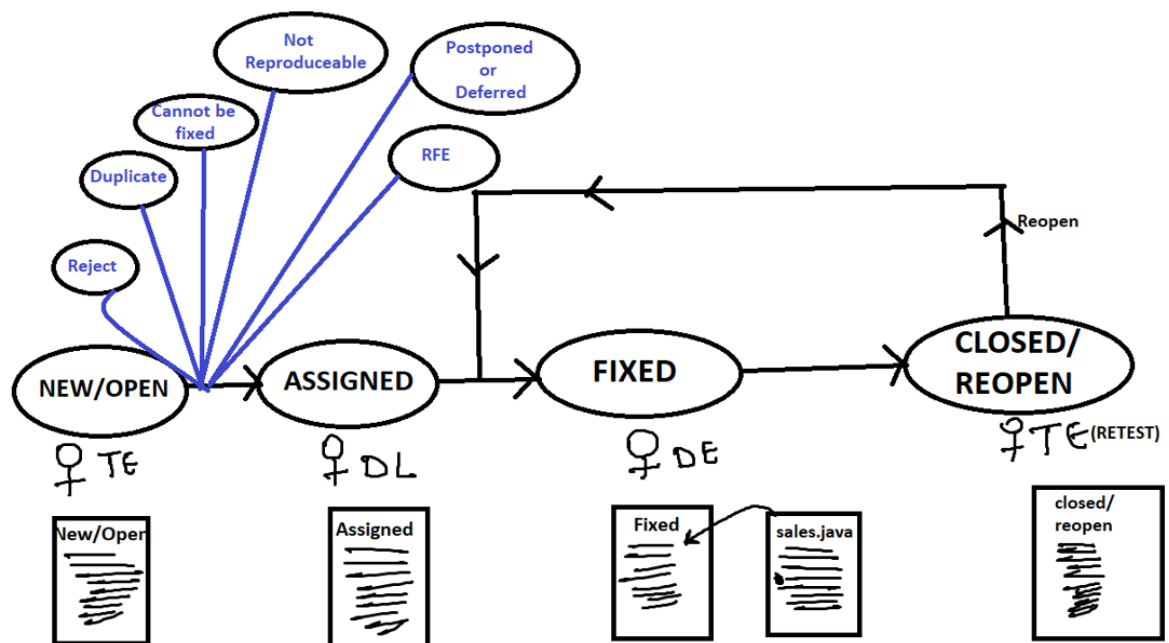
Why we should keep CC for every report to test lead?

- Test Lead is the one who attends managements developers and customer meeting, so he should be aware of all the issue that are present in the product
- To get visibility that test engineer is working

As soon as you find defect, immediately we have to communicate defect to development team. Why?

- Developers will get sufficient time to find the defect
- Someone else might send your defect
- Chances are there TE might forget the defect

DEFECT LIFE CYCLE



Test Engineer:

1. Test engineer finds the defects
2. Prepare defects report
3. He will put the status as new/open
4. Send the report to development lead

Development Engineer:

1. DL reads the report & understand the problem
2. Identifies the developer who did the mistake
3. Change the status to assign
4. Sends it development engineer

Development Engineer:

1. DE reads the report & understand the problem
2. Goes the source code & fix the bug
3. Change the status to fixed
4. Send the report to test engineer & CC to development lead

Test Engineer:

1. TE reads the report & understand the problem fixed
2. Retest the fix bug, if the bug is fixed change the status to closed
3. Otherwise change the status to reopen
4. Send the report to development engineer & also CC to development lead

Defect life cycle consists of below mentioned status:

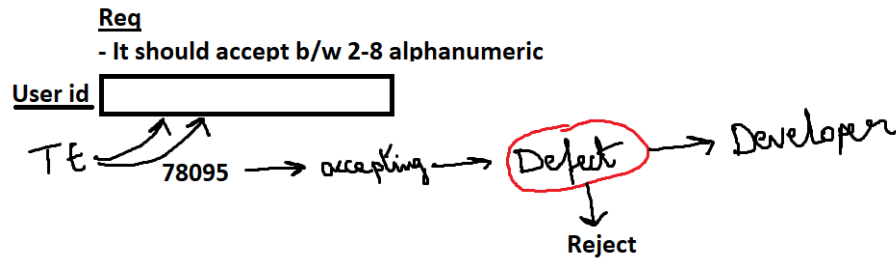
1. New/open
2. Assigned
3. Fixed
4. Closed
5. Reopen
6. Reject
7. Defect cannot be fixed
8. Postponed
9. Duplicate
10. Issue not reproduceable
11. Request for enhancement (RFE)

I. What is Reject status?

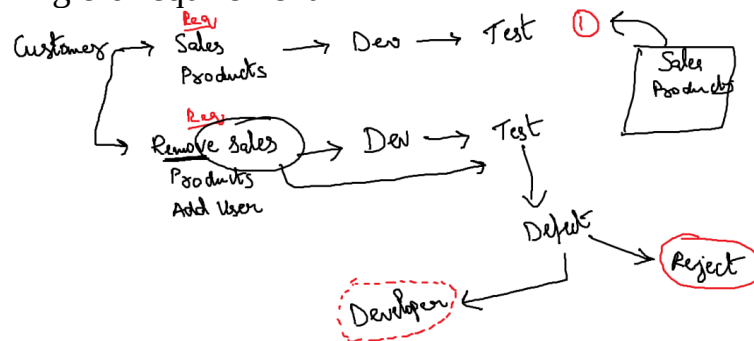
TE will find a defect and send it to developer, now developer says that it is not a defect, it is a feature, in this case developers will change the status as Reject.

Why we get Reject status?

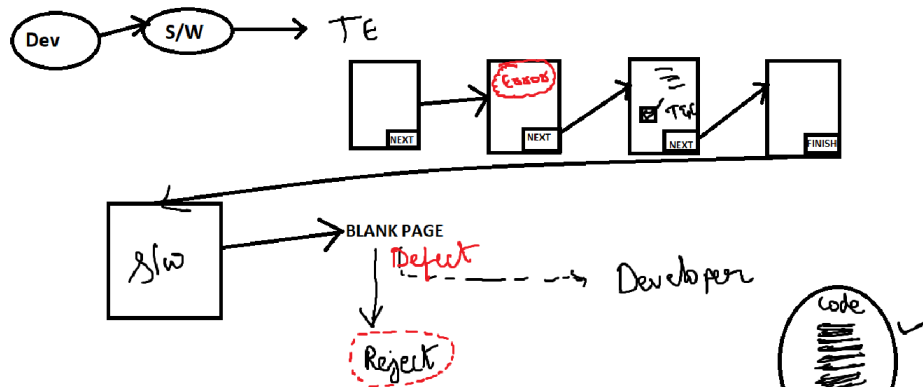
i. Because of misunderstanding the requirement.



ii. Because of referring old requirement



iii. Whenever build or software is wrongly installed or wrongly configured.

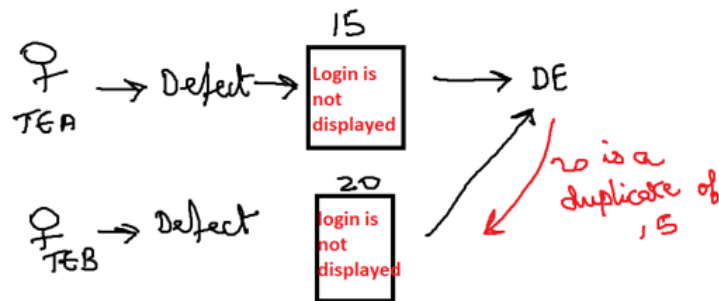


II. What is Duplicate status?

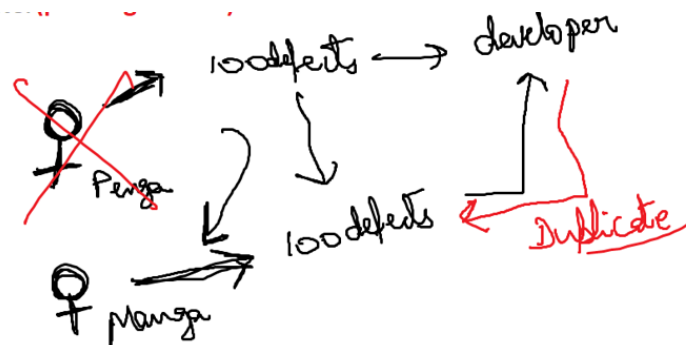
If bug is already sent tracked by someone else, they might change the status to duplicate.

Why we get Duplicate status?

i. Because of testing common feature.

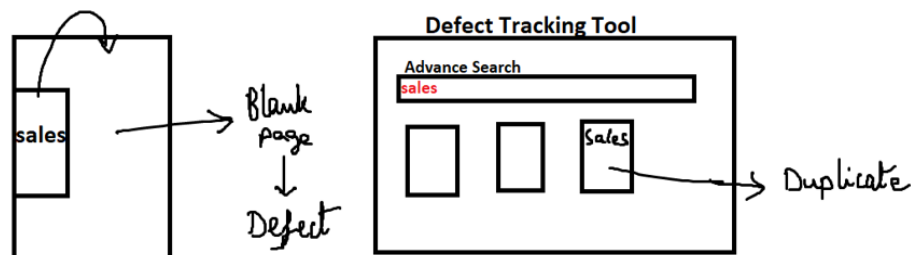


ii. Assume that old TE has found lot of defects and communicated to developer, in that some defects are fixed and some are pending. If new TE joins same project and communicate old defects then developers say this new defect is duplicate of old defects.



How to avoid duplicate bugs?

Before we prepare the report and log the bug, make sure that it is not duplicate or you should search for duplicate bugs in the tool by entering or using certain keywords (In this example sales is the keyword)

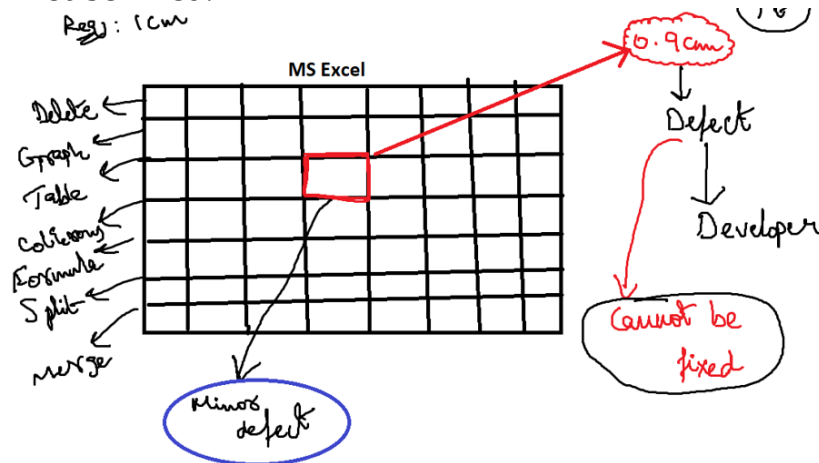


III. What is cannot be fixed status?

Here the developers are accepting that it is a defect but they are not in a position to fix it due to some reason.

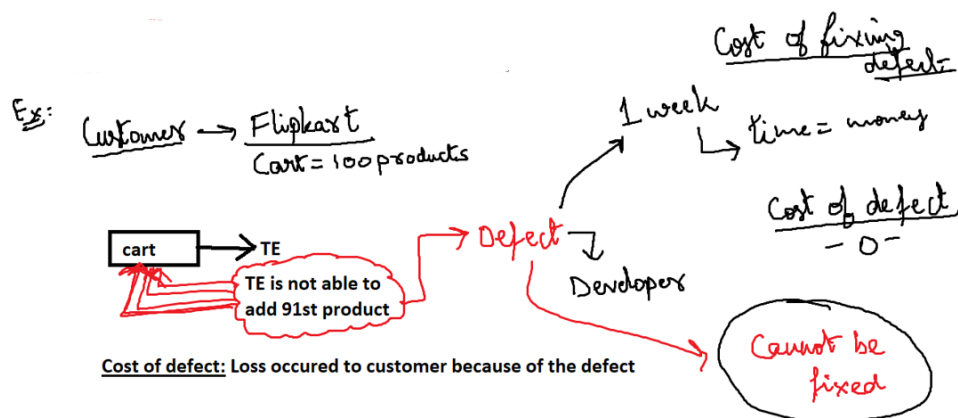
Why we get Defect cannot be fixed?

- i. If TE finds a defect in the root of the product and if it is a minor defect developer says defect cannot be fixed.



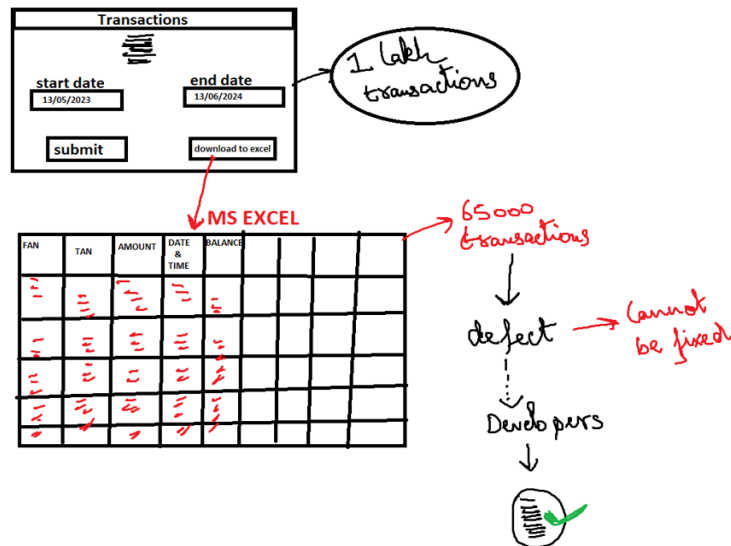
- ii. If the cost of fixing the defect is more than cost of defect then developer says defect cannot be fixed.

(Here the cost of defect means loss in the business because of having bug in the software)



- iii. When the technology itself doesn't support.

Customer $\rightarrow$ Citibank

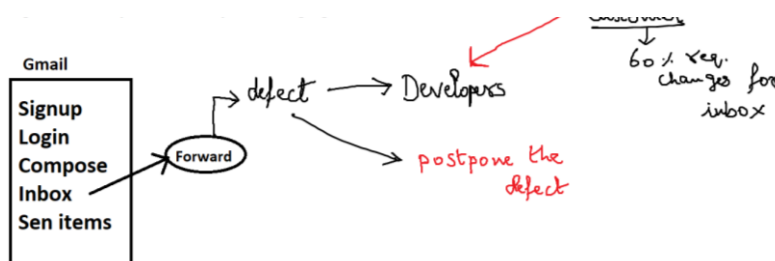


IV. What is Postpone status?

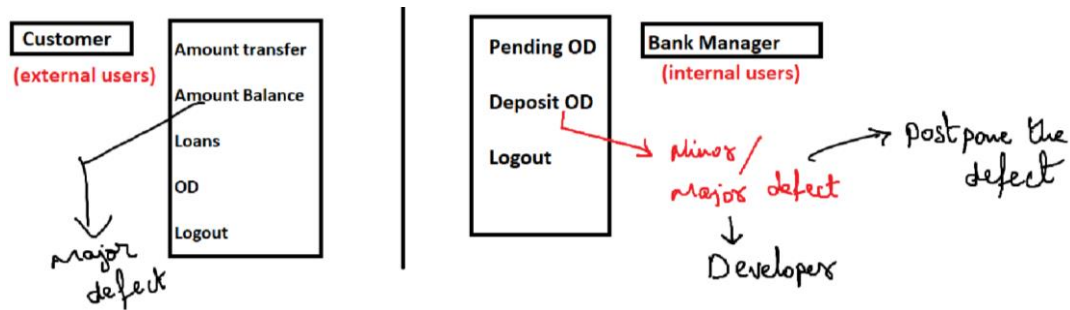
Here developers are accepting that it is defect but they want to fix it a little later. In this case developers gives status as Postpone.

Why we get Postpone status?

- i. If TE finds minor defect and if developer is not having sufficient time to fix the defect in this case developer will give Postpone status.
- ii. If TE finds a defect in a feature where customer wants to do lot of changes in the requirement (same feature) and even customer might remove feature as well. In this case developer will give Postpone status.



- iii. TE finds a defect in the feature which is exposed to internal users and if it is a major or minor defect then developer will give the postpone status.



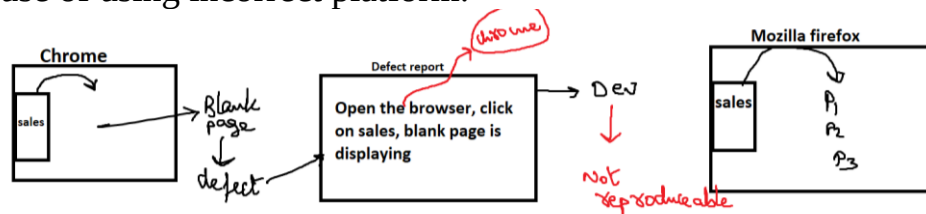
V. What is Not Reproducible status?

TE is able to see the defect but developer is not able to see the same defect. In this case developer says that issue not reproducible.

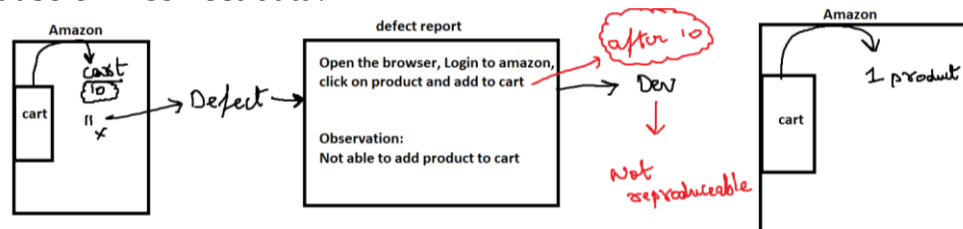
Why we get not reproducible status?

i. Because of improper defect report

- a. Because of using incorrect platform:



- b. Because of incorrect data:



ii. Because of inconsistent defect

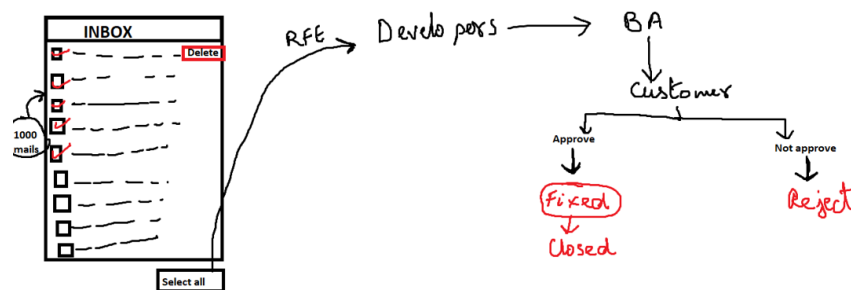
There is a bug, sometimes it comes up and sometimes it disappears and we don't know when it comes up and when it disappears. In such a state it is called as Inconsistent bugs.

How do you convince the developer that it is a defect whenever they say not reproducible?

1. I will send the proper information like data, OS, Browser used
2. I will write the proper procedure. If still dev is not able to reproduce, I will send the screenshot, Then I will give a demo of how I'm getting the defect by asking him to connect to my computer.
3. Still if it is not reproducible, I will try to reproduce in 4-5 computers around me. If still I'm not getting the bugs, I will escalate it to test lead

VI. What is Request for Enhancement (RFE)?

There is a problem but it is not a part of the requirement then it is called Request for Enhancement (RFE) or Change Request (CR).



DEFECT REPORT

1. Defect id 8109	15. Brief description/Defect summary: When user click on Compose, blank page is displayed
2. Project Name: Gmail	16. Detailed description:
3. Release Name: TIGER	OS: Windows OS Version: 11 Browser: Chrome Browser Version: 10 <u>Steps to reproduce the defect:</u> Step 1: Open the browser Step: Enter the URL Step 3: Enter the USN and PWD & click on Login Step 4: Click on Compose button Observation: Blank page is displayed
4. Requirement No. : 12.3	17. Attach screenshot
5. Module name: Compose	18. Expected Result: Compose page should be displayed
6. Build ID: Build B021	19. Actual Result: Blank page is displayed
7. Status: New/Open	20. CC: -----
8. Severity:	
9. Priority:	
10. Reporter: Niveditha@qspiders.in	
11. Assign to: DL email id/ DE email id	
12. Test case name: Gmail_Compose_TC1	
13. Test data: N/A	
14. Test URL: https://qa_gmail/login.jsp	
Submit Reset Bug history Cancel	

What is Defect Tracking Tool? Name some defect tracking tools.

Defect tracking tool is a software which is mainly used to track or store the defects in centralized place and communicate defect to developer in an organized way.

Defects tracking tools are:

1. Bugzilla
2. QC/ALM
3. JIRA
4. Mantis
5. Bugnet

SEVERITY

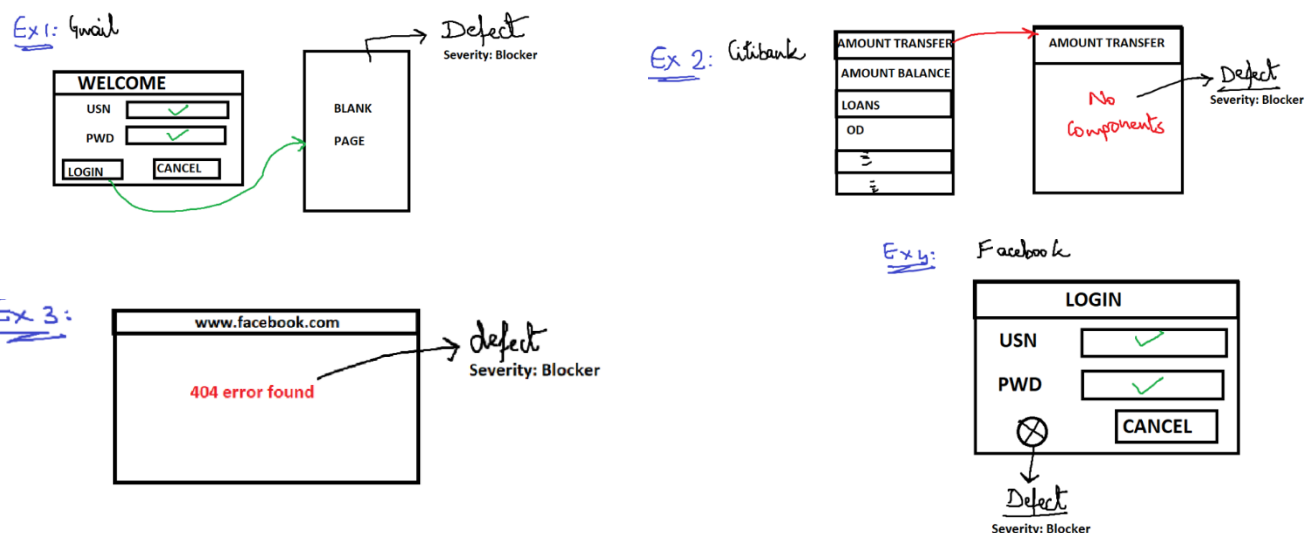
It is the impact of the defect on customer business workflow.

There are 4 levels of Severity: -

1. Blocker defect
2. Critical defect
3. Major defect
4. Minor defect

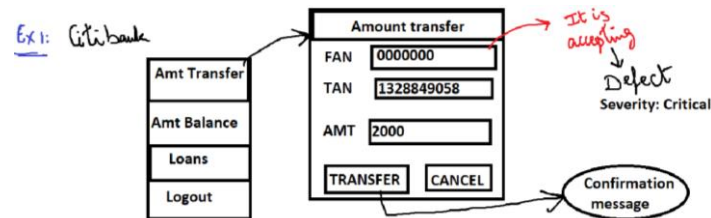
1) Blocker:

Assume that there is a defect in the software and because of this defect TE is completed blocked to test his respective module and TE cannot continue testing that module, these kinds of defects are called blocker defects.



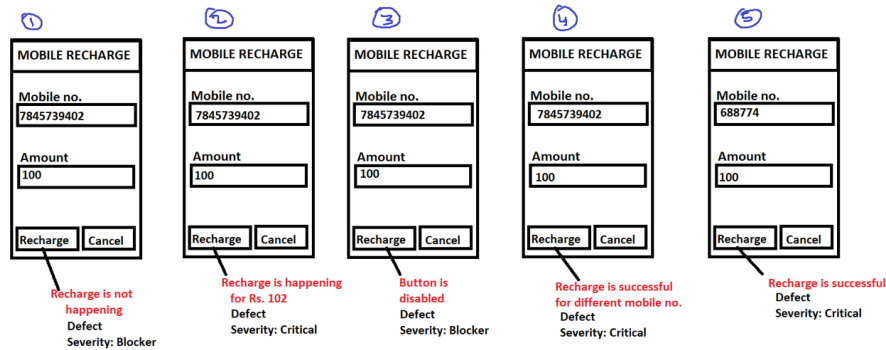
2) Critical:

Assume that there is defect in the application, I am 100% sure that this defect will affect customer's business work flow but here the test Engineer is not blocked, we can continue that module. These kinds of defects are called critical defects.



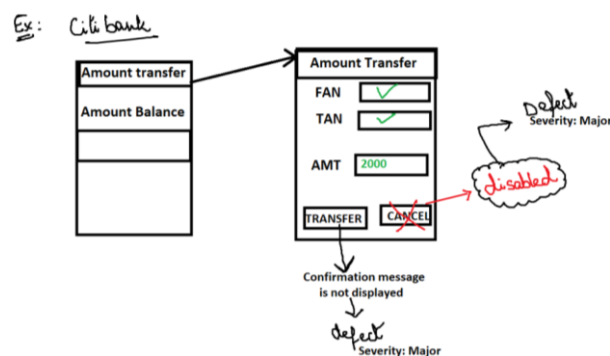
Ex: Mobile recharge

Req: If you recharge for Rs. 100, you should get Rs. 100 full talk time.



3) Major:

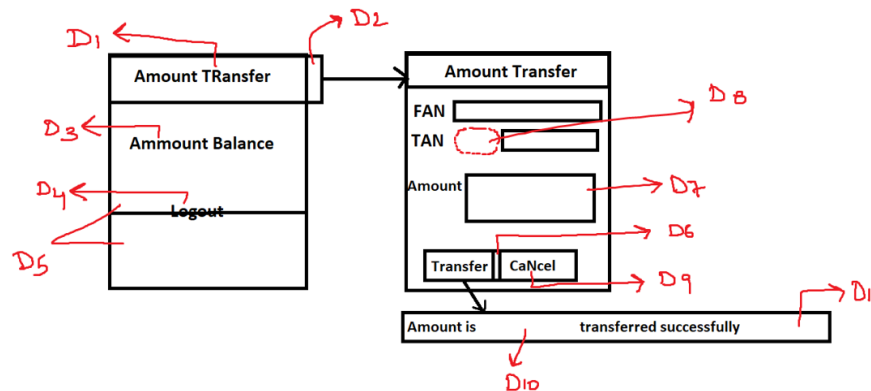
Assume that there is a defect in the application but we are not sure that how this defect is going to affect customer business work flow, this type of defects is called Major Defect.



4) Minor Defect:

Assume that there is defect in the application, I am 100% sure that this defect will never affect customer business work flow, this type of Defect is called Minor defect.

Example: Spelling mistake, color mistake, overlapping issue, alignment issue etc.



PRIORITY

- Importance given to fix the defect is called as Priority (OR) How soon the defect must be fixed by the developer.
- There are 3 levels of Priority:
 1. High or P1
 2. Medium or P2
 3. Low or P3

High or P1:

If the defect is having priority as high/ P1, then the developer should fix the defect immediately.

Medium or P2:

If the defect is having priority as medium/ P2, then he can fix the defect, within some test cycle or builds or within a release

Low or P3:

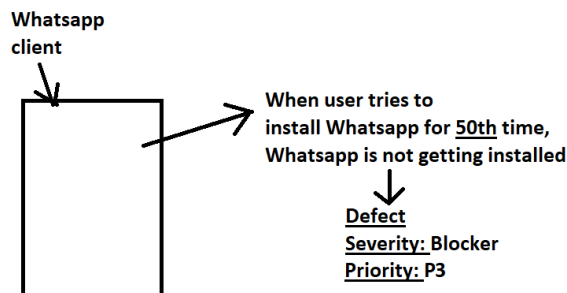
If the defect is having priority low/ P3 then developer can fix the defect in upcoming release or within some 2-3 release.

There are 4 combinations:

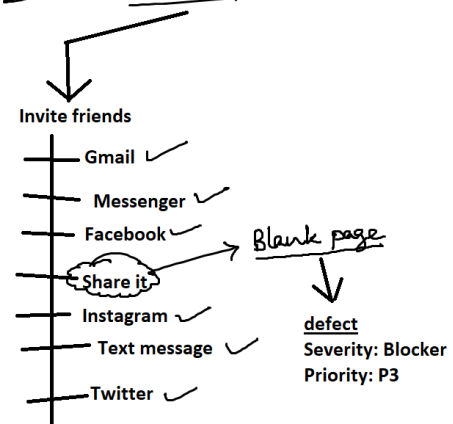
1. High severity and low priority
2. Low severity and high priority
3. High severity and high priority
4. Low severity and low priority

Example for High severity and low priority:

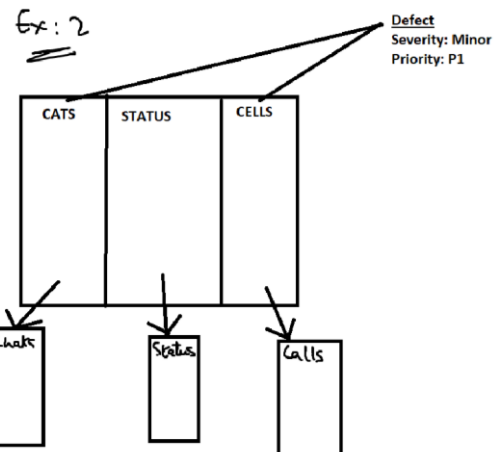
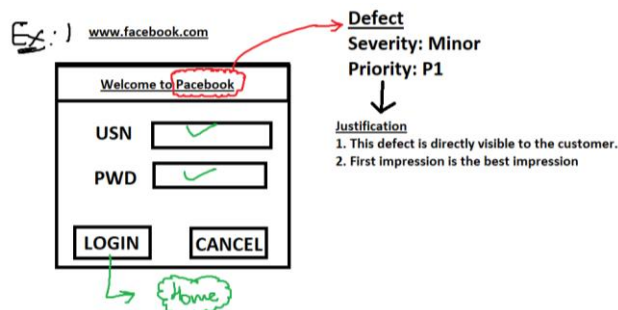
Ex: 1 Whatsapp



Ex: 2: Whatsapp



Example for Low severity and high priority:



Defect Triage:

When time is very less and there are too many pending bugs to be fixed they will categorize the defects into different status

1. All the business-critical defects are moved to assigned status
2. There are some defects which has to be fixed but not on urgent basis, they move to postponed status
3. There are defects which need not to be fixed they move such defects to cannot be fixed status

This type of grouping defects is known as triage

Defect masking:

If any defect is hiding another defect, it is called as defect masking.

Defect seeding:

Intentionally introducing defects in the software to check the efficiency of test engineer is called as defect seeding

TEST PLAN

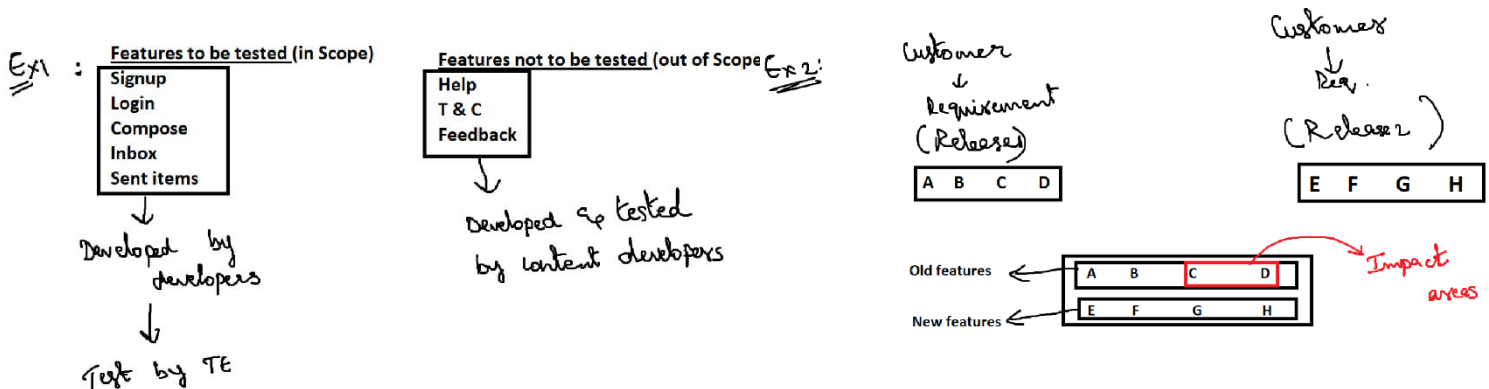
- It is a document which drives all the future testing activities.
- It is generally prepared by Test manager/Test lead/Sr TE
- It has got different sections like:
 1. Objective
 2. Effort Estimation
 3. Scope
 4. Approach
 5. Assumption
 6. Risk
 7. Backup plan/ Mitigation plan
 8. Testing methodology
 9. Schedule
 10. Test environment
 11. Defect tracking
 12. Test automation
 13. Deliverables
 14. Entry and exit criteria
 15. Test stop criteria
 16. Role and responsibilities
 17. Templates

1) **Objective:** This section covers aim of preparing the test plan

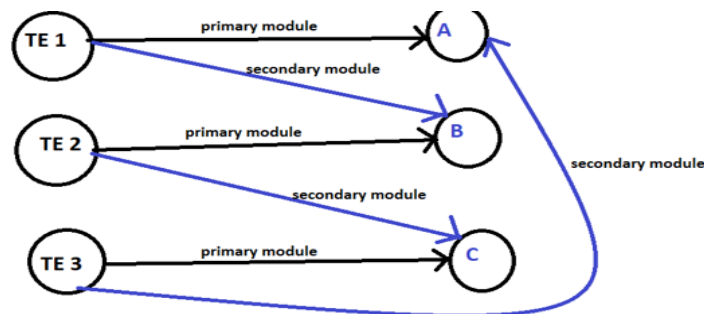
2) **Effort estimation:** This section covers estimation of how long it will take to complete the project & also we estimate how many engineers are needed & the cost needed to complete the task and the cost of testing.

feature	system study	identify scenarios	write tc	review tc	fix review comments	verify the fix	test execution	defect tracking	Report	Total
compose	4 hrs	50 scenarios 5 min to identify 5 min x 50 = 250 min 4 hrs	5 min to write 5 min x 50 = 250 min 4 hrs	4 min to review 50 x 4 min = 200 min 4 hrs	20% of mistakes are expected. 20% of 50 10 mistakes 10 min to correct 10 min x 10 = 100 min 2 hrs	5 min to verify 5 min x 10 = 50 min 1 hr	13 hrs	20% tc failed 10 * 2 hrs 20 hrs	2 hrs * 5 cycles = 10 hrs	= 62 hours

- 3) **Scope:** This section covers what are the features to be tested and what are the features not to be tested.



- 4) **Approach:** This section covers how we are going to test the product in future.
- 5) **Assumption:** This section covers assumptions that we have made while planning.
- 6) **Risk:** This section covers if any assumptions fail that becomes risk.
- 7) **Backup Plan/ Mitigation plan:** This section covers how to overcome (or) how to face the risk.



According to the example, assume that 3 TE are hired to work on 3 modules of the project A, B, C respectively. Assume due to some reason, TE1 quits the job, then we might be in risk. In order to overcome this, we initially assign the TE with a secondary module, so that all the modules are tested before releasing s/w to customer

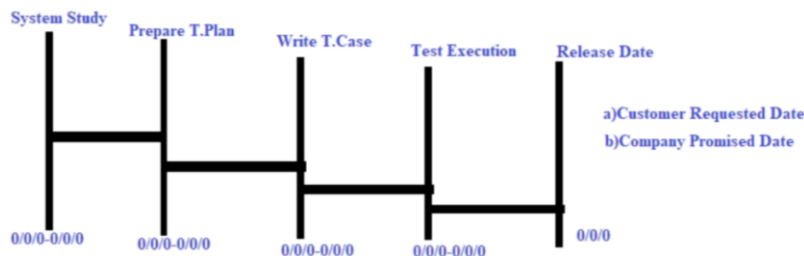
- 8) **Test methodology:** This section covers what are the types of testing that we are planning to conduct.

Ex: Standalone application

Note: We don't do
Performance Testing for
Standalone applications

Smoke	Compatibility Testing
FT	Regression Testing
IT	Accessibility Testing
ST	Globlization Testing
Adhoc	
Usability Testing	

9) Test Schedule: This section covers when exactly we should start and end and activity.



10) Test Environment: This section covers how we go about setting up the test environment in future (or) how we setup the environment in future.

Ex: 12.1 Procedure to install the build

12.2 Hardware

12.2.1 *Server side*

HP starcat 1500

12.2.2 *Client side*

6 computers with following configurations

1 gz speed Intel processor

1 GB RAM

12.3 Software

12.3.1 *Server side*

OS: Linux, Version:

Web server: Tomcat, Version:

App server: Web sphere

DB server: Oracle, version:

12.3.2 *Client side*

OS: Win 10, win 8

Browser: Mozilla Firefox, Chrome

11) Defect Tracking: This section covers in future when we find defects how it should be tracked and also covers the procedure, status, severity and priority.

11.1) Procedure to track the defect

11.2) Severity

Blocker
Critical
Major
Minor

11.3) Priority

High
Medium
Low

11.4) Defect tracking tool that we are planning to use

12) Test Automation: This section covers what are the features to be automated & what are the features not to be automated & the complete automation strategy.

Ex: 6.1 Features to be Automated

6.2 Features not be automated

6.3 Automation framework you are planning to use

6.4 Automation tool that you are planning to use

Selenium

13) Deliverables: This section covers which all documents that has to be provided by the testing team at the end of test cycle.

Ex: Test cases, Traceability matrix, test execution report, defect report, release note, graphs & matrices.

Release Note: Last day along with the product, we release a note to the customer called as release note.

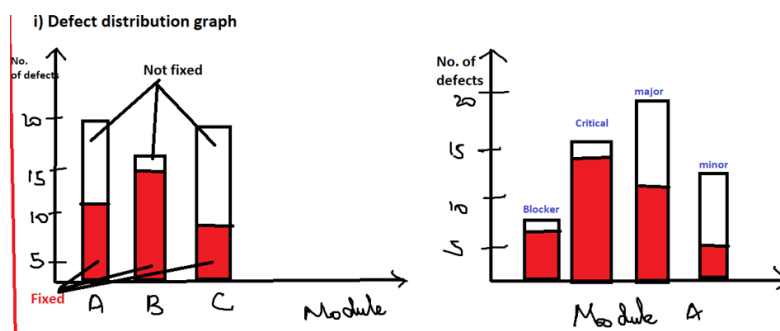
Release note consists of:

1. List of open defects which are there in the product.
2. List of bugs that are found in the previous release production and fixed in the current release.
3. List of pending bugs in previous release & fixed in current release
4. List of features added, modified or removed in current release.
5. Procedure to install the software.
6. Version of the product.

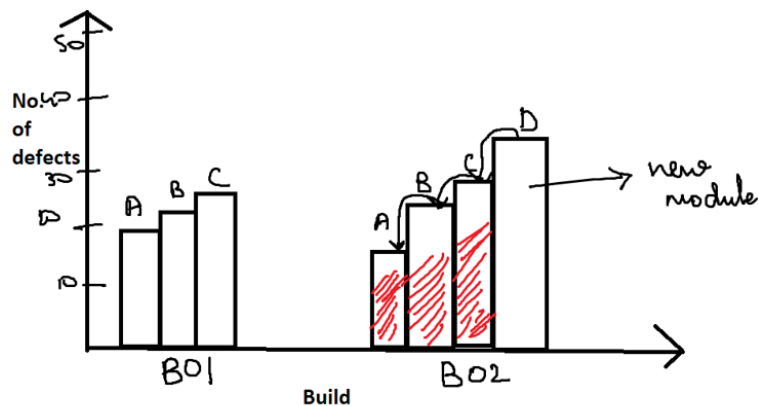
Graphs & matrices:

Graphs:

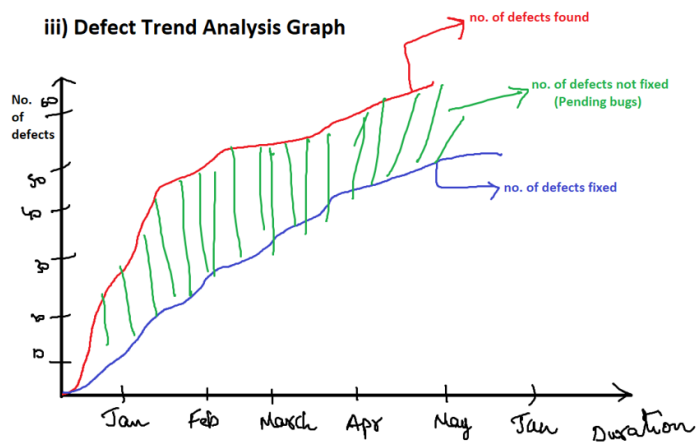
a. Defect density (or) defect distribution graph



b. Build-wise defect distribution graph



c. Defect trend analysis graph

Metrices:

It is the past data collected between two parameters so that we can analyze and take better decision to perform future activities.

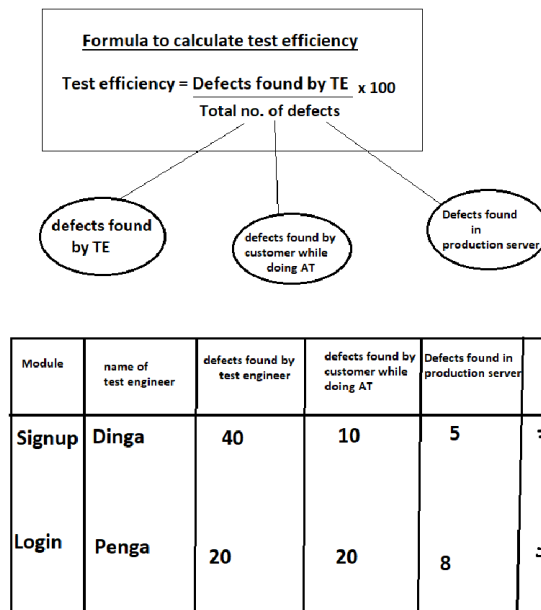
name of TE	Blocker		Critical		Major		Minor	
	No.of defects found	No.of defects fixed	No.of defects found	No.of defects fixed	No.of defects found	No.of defects fixed	No.of defects found	No.of defects fixed
Durga	20	10	30	20	50	25	100	15
Panga	40	20	60	25	70	65	75	25

❖ We can generate n no. of matrices.

Ex: Defect distribution metrics

Productivity metrics

Test engineer efficiency metrics



14) Entry & Exit Criteria:

Entry criteria: This section covers list of criteria that should be met to start the activity.

Exit criteria: This section covers list of criteria that should be met to say that activity is over.

Ex:

System study:

Entry criteria for system study:

1. Should have got approved requirement from customer
2. Should have assigned engineers to do system study

Exit criteria for system study:

1. Should have completed reading the requirement.

2. Should have got answers for all the queries.

Prepare Test Plan:

Entry criteria for test plan:

1. Test plan template should be ready.
2. Should have assigned someone to prepare test plan & review the test plan.
3. Should have met the exit criteria of system study.

Exit criteria for test plan:

1. Should have got approval for test plan.

Write Test case:

Entry criteria for writing test case:

1. Test case template should be ready.
2. Should have met exit criteria of test plan.
3. Should have assigned the module to engineer.

Exit criteria for writing test case:

1. Test case should be reviewed, approved & stored in repository.

15) Test stop criteria: This section covers when exactly we should stop testing.

We stop testing when the product quality is very good or product quality is very bad

- Product quality is very good means if all the end-to-end business scenarios are working fine.
- If there are no blocker or critical defects.
- There are few bugs left out which are all minor or major but are less than the acceptable limit set by the customer.
- If all the features requested by the customer are ready
- Product quality is bad means there are too many blocker & critical bugs.
- If it is crossing the budget.
- If it is crossing the schedule/deadline.

16) Roles & responsibility: This section covers what each engineer should do in different stages of test life cycle.

Roles and responsibilities of Test manager:

- Write and review Test plan.
- Interact with testing team, development team, if needed with customer.
- Should handle all the issues and escalations.
- He should approve release note.

Roles and responsibilities of Test lead:

- Write and review Test Plan.
- Allocate work to each engineer and make sure that they are going to work and complete the task within the schedule.
- Consolidate all the reports which are sent by every TE and communicate with testing team, development team, project manager and customer.
- He conducts impact analysis meeting.

Roles and responsibilities of Test Engineer:

- Write Test case
- Review test case of another test engineer.
- Execute test case for his allocated features.

17) Templates: This section covers formats for all the documents that we are planning to prepare in the entire test life cycle.

- i. Test case template
- ii. Traceability matrix template
- iii. Defect report template
- iv. Test case review template
- v. Test execution report template

1. Who will write the test plan?

Test Lead/ Test Manager

2. Who will review the test plan?

Customer, development team or test engineers

3. Who will approve the test plan?

Customer/ Project Manager

TEST PLAN TEMPLATE

Ex: Citibank online test plan

Page - 01 Revision History					
Version	Author	Reviewed by	Approved by	Comments	Approval date
V 1.0	Dinga	Penga	Manga	In this version, we have Amount Transfer, Amount Balnace feature	19-06-2023
V 1.1	Penga	Dinga	Manga	This version consists of Amount Transfer, Amount balance, Loans & OD	17-07-2023

Page - 02 Table of Contents	
1. Objective-----	4
2. Effort Estimation-----	6
3. Scope-----	10
4. Approach-----	15
5. Assumption-----	20
6. Risk-----	23
7. Backup plan-----	28
8. Testing methodologies-----	32
9. Schedule-----	35
10. Test Environment-----	39
11. Defect Tracking-----	44
12. Test automation-----	47
13. Deliverables-----	50
14. Entry and Exit criteria-----	56
15. Test stop criteria-----	62
16. Roles and responsibilities-----	66
17. Templates-----	70

Page - 80 References	
1) <u>CRS</u>	
2) <u>SRS</u>	

TEST CASE

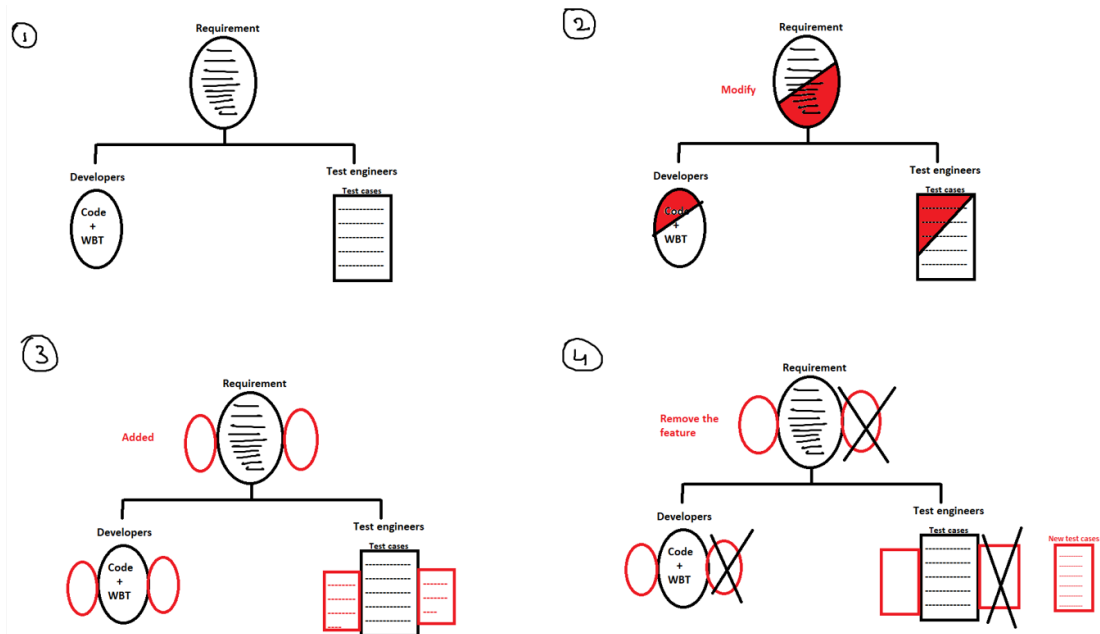
Test case is a document that covers all possible scenarios for a specific requirement. It contains different sections like Step number, Input, Expected result, Actual result, Status, and Comments.

What is the drawback of seeing the requirements and testing the software?

1. There will be no consistency in testing if you look into requirements and test the software.
2. Quality of testing varies from person to person
3. Quality of testing depends on the memory power of the test engineer.
4. Quality of testing depends on the mood of the T.E.

When do we write test case?

1. When developers are busy in building the product, the testing team will be busy writing the test cases.
2. When the developers will modify or change the feature, parallelly T.E. will modify or change the test cases.
3. When the developers will add the features parallelly test engineers will add new test cases.
4. When the customer is removing the requirement, developers will remove the feature, and parallelly test engineer will remove the test cases to make sure that features are removed from the s/w or not.



Why do we write Test cases?

1. We write test cases to have better test coverage.
 - a. When the requirement comes in developers are busy building the product same time test engineers are free, so they identify all possible scenarios and document it.
 - b. When the build comes we can spend time executing the scenarios, because of this no. of scenarios that you are covering will be more.
2. To have consistency in test execution.
 - a. It means if you have documented the scenarios, you can make sure that you are executing all the scenarios in all the test cycles, sprints, or releases.
3. To depend on the process rather than person.
4. To avoid training every new engineer on the product or on the requirement.
5. Test case is the only document that acts like as proof for customer, development team, and also manager that we have covered all possible scenarios.
6. Test case acts like a base document for writing automation scripts, if you refer to the test case and write automation scripts you can ensure the same kind of coverage even in automation.
7. If you documented the test case, no need to remember the scenarios.
8. If you have documented the test cases, test execution happens in a very organized way.
9. If you have documented the test cases, the time taken to execute is much less.

TEST CASE DESIGN TECHNIQUES

It is a technique which is used while writing test case in order to improve test coverage.

Types of Test case design techniques:

1. Error guessing
2. Equivalence class partitioning
3. Boundary value analysis (BVA)

1. Error Guessing:

Here we guess all possible errors and we derive the scenarios.

We guess errors based on:

- i. Requirement
- ii. Experience
- iii. Intuition

The form is titled 'Amount Transfer'. It contains three input fields: 'FAN', 'TAN', and 'Amount'. Below these fields are two buttons: 'Transfer' and 'Cancel'. Red handwritten annotations are present next to the 'Amount' field, with arrows pointing to it. The annotations include: 'Blank', '100.12', '-100', '#?', 'ten', and '1,000'.

2. Equivalence class partition:

a. Pressman Rule:

Rule 1: If the input is a range of values, then design test case for one valid and two invalid inputs.

Ex:-
Req:- 100 - 5000
Range = [100 - 5000]

1 valid → 500
2 invalid → 90
 → 6000

Rule 2: If the input is in a set of values, then design test case for one valid and two invalid inputs.

Ex:-
Online Shopping
Product id
Submit Cancel
Printer = 10
Scanner = 20
Webcam = 30
Set = [10, 20, 30]

1 valid → 20
2 invalid → 25
 → 40

Rule 3: If the input is in Boolean, then design the test case for both true and false values.

Ex:

Male ☒
 Female ☐

Select this and test (true)
 unselect this and test (false)

☒ BCA
☐ Bcom
☐ Bsc
☐ Btech

Select $\Rightarrow$ True
 Deselect $\Rightarrow$ False

b. Practice Method:

If the input is in range of values then divide the range into equivalent parts and test for all the values, make sure that you are testing for at least two invalid values.

```
if(Amount 100 - 2000)
{
    deduct 1%
    transfer
}

else if(Amount 2001 - 5000)
{
    deduct 2%
    transfer
}

else
{
    throw error message
}
```

Requirement

1. Should accept b/w 100- 5000
2. 100 - 2000-----> 1% deduct
3. 2001 - 5000-----> 2% deduct

90
 100
 500
 1000
 2000
 3000
 4000
 5000
 6000

500 ✓
 50 ✓
 6000 ✓

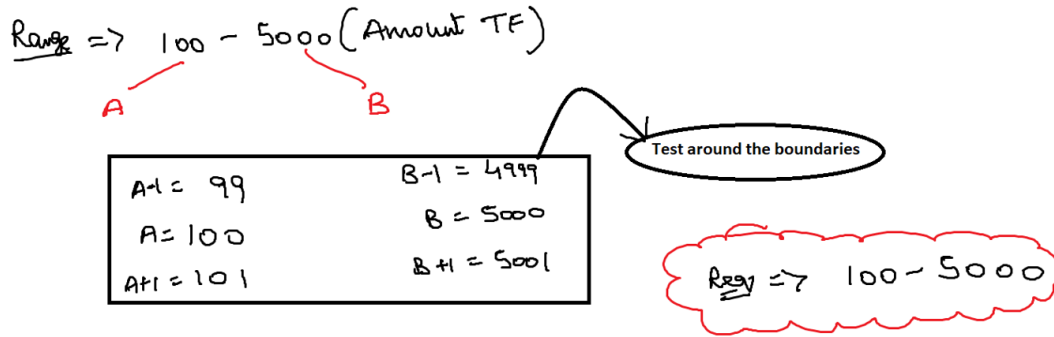
6000 ✓

Note:

1. If there is a deviation between the range of values then we go for Practice method.
2. If there is no deviation between the range of values then we go for Pressman rule.
3. By looking into requirements, we will get to know whether there is a deviation or not.

3. Boundary Value Analysis:

If the input is range of values b/w A to B then design test case for A, A+1, A-1 and B, B+1, B-1.



Why we catch more bugs around the boundary?

```
if(Amount > 100 && Amount < 5000)
{
    Transfer
}
else
{
    Throw error msg
}
```



According to this example, developers will do lot of mistakes while writing code for conditional/ logical operators and that's why chances of finding defects will be more around the boundary.

Test case optimization:

The process of removing the duplicates from the test cases (OR) Removal of repetition of test cases is called test case optimization. We can cover both 1 valid and 2 invalids in BVA itself, so we can skip equivalence partitioning (in only specific cases).

Difference between test case & test scenarios:

<i>Test scenarios</i>	<i>Test cases</i>
It is a high-level document of all the customer business workflow according to the customer's requirement.	It is a detailed document of the scenario that helps us to test the application.
We write Test scenarios by looking into the requirements.	We write test cases by looking into both requirement and test scenarios.
By looking into test scenarios, we can't test any application until you have good product knowledge.	We can test any application by looking at the test case, no matter if you have product knowledge or not.
Here we mention what to test.	Here we mention how to test.

TEST CASE TEMPLATE

- Every TE will write test case in Test case Template only
- Test Case can be prepared in “Test Case Management Tool” or “MS Excel”
- Test case template is divided into 3 sections:
 1. Header
 2. Body
 3. Footer

1. HEADER

1. Test case name:-
 2. Requirement no:-
 3. Test data:-
 4. Pre condition:-
 5. Severity:-
 6. Test case type:-
 7. Brief description:-

2. BODY

Step No.	Description	Input	Expected result	Actual Result	Status	Comments



While execution or after execution we need to fill

3. FOOTER

1) Author:-
 2) Reviewed by:-
 3) Approved by:-
 4) Approval date:-

How to fill the header?

1. *Test case name:*
 Format: Projectname_ModuleName_scenario
Ex: CB_AmountTransfer_integration
2. *Requirement Number:*
 BA when he converts CRS to SRS, in SRS for each requirement he will write the requirement no.

Ex: 30.1 Amount Transfer

30.1.1 FAN text field

30.1.2 TAN text field

30.1.3 Amount text field

3. *Test data:*

It is the data written by a TE and has to be done before the test execution.

Ex: TE should have UN, PWD, URL, a/c number

4. *Pre-condition:*

It is a set of actions or settings which should be ready/done by TE before executing the 1st test case.

Ex: User should have balance in his account.

5. *Severity:*

TE will give severity for every individual test case, based on how important and complex the feature is from customer's point of view.

TE will execute test case based on severity.

There are 3 types of severity for Test cases: Critical/ Major/ Minor

Features	Test case	Severity	Order of execution
Login	TC 1	Critical	TC 1
Drafts	TC 2	Major	TC 5
Inbox	TC 3	Critical	TC 3
Trash	TC 4	Minor	TC 2
Compose	TC 5	Critical	TC 4

6. *Test case type:*

Here the TE mention what type of test case he is writing.

Ex: Functionality test case, Integration test case, system test case.

7. *Brief Description:*

It describes about the complete test case and the behavior of the test case.

Ex: In the amount transfer module, it should accept only +ve integers.

How to fill the footer?

1. Author: Person who writes the test case

Ex: Dinga

2. Reviewer: Person who reviews the test case

Ex: Dingi

3. Approved by: Person who approves the test case

Ex: Test lead

4. Approval date:

Ex: 01-01-2023

Lessons to remember while writing test cases:

1. Before we actually write test cases, we should come up with options and select the best option out of it.
2. Always use should be/must be in expected results. Don't use maybe/ may not be, can be, cannot be.
3. Elaborate only those steps in which you have to focus. Don't elaborate all the steps.
4. Start writing the test case with navigation steps.
5. Never write hardcore test cases, always write generic test cases.
6. While writing test cases we should imagine/visualize the application.
7. If you organize the scenarios properly, the total no. of steps can be reduced
8. Whatever we have covered in FT, don't cover the same in Integration test cases. If something is covered in integration test cases then don't cover it in the system test cases.

How to write Functionality Test Case?

1. Test case name: citibank_amount transfer_functionality scenarios 2. Requirement no: 30.1 Amount Transfer 3. Test data: TE should have UN, PWD and account no. of users 4. Pre condition: Check for internet connection Check whether s/w is installed or not 5. Severity: Critical 6. Test case type: Functionality test cases 7. Brief description: Following test cases covers functionality scenarios for Amount transfer						
Step no.	Description	Input	Expected Result	Actual Result	Status	Comments
1	Open the browser and enter the test URL	https://<server name>cb.com	Welcome page should be displayed			
2	Enter valid UN and PWD and click on Login button	UN: Valid UN PWD: Valid PWD	Home page should be displayed			
3	Click on Amount Transfer link	N/A	(a) Amount transfer page should be displayed (b) Following components should be displayed: 1. FAN TF 2. TAN TF 3. Amount TF 4. Transfer button 5. Cancel button			
4	Following are the steps to test FAN text field					
4.1	Enter 10 digit valid a/c no. into FAN text field and click on Transfer button	valid 10 digit a/c no (6789745289)	Error message should not be displayed for FAN text field			
4.2	Enter character into FAN TF and click on Transfer button	ABCDGH	Appropriate error message should be displayed			
4.3	Enter special character into FAN TF and click on Transfer button	&*#	Appropriate error message should be displayed			
4.4	Enter decimal value into FAN TF and click on Transfer button	100.89	Appropriate error message should be displayed			
4.5	Enter combination of special characters and alphabets into FAN TF and click on Transfer button	ghsgs@#\$	Appropriate error message should be displayed			
4.6						
4.7						
4.8						
4.9						
4.10	Enter 10 digit invalid a/c no into FAN TF and click on Transfer button	22222 22222	Appropriate error message should be displayed			
4.11	Enter 9 digit invalid a/c no into FAN TF and click on Transfer button	123456789	Appropriate error message should be displayed			
4.12	Enter 11 digit invalid a/c no into FAN TF and click on Transfer button	12345678901	Appropriate error message should be displayed			
5	Following are the steps to test TAN text field (make sure that FAN text field is having valid data)					
5.1						
5.2						
5.3						
5.4						
5.5						
5.6						
5.7						
5.8						
5.9						
5.10	Enter same a/c no which is entered for FAN text field into TAN text field and click on Transfer button	6789745289	Appropriate error message should be displayed			
6	Following are the steps to test Amount text field (make sure that both FAN & TAN text field is having valid data)					
6.1	Enter valid amount into amount text field and click on transfer button	100	Confirmation message should be displayed			
6.2	Click on browser back button	N/A	Home page should be			
6.3	Enter decimal value into amount text field and click on transfer button	100.5	Appropriate error message should be displayed			
6.4	Enter alphabets with numbers into amount text field and click on transfer button	RS 100	Appropriate error message should be displayed			
6.5	Enter -ve integer into amount text field and click on transfer button	-100	Appropriate error message should be displayed			

1. Go to the body of test case. Start with navigation steps.
2. Take first field
 - i. Start with valid data
 - ii. Write error guessing scenario
 - iii. Write equivalence partitioning scenario
 - iv. Write all BVA scenarios
3. Take second component
 - i. Start with valid data
 - ii. Write error guessing scenario
 - iii. Write equivalence partitioning scenario
 - iv. Write all BVA scenarios
4. -----

[illegible]

How to write Integration test cases?

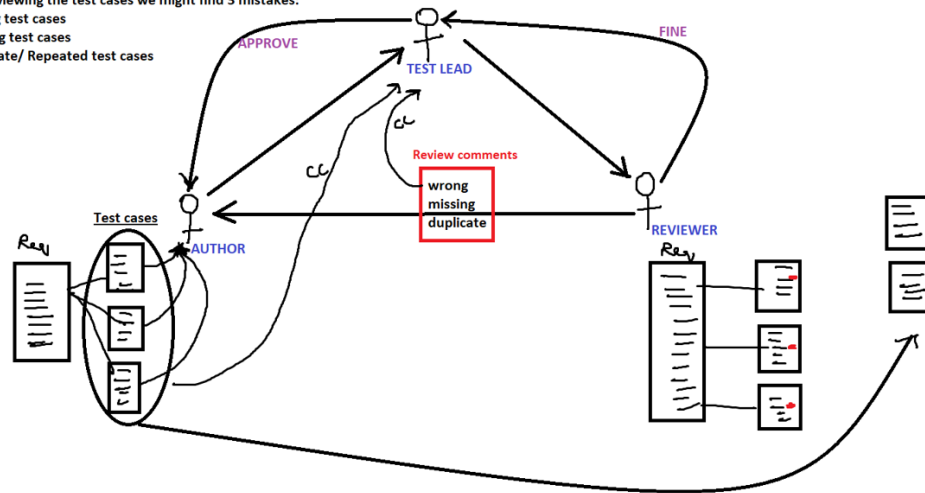
Scenario 1: Login as A, transfer the money, check whether the amount is deducted in amount balance of A					
Step no	Description	Expected Result	Actual Result	Status	Comments
1	Open the browser and enter the URL https://<servername>.cb.com	Welcome page should be displayed			
2	Login as user A with valid UN and PWD	Home page should be displayed			
3	Click on Amount Transfer link	Amount transfer page should be displayed			
4	Enter the user A's acc no into FAN text field and B's acc no in TAN text field and 1000 into Amount text field and click on transfer button	Confirmation page should be displayed			
5	Click on Amount Balance link	a) Amount balance page should be displayed b) Balance amount should be old balance - transferred amount			
Scenario 2: Login as A, transfer the amount, transferred money should be recorded in transaction of A					
6	Click on Transaction link	a) Transactions page should be displayed b) Transferred amt, Date, Time and user B's acc no should be displayed			
Scenario 3: Login as A, transfer money to B, transferred money should be added to B's balance					
7	Click on Logout link	Welcome page should be displayed			
8	Login as user B with valid UN and PWD	Home page should be displayed			
9	Click on Amount Balance link	a) Amount balance page should be displayed b) Balance amount should be old balance + transferred amount			
Scenario 4: Login as A, transfer money to B, transferred money should be recorded in transaction of B					
10	Click on Transaction link	a) Transactions page should be displayed b) Transferred amt, Date, Time and user A's acc no should be displayed			

How to write System test cases?

Step no.	Description	Expected Result	Actual Result	Status	Comments
1	Open the browser and enter the test URL https://<server name>.cb.com	Welcome PSBD			
2	Login as User A with valid UN and PWD	Home PSBD			
3	Click on OD link	OD PSBD			
4	Click on apply OD	Apply OD PSBD			
5	Enter valid amount (20000) and click on Apply button	Confirmation PSBD			
6	Click on Logout link	Welcome PSBD			
7	Login as Manager	Home PSBD			
8	Click on pending OD link	Pending OD PSBD			
9	Select the customer and click on Approve OD button	Confirmation PSBD			
10	Click on Logout link	Welcome PSBD			
11	Login as User A with valid UN and PWD	Home PSBD			
12	Click on OD link	OD PSBD			
13	Click on OD balance link	a) OD balance PSBD b) 20000 blance amt should be displayed			
(change the server date to next 30 days)					
14	Click on Logout link	Welcome PSBD			
15	Login as Manager	Home PSBD			
16	Click on Deposit OD link	Deposit OD PSBD			
17	Enter valid amt (650) and click on deposit button	Confirmation PSBD			
18	Click on Logout link	Welcome PSBD			
19	Login as User A with valid UN and PWD	Home PSBD			
20	Click on OD link	OD PSBD			
21	Click on Repay OD link	Repay OD PSBD			
22	Enter valid amt (20650) and click on repay button	Confirmation PSBD			
23	Click on OD balance link	a) OD balance PSBD b) OD blance need to be repaid should be 0000			

TEST CASE REVIEW PROCESS

While reviewing the test cases we might find 3 mistakes:
 1. Wrong test cases
 2. Missing test cases
 3. Duplicate/ Repeated test cases



On what basis you assign someone to review test case?

They will assign to the person:

1. Who is working on a similar or related module in the project.
2. Who has worked on same module in the previous project.
3. Who has been working in the project since the beginning knows every corner of the product.
4. Who is responsible, and who will understand the requirement very fast and identify more mistakes.

Review ethics:

1. Always review the content, and not the author.
2. Reviewer should spend more time in finding the mistakes rather than giving the solution for it
3. Even after review if there are still any mistakes, both author and reviewer are responsible

Why we review test case (OR) if I give you the test cases what is the approach to review the test cases (OR) what kind of mistakes will you find while reviewing the test case?

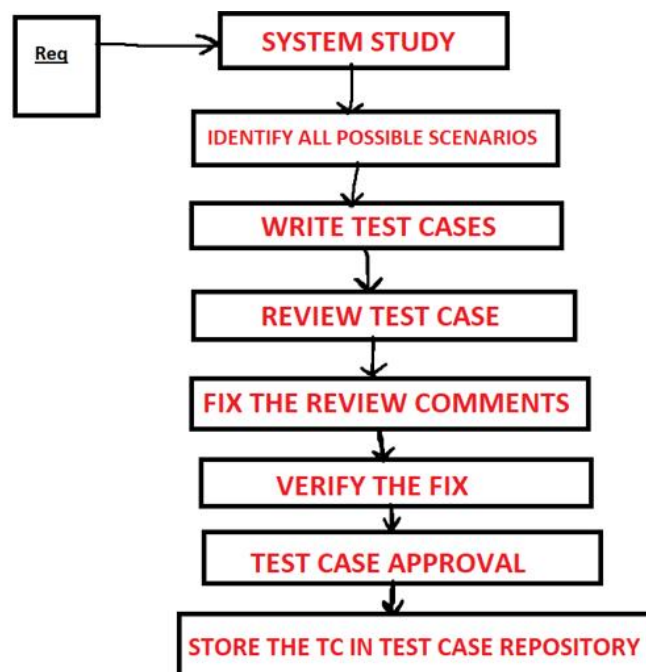
1. I will look into the header of the test case and understand the req for which test case is written and then go to body of the test cases and try to find:
 - a. Missing scenarios
 - b. Wrong scenarios
 - c. Repeated scenarios

2. I will check whether organized or not so that when executed it should take less time
3. I will check whether it is simple to understand so that the it is given to ne TE he should be able to execute without asking any questions
4. I will look into the header of the test case and try to find:
 - a. All the attributes are covered or not
 - b. Check whether content in all the attributes are cover or not
5. I will check whether test case format is according to standers defined in the project

TEST CASE REVIEW DOCUMENT

Sl no.	Test case name	Step no.	Reviewer		Author
			Comments	Severity	
1	CB_AT_Funcnality scenarios	8	Transfer more than the balance scenario is missing	Critical	Not fixed (he will give the reason)
		Pre condition	Pre condition is missing	Critical	Fixed
		Header	Test case type is missing	Major	Fixed

Procedure to write the test case



1. System study:

Read the requirement, understand the requirement, and if you have any queries interact with the customer, B.A.

2. Identify all possible scenarios:

- i. Identify
- ii. Brainstorming sessions
 - a. Presentation of feature: Improved product knowledge
 - b. Present scenarios: Missing, Wrong and Repeated scenarios
- iii. Measure the efficiency of Brain storming session

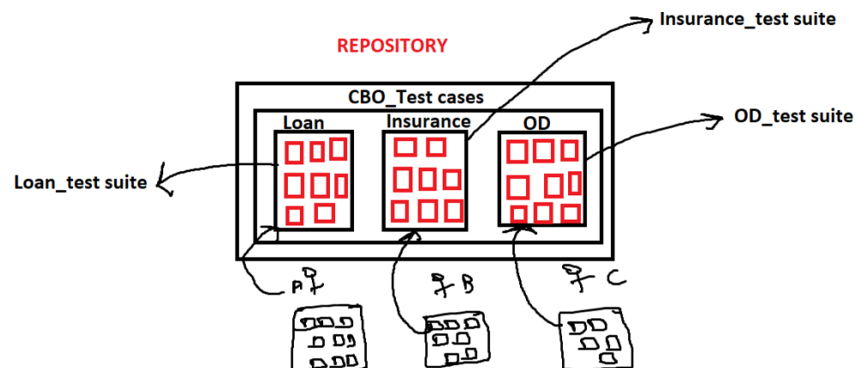
3. Write the test cases:

- Document the identified scenarios
- Group all related scenarios.
- Prioritize the scenarios within each group.
- Apply test case design technique.
- Use standard test case format

4. Store in test case repository:

Test case repository: It is a centralized place where all the test cases are stored.

Test Suite: Collection/ group of related test cases



Q.1 Where do you write the test case?

1. In Excel sheet
2. In Word file
3. In a test case management tool

Ex: Test link, ALM

Q.2 Where do you store the test cases?

1. In a shared folder
2. In a test management tool Ex: Test link, ALM
3. In a version control tool Ex: CVS, Subversion

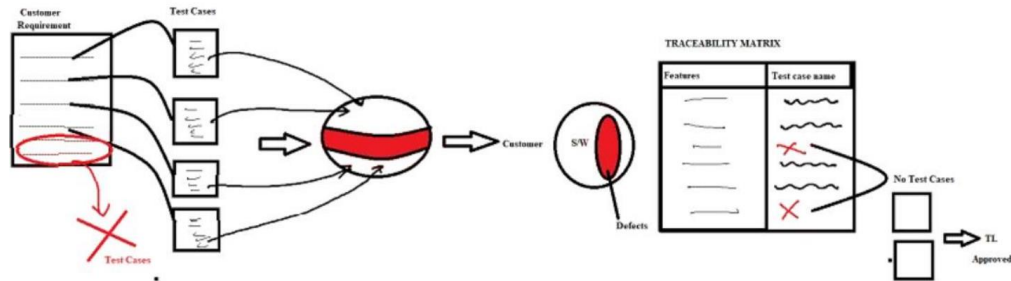
Q.3 How do you ensure that your test coverage is good? Or How do you convince you customer / management/developer that you have tested everything?

My test coverage is good because my test cases are good. My test cases are good because I have followed a strict procedure Strict procedure means:

1. I did a thorough system study because of which I had a deeper knowledge of the product because of that I was able to find more numbers of scenarios
2. I identified all possible scenarios then I did a brainstorming session because if which I was able to find more numbers of scenarios
3. While writing test case I applied test case design technique because of that there was improvement in the coverage
4. I got all my test cases reviewed and found missing scenarios because of this there was as improvement in the coverage
5. While executing test cases, I identified few new scenarios, I added them back into the test cases because of this there was as improvement in the coverage
6. Also, I performed Adhoc testing because of that there was an improvement in the coverage
7. I wrote traceability matrix and ensured that every requirement has got at least one test case because of that there was an improvement in the coverage

Traceability Matrix/ Requirement Traceability Matrix (RTM)/ Cross Reference Matrix (CRM)**What is Traceability Matrix?**

It is the document which we prepare to make sure that every requirement has at least one test case.



Sl No	Module Name	High level requirement	Detailed Requirement	Test Case Name	Automation
1.	Loan	1.1 Personal Loan	1.1.1 ----- 1.1.2 -----	CB_PL_Approval CB_PL_Rejection	CB_PL_Approval CB_PL_Rejection
		1.2 Home Loan	1.2.1 ----- 1.2.2 -----	CB_HL_Approval CB_HL_Rejection	No No
2.	Amount Transfer	2.1 FAN text field	2.1.1 should accept only 10 digit numbers 2.1.2 should accept only those numbers which is created by manager.		No No
		2.2 TAN text field	2.2.1 ----- 2.2.2 -----		Yes Yes
		2.3 Amount text field	2.3.1 should accept only positive integer b/w 100-5000		

Advantages:

1. It ensures that every requirement has got at least one test case, which indirectly assures you that you are testing every feature at least once
2. It gives traceability from high level requirement till automation script name

Drawback

1. It will not ensure that you that got 100% coverage

Types of Traceability Matrix:

There are 3 types of traceability Matrix.

1. **Forward Traceability Matrix:** Mapping from the root document to derived document is called forward traceability matrix.
Ex: Mapping from Req to test case and test case to automation script
2. **Backward Traceability Matrix:** Mapping from derived document to root document is called as Backward Traceability Matrix.
Ex: Mapping from test scripts to test cases and test cases to requirement.
3. **Bi-Directional traceability Matrix:** Doing both forward and backward traceability matrix is called as Bi-directional traceability Matrix

Interview Questions:**Q.1 Difference between Traceability Matrix and Test case review:**

<i>Traceability Matrix</i>	<i>Test case review</i>
Here we check every req has got at least one test case.	Here we check test case is covering all possible scenarios for specific requirement.
Here we don't check whether test case is covering all possible scenarios for a specific requirement.	Here we don't check whether every requirement has got at least one test case.

Q.2 If there is no requirement then how will you prepare traceability matrix?

I will explore the application and understand it, identify the scenarios, document it, write test cases. Once after writing the test cases what is the guarantee that you have written all the test case for all the features, to ensure that we write traceability matrix Here we list all the features that are there in the product and map it to test case.

<u>Features</u>	<u>Test case name</u>
Login	-----
Signup	-----
Forgot password	No

Q.3 Assume that last day when you are about to release the product to the customer, if there is a critical / blocker defect what will you do? Will you release product to the customer or not?

As a TE I'm not a decision maker so I will list all the defects which are pending and the impact of the defect on the business also I suggest whether to release or not and send it to PM. It is the manager who should take a calculated decision and decide

Q.4 What are the levels of software testing?

1. Unit testing
2. Functionality Testing
3. Integration Testing
4. System Testing
5. Acceptance Testing

Q.5 What is the difference b/w Verification and Validation?

VERIFICATION	VALIDATION
1)It involves review,walkthrough & inspection 2)It involves different activities like a)Requirement review b)Design review c)Code review d)Test Case review e)Test Plan review 3)Here we check whether are we building right product	1)It involves actual testing 2)It involves different activities like a)F.T b)I.T c)System Testing d)Compatibility e)Smoke f)Adhoc 3)Here we check whether are we building Product right

Q.6 What is the difference b/w Static and Dynamic testing?

STATIC	DYNAMIC
1)It is a verification process 2)It involves different activities like a)Requirement review b)Design review c)Code review d)Test Case review e)Test Plan review 3)To do this we need not execute a program 4)We do this to prevent defects 5)To do this we should have checklist	1)It is a validation Process 2)It involves different activities like a)F.T b)I.T c)System Testing d)Compatibility e)Smoke f)Adhoc 3)To do this we should run the program 4)We do this to find defects 5)To do this we should have test cases

Q.7 What is the difference b/w Quality Assurance(QA) and Quality Check(QC)?

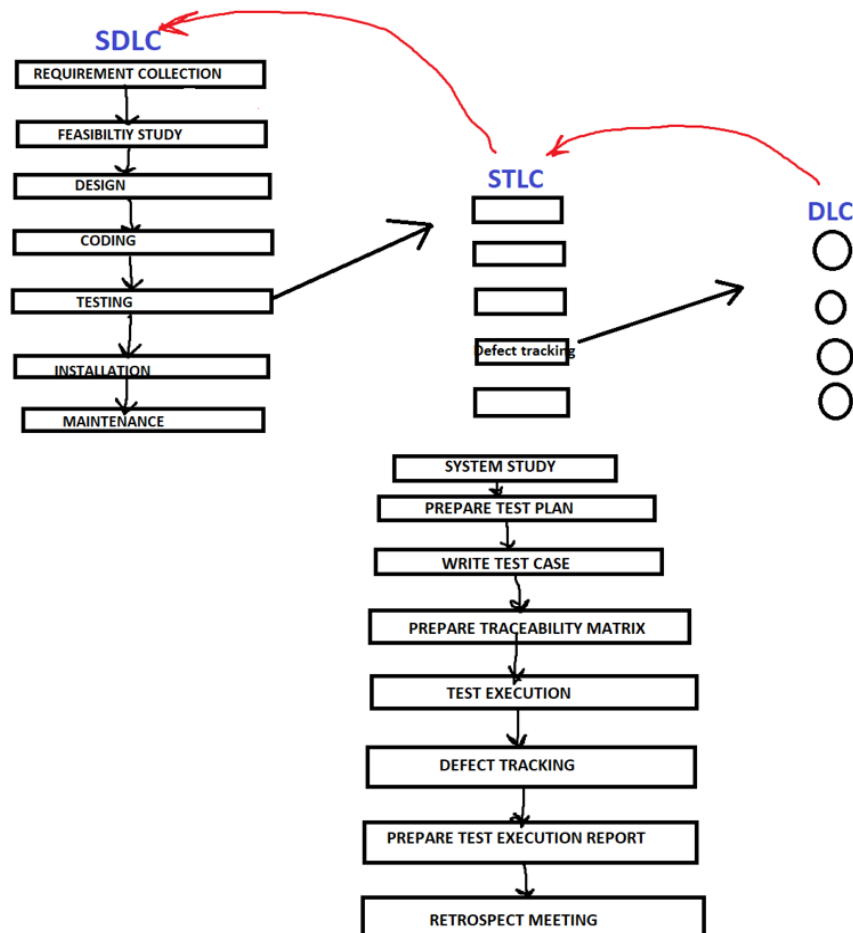
QUALITY ANALYST	QUALITY CHECK
1)It is process oriented 2)It involves different stages like a)Identify the need for process b)Define Process c)Make sure that people implement the process d)Verify the process & find the mistakes	1)It is product oriented 2)It involves actual testing wherein we check whether product works according to requirement specification

Q.8 What is the difference b/w Functional and Non-Functional Testing?

Functional testing	Non- Functional testing
1.Here we check the application works according to functional requirement specification 2.Here we check whether feature is working or not 3.It involves different types of testing like FT, IT, ST, smoke, adhoc, compatibility	1.Here we check the application works according to non-functional req specification 2.Here we check whether look, performance I good or not 3.It involves different types of testing like performance----- load testing, stress testing, volume testing, soak testing, comparison testing, usability testing

SOFTWARE TEST LIFE CYCLE (STLC)

- STLC is a procedure to test the software.
- STLC is a part of SDLC
- Defect life cycle is a part of STLC



1. System Study:

Read the requirement, understand the requirement if you have any queries, interact with BA, developers or customer.

2. Prepare Test plan:

Once after reading and understanding the requirement, we go for preparing the test plan.

Test plan is a document which drives all the future testing activities:

- a. Here we decide how many engineers we require to complete the testing.
- b. What is the total time for completing the testing/project.
- c. What each engineer should do in different stages of testing.
- d. What are types of testing we will conduct in future.
- e. What are the features that are to be tested and not to be tested
- f. What is the testing approach
- g. When each activity should start & end.

3. Write test cases:

Test case is a document which contains all possible scenarios. This stage itself has got different activities like:

- i. Identify all possible scenarios
- ii. Write test case
- iii. Review test case
- iv. Fix the review comments
- v. Verify the fix
- vi. Test case approval
- vii. Store the test case in test case repository

4. Prepare traceability matrix:

Once after we write test case, the biggest question is what is the proof that each and every requirement has got a test case?

We prepare traceability matrix to ensure that each and every requirement has got at least one test case.

Requirement no.	Test case name
1	CB_Signup_FT
2	CB_Login_FT
3	CB_AT_FT
4	CB_AB_FT
5	
6	CB_Loans_FT

5. **Test Execution:** This is the stage wherein we test the product for 4-8 cycles (if it is agile) and 30-60 cycles (if it is traditional model).

- This is the stage where we execute all the test cases.
- This is where we conduct all types of testing
- This is where we catch the bugs and help the developers to improve the quality of the product. This is where TE are productive to the organization
- This is the stage where the T.E spends a lot of time
- This is the stage where test engineers become productive to the organization

6. Defect tracking:

Once after test execution, obviously we are going to find the defects. Each defect that we find should be tracked in an organized way. This is called as Defect tracking.

7. Test execution report:

At the end of every test cycle we prepare test execution report. It is a document which we prepare and provide to the customer at the end of every test cycle.

This report covers:

- Total no. of test cases
- Total no. of test cases executed.
- Total no. of test cases not executed.
- No of test cases passed.
- No of test cases failed
- What is the pass percentage?
- What is the fail percentage?

We will prepare this document and send it to the customer. From customer's POV this is the end of the project. But from company's POV we have one more activity called as Retrospective meeting.

8. Retrospective meeting/Project closure meeting/Post-mortem meeting:

Here the entire team will sit together and discuss about the list of achievements and mistakes, they document all this, that document is called as Retrospect document. In the next release/sprint they open this document in the planning stage and plan in such a way that all the achievements are adopted and all the mistakes are avoided.

